

Contents

1	Activity Diagram - Accept/Reject Incoming Jobs	8
1.1	Overview	8
1.2	Swim Lanes	8
1.3	Process Flow	8
1.3.1	Phase 1: Accept Job	9
1.3.2	Phase 2: Ship Job	9
1.3.3	Phase 3: Deliver Job	9
1.4	Decision Points Summary	9
1.5	Key Implementation Details	9
2	Activity Diagram - Admin Model Moderation	10
2.1	Overview	10
2.2	Swim Lanes	10
2.3	Process Flow	11
2.4	Decision Points Summary	11
2.5	Notes	11
3	Activity Diagram - Admin User Moderation	11
3.1	Overview	11
3.2	Swim Lanes	12
3.3	Process Flow	12
3.4	Decision Points Summary	13
3.5	Notes	13
4	Activity Diagram - Analytics	13
4.1	Overview	14
4.2	Swim Lanes	14
4.3	Process Flow	14
4.4	Decision Points Summary	14
4.5	Notes	14
5	Activity Diagram - Audit Admin Actions	15
5.1	Overview	15
5.2	Swim Lanes	16
5.3	Process Flow	16
5.4	Decision Points Summary	16
5.5	Key Implementation Details	16
6	Activity Diagram - View Best Model of the Week	16
6.1	Overview	17
6.2	Swim Lanes	17
6.3	Process Flow	17
6.4	Decision Points Summary	18
6.5	Key Implementation Details	18
7	Activity Diagram - Comment on a Post	18
7.1	Overview	18
7.2	Swim Lanes	18
7.3	Process Flow	18
7.4	Decision Points Summary	20
7.5	Key Implementation Details	20
8	Activity Diagram - Complete Profile	20

8.1	Overview	20
8.2	Swim Lanes	20
8.3	Process Flow	20
8.4	Decision Points Summary	22
8.5	Notes	22
9	Activity Diagram - Configure Print Materials	22
9.1	Overview	22
9.2	Swim Lanes	22
9.3	Process Flow	24
9.4	Decision Points Summary	24
9.5	Key Implementation Details	24
10	Activity Diagram - Create Curated Wishlist/Collection	24
10.1	Overview	24
10.2	Swim Lanes	25
10.3	Process Flow	25
10.4	Decision Points Summary	25
10.5	Key Implementation Details	26
11	Activity Diagram - Define Licenses & Pricings	26
11.1	Overview	26
11.2	Swim Lanes	26
11.3	Process Flow	26
11.4	Decision Points Summary	28
11.5	Key Implementation Details	28
12	Activity Diagram - Define Print Capability	28
12.1	Overview	28
12.2	Swim Lanes	28
12.3	Process Flow	28
12.4	Decision Points Summary	30
12.5	Key Implementation Details	30
13	Activity Diagram - Dexie Picks	30
13.1	Overview	30
13.2	Swim Lanes	30
13.3	Process Flow	31
13.4	Decision Points Summary	31
13.5	Key Implementation Details	31
14	Activity Diagram - Dexie Tagline	33
14.1	Overview	33
14.2	Swim Lanes	33
14.3	Process Flow	33
14.4	Decision Points Summary	35
14.5	Key Implementation Details	35
15	Activity Diagram - Enable 2FA Security	35
15.1	Overview	35
15.2	Swim Lanes	35
15.3	Process Flow	37
15.4	Decision Points Summary	37
15.5	Key Implementation Details	37

16 Activity Diagram - Follow Toggle	37
16.1 Overview	37
16.2 Swim Lanes	37
16.3 Process Flow	38
16.4 Decision Points Summary	38
16.5 Key Implementation Details	38
17 Activity Diagram - Google OAuth	39
17.1 Overview	39
17.2 Swim Lanes	39
17.3 Process Flow	39
17.4 Decision Points Summary	41
17.5 Notes	41
18 Activity Diagram - Local Login	41
18.1 Overview	41
18.2 Swim Lanes	43
18.3 Process Flow	43
18.4 Decision Points Summary	43
18.5 Relationship to Sequence Diagram	43
18.6 Notes	43
19 Activity Diagram - Manage User Profile	44
19.1 Overview	44
19.2 Swim Lanes	45
19.3 Process Flow	45
19.4 Decision Points Summary	45
19.5 Key Implementation Details	45
20 Activity Diagram - Model Upload	46
20.1 Overview	46
20.2 Swim Lanes	46
20.3 Process Flow	46
20.4 Decision Points Summary	47
20.5 Notes	47
21 Activity Diagram - Notifications	47
21.1 Overview	47
21.2 Swim Lanes	47
21.3 Process Flow	48
21.4 Decision Points Summary	48
21.5 Key Implementation Details	48
22 Activity Diagram - Post Create	48
22.1 Overview	49
22.2 Swim Lanes	49
22.3 Process Flow	49
22.4 Decision Points Summary	51
22.5 Key Implementation Details	51
23 Activity Diagram - Proceed to Checkout	51
23.1 Overview	51
23.2 Swim Lanes	51
23.3 Process Flow	53
23.4 Decision Points Summary	53

23.5 Key Implementation Details	53
24 Activity Diagram - Proceed with Payment	53
24.1 Overview	53
24.2 Swim Lanes	54
24.3 Process Flow	54
24.4 Decision Points Summary	55
24.5 Key Implementation Details	55
25 Activity Diagram - Rate Counterparty	55
25.1 Overview	55
25.2 Swim Lanes	57
25.3 Process Flow	57
25.3.1 Phase 1: Eligibility Check	57
25.3.2 Phase 2: Review Submission	57
25.4 Decision Points Summary	57
25.5 Key Implementation Details	58
26 Activity Diagram - User Registration	58
26.1 Overview	58
26.2 Swim Lanes	59
26.3 Process Flow	59
26.4 Decision Points Summary	59
26.5 Notes	59
27 Activity Diagram - Report Content/User	59
27.1 Overview	61
27.2 Swim Lanes	61
27.3 Process Flow	61
27.4 Decision Points Summary	61
27.5 Key Implementation Details	61
28 Activity Diagram - Request Earnings Payout	62
28.1 Overview	62
28.2 Swim Lanes	63
28.3 Process Flow	63
28.4 Decision Points Summary	63
28.5 Key Implementation Details	63
29 Activity Diagram - Request Print	63
29.1 Overview	65
29.2 Swim Lanes	65
29.3 Process Flow	65
29.4 Decision Points Summary	65
29.5 Key Implementation Details	66
30 Activity Diagram - Resolve Abuse Reports	66
30.1 Overview	66
30.2 Swim Lanes	66
30.3 Process Flow	67
30.4 Decision Points Summary	67
30.5 Notes	67
31 Activity Diagram - Search Models & Filters	68
31.1 Overview	68

31.2 Swim Lanes	68
31.3 Process Flow	69
31.4 Decision Points Summary	69
31.5 Notes	69
32 Activity Diagram - Stats Aggregation	69
32.1 Overview	70
32.2 Swim Lanes	70
32.3 Process Flow	70
32.4 Decision Points Summary	71
32.5 Notes	71
33 Activity Diagram - Submit Review	71
33.1 Overview	71
33.2 Swim Lanes	71
33.3 Process Flow	73
33.4 Decision Points Summary	73
33.5 Key Implementation Details	73
34 Activity Diagram - Submit Role Application	73
34.1 Overview	73
34.2 Swim Lanes	73
34.3 Process Flow	74
34.4 Decision Points Summary	75
34.5 Key Implementation Details	75
35 Activity Diagram - Suspend Malicious Nodes	75
35.1 Overview	76
35.2 Swim Lanes	76
35.3 Process Flow	76
35.4 Decision Points Summary	76
35.5 Key Implementation Details	76
36 Activity Diagram - Sync Social Platform	77
36.1 Overview	77
36.2 Swim Lanes	77
36.3 Process Flow	78
36.4 Decision Points Summary	78
36.5 Key Implementation Details	78
37 Activity Diagram - Toggle Dexie On/Off	78
37.1 Overview	78
37.2 Swim Lanes	78
37.3 Process Flow	80
37.4 Decision Points Summary	80
37.5 Key Implementation Details	80
38 Activity Diagram - Toggle Content Visibility (NSFW)	80
38.1 Overview	82
38.2 Swim Lanes	82
38.3 Process Flow	82
38.4 Decision Points Summary	82
38.5 Key Implementation Details	82
39 Activity Diagram - Unlock Downloads	82

39.1 Overview	83
39.2 Swim Lanes	83
39.3 Process Flow	83
39.4 Decision Points Summary	84
39.5 Key Implementation Details	84
40 Activity Diagram - Update Shipping Address	84
40.1 Overview	84
40.2 Swim Lanes	84
40.3 Process Flow	86
40.4 Decision Points Summary	86
40.5 Key Implementation Details	86
41 Activity Diagram - Update Tracking Info	86
41.1 Overview	86
41.2 Swim Lanes	86
41.3 Process Flow	87
41.4 Decision Points Summary	87
41.5 Key Implementation Details	88
42 Activity Diagram - Verify Licensing & Terms	88
42.1 Overview	88
42.2 Swim Lanes	88
42.3 Process Flow	89
42.4 Decision Points Summary	89
42.5 Key Implementation Details	89
43 Activity Diagram - View Cart	89
43.1 Overview	89
43.2 Swim Lanes	89
43.3 Process Flow	90
43.4 Decision Points Summary	90
43.5 Key Implementation Details	91
44 Activity Diagram - View Delivery Status	91
44.1 Overview	91
44.2 Swim Lanes	91
44.3 Process Flow	91
44.4 Decision Points Summary	93
44.5 Key Implementation Details	93
45 Activity Diagram - View Model Details	93
45.1 Overview	93
45.2 Swim Lanes	93
45.3 Process Flow	93
45.4 Decision Points Summary	95
45.5 Key Implementation Details	95
46 Activity Diagram - View Post	95
46.1 Overview	95
46.2 Swim Lanes	95
46.3 Process Flow	97
46.4 Decision Points Summary	97
46.5 Key Implementation Details	97

47 Activity Diagram - View User Profile	97
47.1 Overview	97
47.2 Swim Lanes	99
47.3 Process Flow	99
47.4 Decision Points Summary	99
47.5 Key Implementation Details	99

1 Activity Diagram - Accept/Reject Incoming Jobs

UML Type: Activity Diagram Category: Print Source: docs/diagrams/activity/print/activity_accept_reje

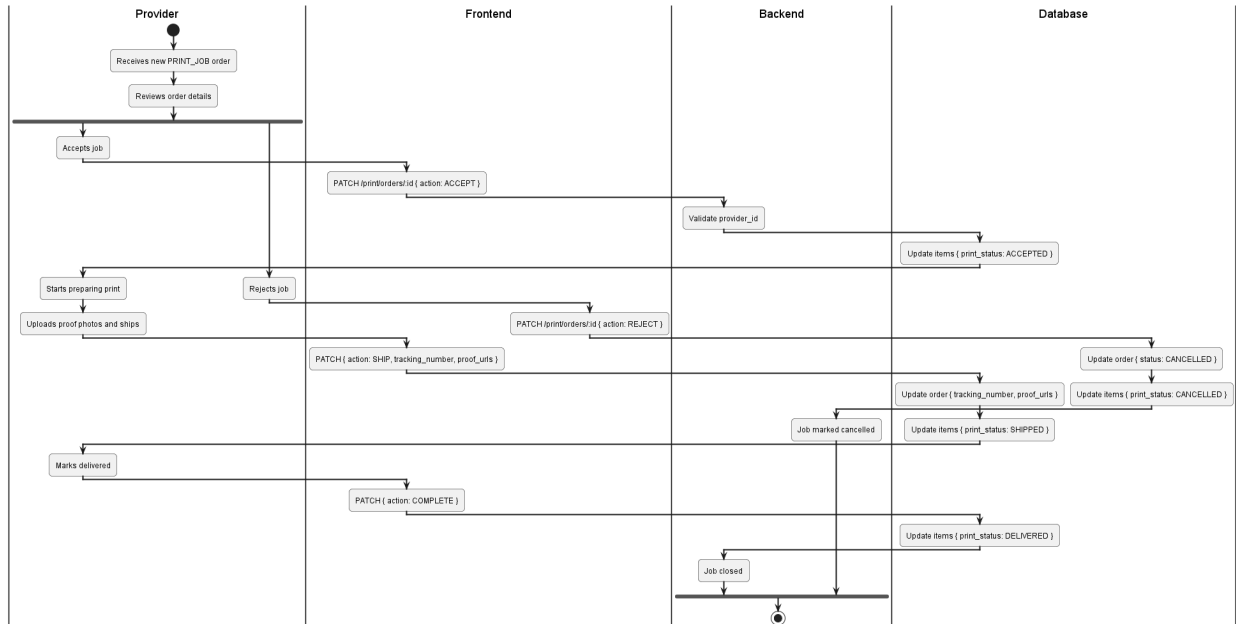


Figure 1: Activity Diagram - Accept/Reject Incoming Jobs

1.1 Overview

This activity diagram details the operational flow of print order fulfillment from the provider's side. It covers three major phases: acceptance, shipment tracking registration, and final completion.

1.2 Swim Lanes

Lane	Responsibility
Provider	Decides actions, processes raw prints, ships physical package
Frontend	Collects shipping inputs, displays status updates, sends API requests
Backend	Verifies job ownership and routes status transitions
Database	Updates order and item print statuses

1.3 Process Flow

1. **[Provider]** Opens the print job management dashboard.
2. **[Frontend]** Dispatches request for the provider's active list (GET /print/jobs).
3. **[Database]** Queries orders: `findMany({ provider_id, type: PRINT_JOB })`.
4. **[Backend]** Returns job cards list.

Fulfillment Execution Phases (Sequential Transitions):

1.3.1 Phase 1: Accept Job

1. [Provider] Reviews specifications and clicks “Accept Job”.
 2. [Frontend] Dispatches `POST /print/orders/:id/accept`.
 3. [Backend] Verifies the requesting user is the assigned `provider_id`.
 - If mismatch: returns 403. Flow ends.
 4. [Database] Updates `Order_Item.print_status = ACCEPTED` for all models in the order.
 5. [Backend] Returns 200 OK.
-

1.3.2 Phase 2: Ship Job

1. [Provider] Finishes 3D printing the model, packages the item, drops it at the courier, and clicks “Ship Job” in the dashboard.
 2. [Frontend] Prompts the provider to input the `tracking_number` and attach proof of shipping photos.
 3. [Provider] Inputs tracking code, uploads photos to S3, and clicks confirm.
 4. [Frontend] Dispatches `POST /print/orders/:id/ship` containing `{ tracking_number, proof_urls }`.
 5. [Database] Updates `Order.tracking_number = tracking_number`, appends `proof_urls` array, and updates `Order_Item.print_status = SHIPPED`.
 6. [Backend] Returns 200 OK.
-

1.3.3 Phase 3: Deliver Job

1. [Provider] Receives delivery notification or clicks “Mark Completed” once delivery is verified.
 2. [Frontend] Dispatches `POST /print/orders/:id/complete`.
 3. [Database] Updates `Order_Item.print_status = DELIVERED`.
 4. [Backend] Returns 200 OK.
 5. [Frontend] Updates order visual state to show complete/closed status.
-

1.4 Decision Points Summary

Decision	Path/Condition	Outcome
Owner check?	Mismatch	Return 403 Forbidden, stop
Action type?	Accept	Update Item print status to ACCEPTED
Action type?	Ship	Update Order tracking parameters; set status to SHIPPED
Action type?	Complete	Update Item print status to DELIVERED

1.5 Key Implementation Details

- **Granular Status Tracking:** Unlike digital purchases where order status transitions directly from PENDING to PAID, physical print jobs require granular item tracking. Thus, the database keeps

print_status directly on the Order_Item table (with states PENDING, ACCEPTED, SHIPPED, DELIVERED).

- **Security Check:** All three endpoints verify that the acting user is the registered provider_id of the target order before processing changes.
- **Proof Arrays:** The proof_urls are stored as an array of S3 strings in the database, showing photos of the printed model before shipment to protect providers against disputes.

2 Activity Diagram - Admin Model Moderation

UML Type: Activity Diagram Category: Admin Source: docs/diagrams/activity/admin/activity_admin_model

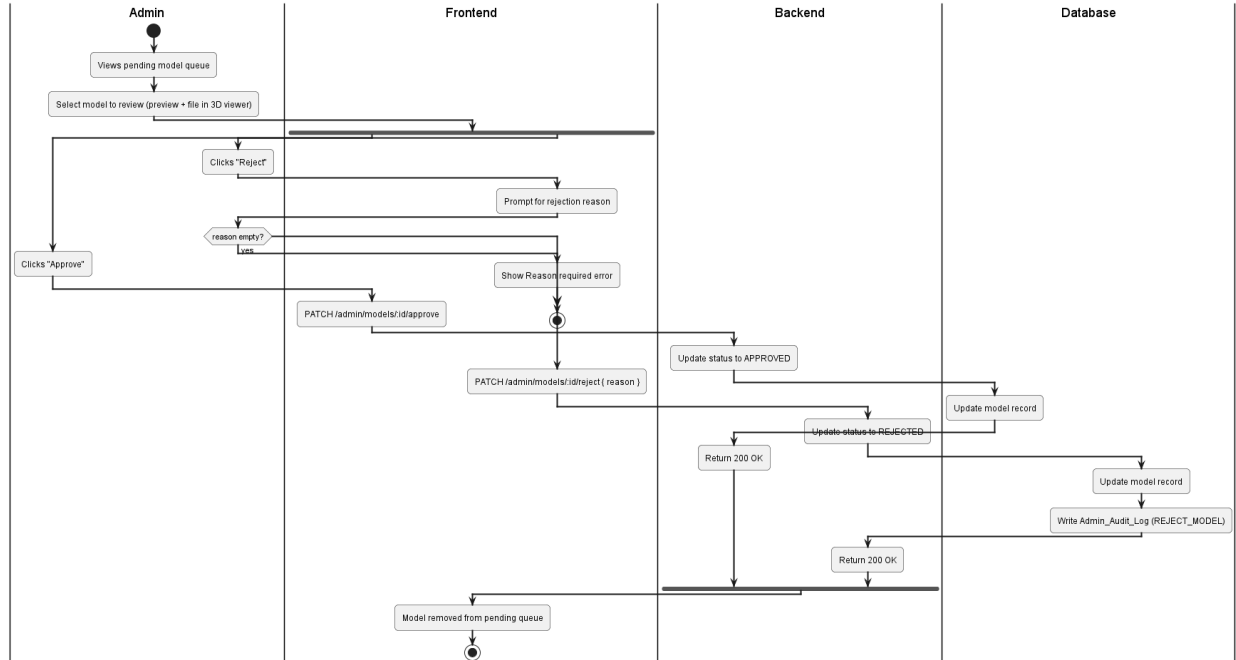


Figure 2: Activity Diagram - Admin Model Moderation

2.1 Overview

This activity diagram shows the model moderation process using a parallel fork to represent the two mutually exclusive admin decisions: approve or reject.

2.2 Swim Lanes

Lane	Responsibility
Admin	Navigates the review queue and makes decisions
Frontend	Renders the queue, prompts for reasons, sends requests
Backend	Updates model status and writes audit logs
Database	Persists changes

2.3 Process Flow

1. **[Admin]** Views the pending model queue on the admin dashboard.
2. **[Admin]** Selects a model to review (previews the 3D file in the viewer).
3. **[Frontend]** Displays action buttons: “Approve” and “Reject”.

Fork - Admin chooses one of:

Approve path:

4A. **[Admin]** Clicks “Approve”. 5A. **[Frontend]** Sends PATCH /admin/models/:id/approve. 6A. **[Backend]** Updates the model: `status = APPROVED`. 7A. **[Database]** Updates model record. 8A. **[Backend]** Returns 200 OK.

Reject path:

4B. **[Admin]** Clicks “Reject”. 5B. **[Frontend]** Prompts the Admin to enter a rejection reason. 6B. **[Admin]** Validates: if reason empty, shows error and stops. 7B. **[Frontend]** Sends PATCH /admin/models/:id/reject { reason }. 8B. **[Backend]** Updates the model: `status = REJECTED`. 9B. **[Database]** Updates model record AND writes Admin_Audit_Log (REJECT_MODEL). 10B. **[Backend]** Returns 200 OK.

Rejoin:

9. **[Frontend]** Model is removed from the pending queue.

2.4 Decision Points Summary

Decision	Yes Path	No Path
Admin chooses Approve?	Approve path	-
Admin chooses Reject?	Reject path	-
Rejection reason empty?	Show error, stop	Submit

2.5 Notes

- Approval does not write an audit log. Rejection does. This asymmetry is intentional.
- The “reason required” validation is enforced at the Frontend level (prompt) and should also be validated on the Backend before the database write.

3 Activity Diagram - Admin User Moderation

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_admin_user

3.1 Overview

This activity diagram shows the same approve/reject pattern applied to Artist and Provider account applications, with parallel fork notation for the two mutually exclusive admin decisions.

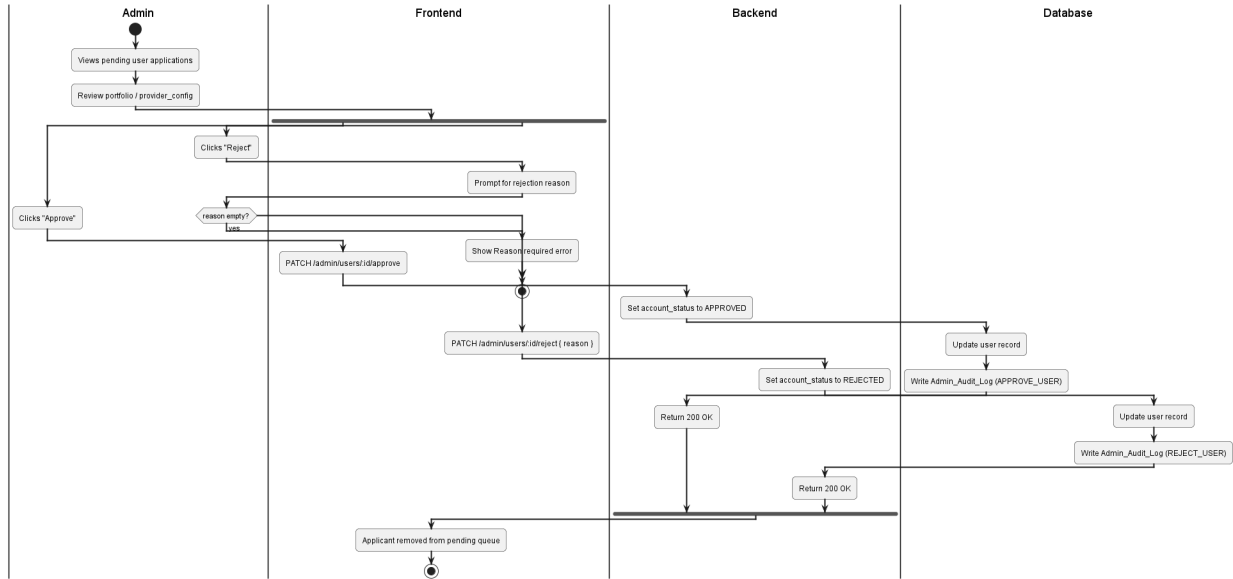


Figure 3: Activity Diagram - Admin User Moderation

3.2 Swim Lanes

Lane	Responsibility
Admin	Reviews applications and makes decisions
Frontend	Renders the application queue and sends requests
Backend	Updates user status
Database	Persists changes and writes audit logs

3.3 Process Flow

1. [Admin] Views the pending user applications list.
2. [Admin] Reviews the applicant's portfolio or `provider_config`.
3. [Frontend] Displays "Approve" and "Reject" action buttons.

Fork:

Approve path:

4A. [Admin] Clicks "Approve". 5A. [Frontend] Sends `PATCH /admin/users/:id/approve`. 6A. [Backend] Sets `account_status = APPROVED`. 7A. [Database] Updates user record AND writes `Admin_Audit_Log (APPROVE_USER)`. 8A. [Backend] Returns 200 OK.

Reject path:

4B. [Admin] Clicks "Reject". 5B. [Frontend] Prompts for a rejection reason. 6B. [Admin] If reason is empty: shows error and stops. 7B. [Frontend] Sends `PATCH /admin/users/:id/reject { reason }`. 8B. [Backend] Sets `account_status = REJECTED`. 9B. [Database] Updates user record AND writes `Admin_Audit_Log (REJECT_USER)`. 10B. [Backend] Returns 200 OK.

Rejoin:

9. [Frontend] Applicant is removed from the pending queue.

3.4 Decision Points Summary

Decision	Yes Path	No Path
Admin chooses Approve?	Approve path	-
Admin chooses Reject?	Reject path	-
Rejection reason empty?	Show error, stop	Submit

3.5 Notes

- Unlike model moderation, both approval AND rejection write audit log entries for user moderation. This reflects the higher accountability required for decisions affecting user roles.
- The `status_history` JSON array on the `User` record is also updated at each approval/rejection, providing a full history of status changes.

4 Activity Diagram - Analytics

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_analytics.

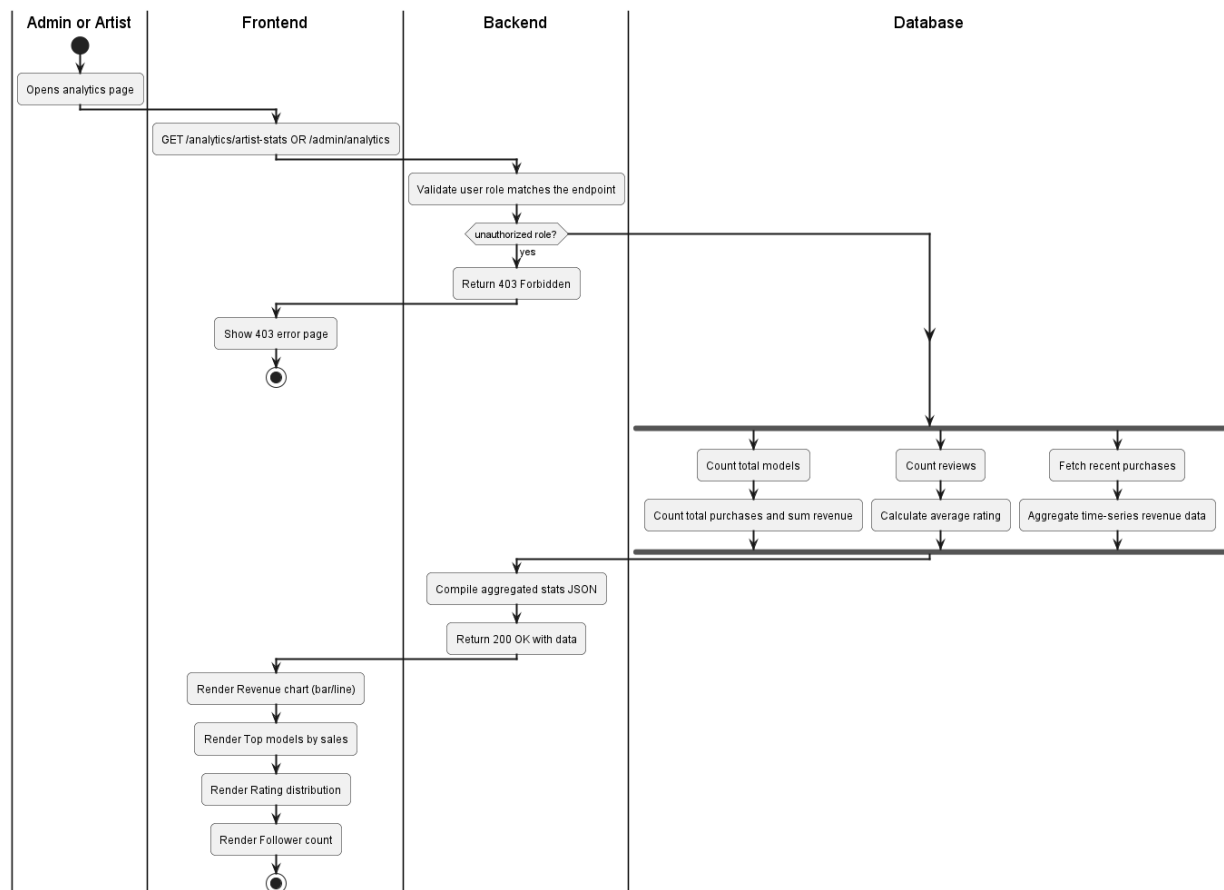


Figure 4: Activity Diagram - Analytics

4.1 Overview

This activity diagram shows how both Artists and Admins access their respective analytics dashboards. The key difference is the role-based access gate and the scope of data returned.

4.2 Swim Lanes

Lane	Responsibility
Admin or Artist	The actor requesting analytics
Frontend	Navigates to the analytics page and renders charts
Backend	Validates role and aggregates data
Database	Executes parallel aggregation queries

4.3 Process Flow

1. **[Admin or Artist]** Opens the analytics page.
2. **[Frontend]** Sends `GET /analytics/artist-stats` (for Artist) or `GET /admin/analytics` (for Admin).
3. **[Backend]** Validates the user's role matches the endpoint.
 - If unauthorized role: returns 403 Forbidden. Frontend shows error page. Flow ends.
4. **[Database]** Executes parallel aggregation queries:
 - Count total models (filtered by artist for Artist, platform-wide for Admin)
 - Count total purchases and sum revenue
 - Count reviews and calculate average rating
 - Fetch recent purchases and aggregate time-series revenue data
 - Find top-selling models
 - Find top-earning artists (Admin only)
5. **[Backend]** Compiles all results into a single aggregated stats JSON.
6. **[Backend]** Returns 200 with the data.
7. **[Frontend]** Renders:
 - Revenue bar/line chart
 - Top models by sales
 - Rating distribution
 - Follower count (Artist view)

4.4 Decision Points Summary

Decision	Yes Path	No Path
Role matches endpoint?	Continue	Return 403

4.5 Notes

- Artist analytics are scoped to that artist's own models and sales. Admin analytics are platform-wide.
- The diagram shows the Database lane with parallel fork notation, indicating that the aggregation queries should ideally be executed concurrently. In the current Prisma implementation, these are likely sequential unless wrapped in `Promise.all`.

- The Frontend renders different chart types depending on the data returned. The backend response shape is the same for both roles; the difference is in the data scope.

5 Activity Diagram - Audit Admin Actions

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_audit_acti

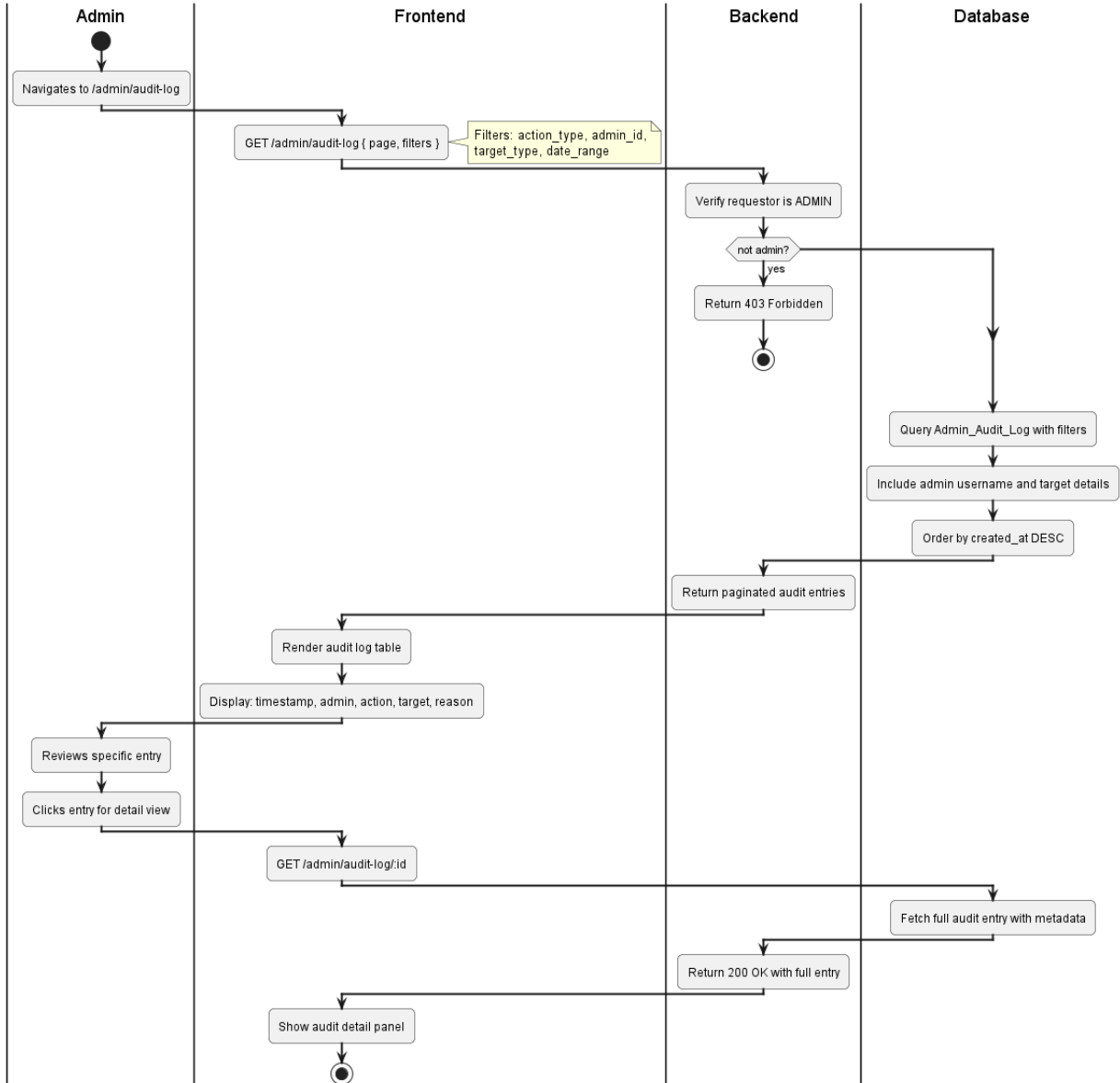


Figure 5: Activity Diagram - Audit Admin Actions

5.1 Overview

This activity diagram describes the flow for viewing the admin audit log — a record of all moderation and administrative actions taken on the platform. This corresponds to the “Audit admin actions” use case in

the UCD.

5.2 Swim Lanes

Lane	Responsibility
Admin	Navigates to and filters the audit log
Frontend	Fetches and renders audit entries
Backend	Validates admin role and returns paginated results
Database	Queries audit log with optional filters

5.3 Process Flow

1. [Admin] Navigates to `/admin/audit-log`.
 2. [Frontend] Dispatches `GET /admin/audit-log { page, filters }`.
 - Filters: `action_type`, `admin_id`, `target_type`, `date_range`.
 3. [Backend] Verifies the requestor is an ADMIN.
 - If not admin: returns 403 Forbidden.
 4. [Database] Queries `Admin_Audit_Log` with filters. Includes admin username and target details. Orders by `created_at` DESC.
 5. [Backend] Returns paginated audit entries.
 6. [Frontend] Renders audit log table showing: timestamp, admin, action, target, reason.
 7. [Admin] Reviews a specific entry and clicks it for detail view.
 8. [Frontend] Dispatches `GET /admin/audit-log/:id`.
 9. [Database] Fetches full audit entry with metadata.
 10. [Backend] Returns 200 OK with full entry.
 11. [Frontend] Shows the audit detail panel.
-

5.4 Decision Points Summary

Decision	Path/Condition	Outcome
Requestor is ADMIN?	No	Return 403 Forbidden

5.5 Key Implementation Details

- **Immutable Log:** Audit log entries are append-only. No admin (including super-admins) can delete or modify audit entries — this ensures an unalterable record of platform governance.
- **Accountability:** Each entry records the specific admin who took the action, the target, the action type, and the timestamp, providing full traceability for every moderation decision.

6 Activity Diagram - View Best Model of the Week

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_best_model

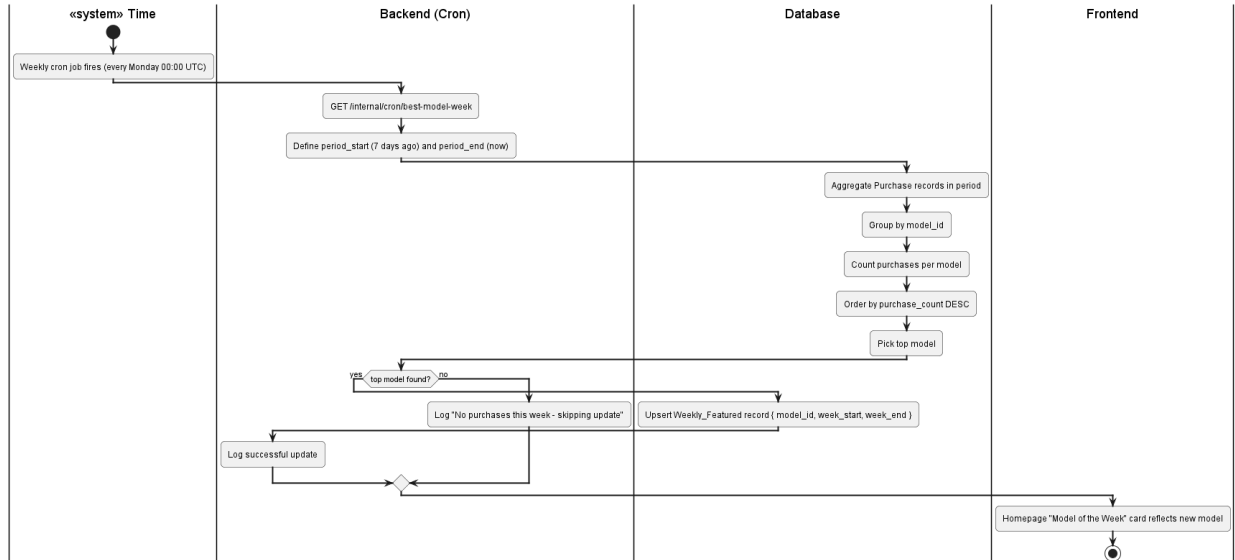


Figure 6: Activity Diagram - View Best Model of the Week

6.1 Overview

This activity diagram describes the automated weekly process of selecting and featuring the best-selling 3D model of the week. The `<<system>> Time` actor triggers the aggregation, and the result is surfaced on the homepage for Guests and Users alike.

6.2 Swim Lanes

Lane	Responsibility
<code><> Time</code>	Fires the weekly cron trigger
Backend (Cron)	Runs the aggregation and upsert logic
Database	Aggregates purchases and stores the featured model
Frontend	Renders the “Model of the Week” card on the homepage

6.3 Process Flow

1. [`<> Time`] Weekly cron job fires every Monday at 00:00 UTC.
2. [Backend (Cron)] Calls `GET /internal/cron/best-model-week`.
3. [Backend (Cron)] Defines `period_start` (7 days ago) and `period_end` (now).
4. [Database] Aggregates `Purchase` records in the period, groups by `model_id`, counts purchases per model, and orders by `purchase_count DESC`.
5. [Database] Picks the top model.
6. [Backend (Cron)] Evaluates result:
 - **Top model found:** Upserts `Weekly_Featured` record `{ model_id, week_start, week_end }`.
 - **No purchases:** Logs “No purchases this week — skipping update”.
7. [Frontend] Homepage “Model of the Week” card reflects the newly featured model.

6.4 Decision Points Summary

Decision	Path/Condition	Outcome
Top model found?	No	Log skip, no update

6.5 Key Implementation Details

- **Cron Actor:** The <<system>> Time actor in the UCD represents the automated scheduler. No human initiates this — it fires on a fixed weekly cadence.
- **Upsert:** The weekly featured record is upserted, not inserted, so there is always at most one active “Model of the Week” at any given time.
- **Guest Visibility:** This featured model is publicly visible to unauthenticated guests on the homepage, serving as a discovery and marketing hook.

7 Activity Diagram - Comment on a Post

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_comment_on

7.1 Overview

This activity diagram describes the flow for posting a comment on a community feed post. Authentication is required, and upon success a notification is sent to the post owner.

7.2 Swim Lanes

Lane	Responsibility
User	Navigates to post and submits comment
Frontend	Validates session, renders input, and submits comment
Backend	Validates content and creates comment record
Database	Persists comment and creates notification

7.3 Process Flow

1. [User] Navigates to a post and clicks “Comment”.
2. [Frontend] Validates the user is authenticated.
 - If not authenticated: redirects to /login.
3. [Frontend] Displays the comment input field.
4. [User] Types comment text and submits.
5. [Frontend] Dispatches POST /posts/:post_id/comments { content }.
6. [Backend] Validates content is non-empty and within the character limit.
 - If invalid: returns 400 Bad Request.
7. [Database] Creates Post_Comment record { post_id, user_id, content }.
8. [Backend] Returns 201 Created with comment data.
9. [Frontend] Appends the new comment to the thread.
10. [Database] Creates a Notification for the post owner.

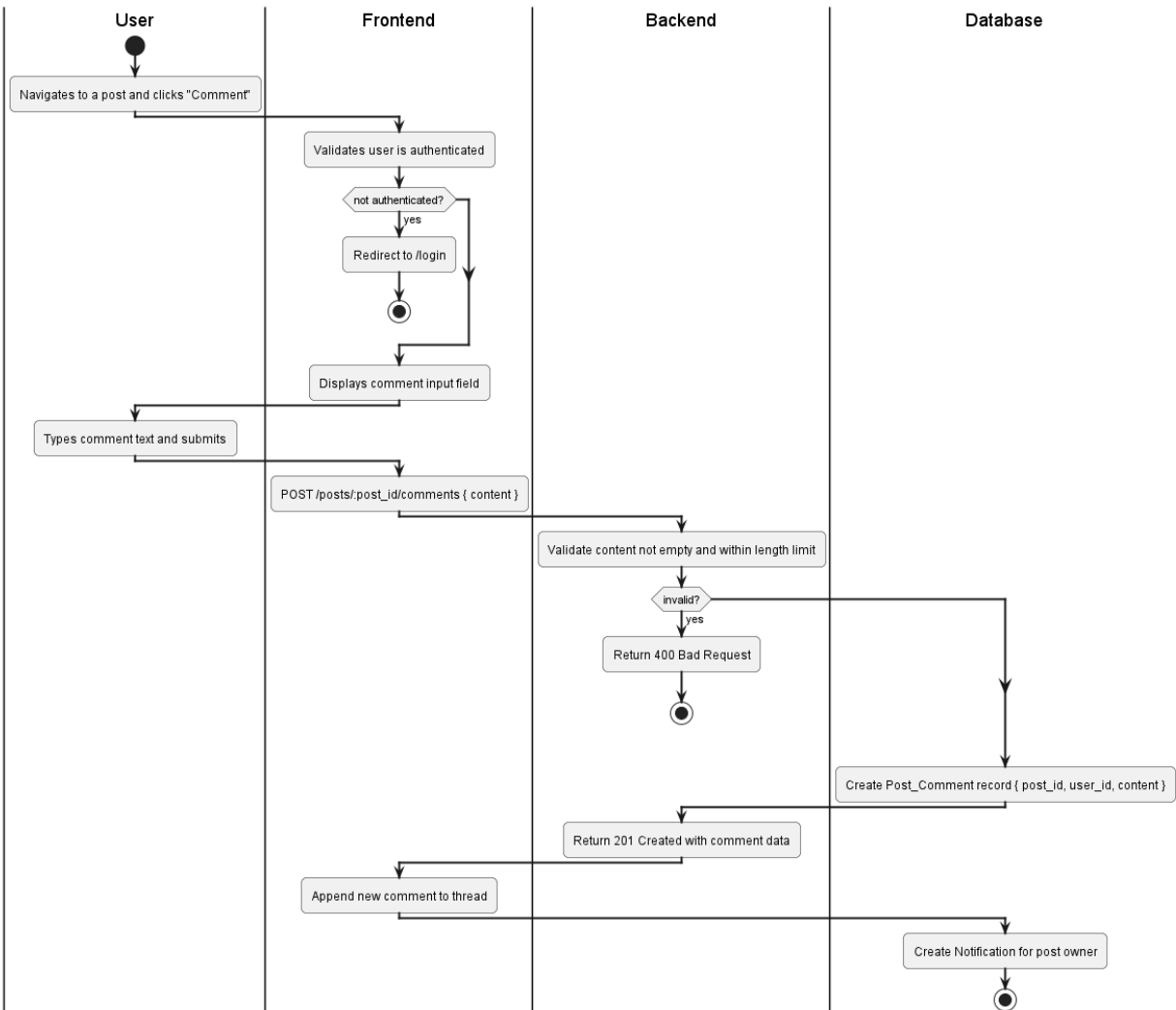


Figure 7: Activity Diagram - Comment on a Post

7.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	No	Redirect to /login
Content valid?	No	Return 400 Bad Request

7.5 Key Implementation Details

- **Auth Gate:** Comments are restricted to authenticated users only. Guests can view comments but must register to participate.
- **Notification Side Effect:** A notification is automatically created for the post owner whenever a new comment is posted, enabling real-time engagement awareness.

8 Activity Diagram - Complete Profile

UML Type: Activity Diagram **Category:** Auth **Source:** docs/diagrams/activity/auth/activity_complete_profile

8.1 Overview

This activity diagram describes the profile completion step that new Google OAuth users must complete before using the platform. After registering via Google, users are redirected here to choose a permanent username.

8.2 Swim Lanes

Lane	Responsibility
User	The new Google OAuth user
Frontend	Enforces the redirect and renders the form
Backend	Validates and saves the chosen username
Database	Checks uniqueness and persists the update

8.3 Process Flow

1. **[Frontend]** After Google OAuth login with `needs_username = true`, redirects to `/complete-profile`.
2. **[User]** Sees the “Choose a username” form.
3. **[Frontend]** Checks auth session. If not authenticated or `needs_username = false`: redirects away. Flow ends.
4. **[User]** Enters desired username.
5. **[Frontend]** Client validates:
 - Username is 3-30 characters, alphanumeric, no spaces.
 - If invalid: shows inline error. User corrects.
6. **[Frontend]** Sends `PATCH /users/complete-profile { username }`.

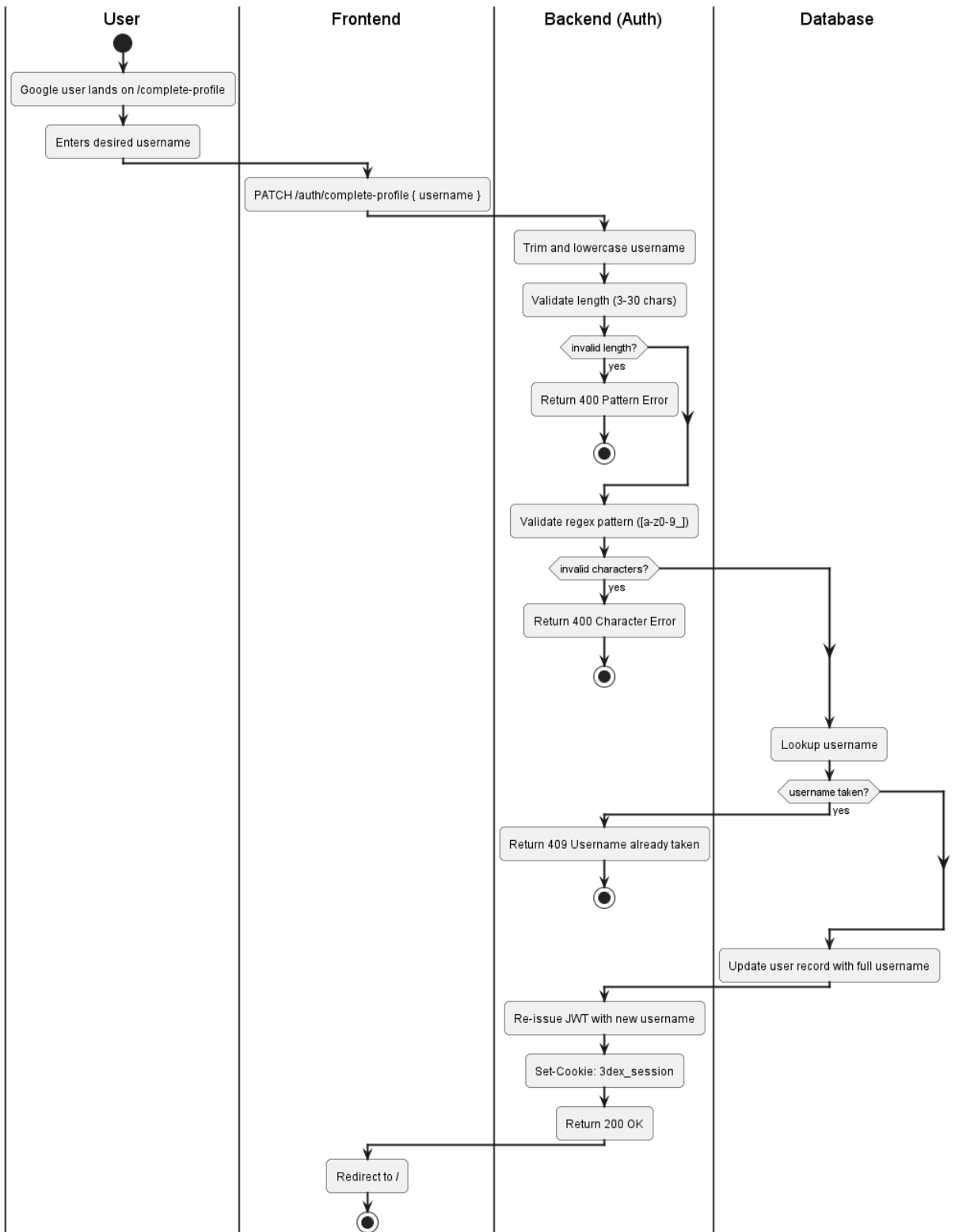


Figure 8: Activity Diagram - Complete Profile

7. **[Backend]** Validates:
 - Username is present.
 - Matches allowed pattern (3-30 chars, `^[a-zA-Z0-9_]+$`).
 - If invalid: returns 400.
8. **[Database]** Checks username uniqueness.
 - If taken (P2002): Backend returns 409 “Username already taken”.
9. **[Database]** Updates user record with the chosen username.
10. **[Backend]** Signs a fresh JWT and sets a new `3dex_session` cookie.
11. **[Backend]** Returns 200 with updated user.
12. **[Frontend]** Hydrates `AuthContext`, redirects to home. User can now use the platform.

8.4 Decision Points Summary

Decision	Yes Path	No Path
Session valid and needs_username?	Show form	Redirect away
Client validation passes?	Submit	Show error, retry
Server validation passes?	Continue	Return 400
Username unique?	Save	Return 409, retry

8.5 Notes

- This is a mandatory step for Google OAuth new users only. Local registration users choose their username during registration.
- After this step, `needs_username` becomes irrelevant since the user now has a permanent username.
- The fresh JWT issued at the end updates the cookie so subsequent `GET /auth/me` calls return the correct username.
- The page redirects away if the user already has a username, preventing repeated visits to the complete-profile form.

9 Activity Diagram - Configure Print Materials

UML Type: Activity Diagram **Category:** Print **Source:** docs/diagrams/activity/print/activity_configure_p

9.1 Overview

This activity diagram describes the flow for configuring print material options before submitting a print job order. In the UCD, “Configure print materials” is an `<<include>>` of “Proceed to checkout” for print orders.

9.2 Swim Lanes

Lane	Responsibility
User	Selects material, color, layer height, and quantity
Frontend	Fetches provider capabilities and computes cost estimate
Backend	Returns provider capability options
Database	Provides the provider configuration record

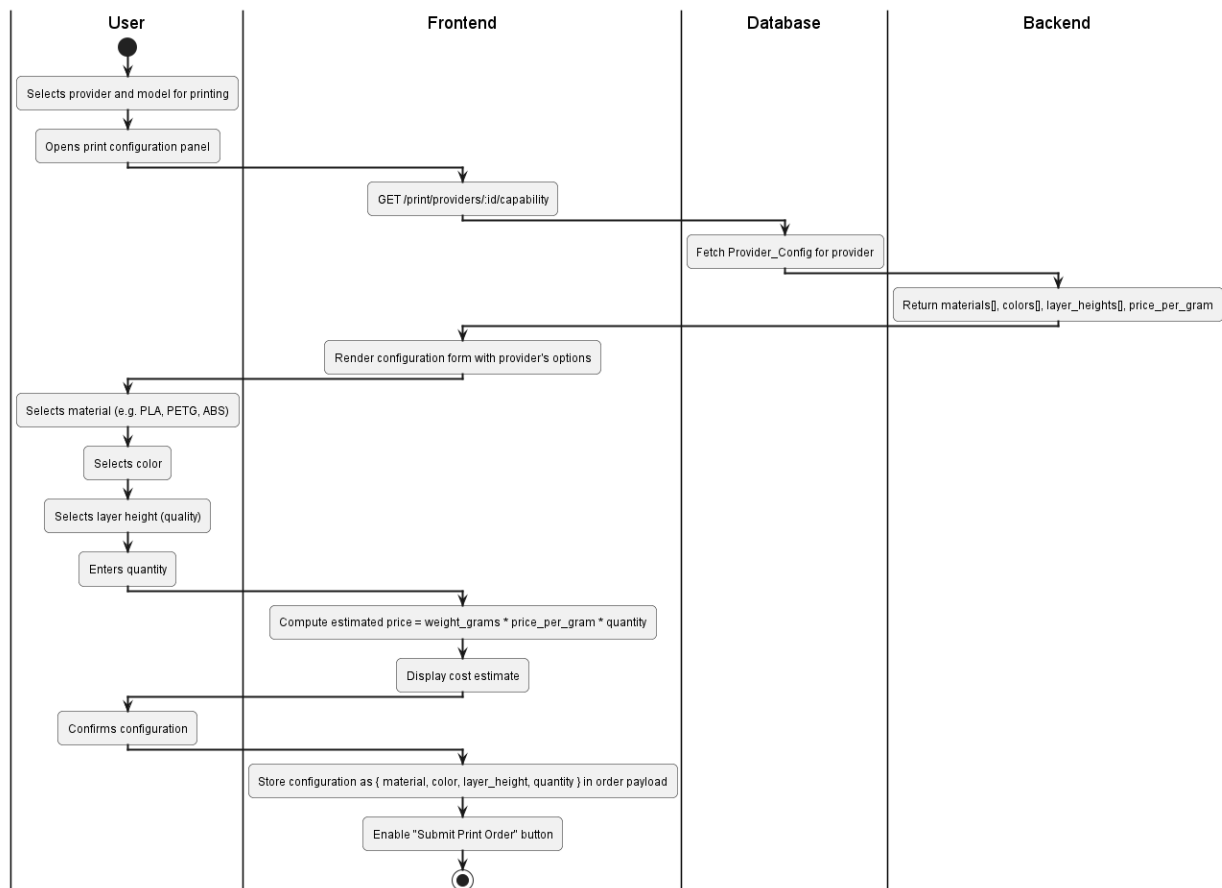


Figure 9: Activity Diagram - Configure Print Materials

9.3 Process Flow

1. [User] Selects a provider and a model for printing.
 2. [User] Opens the print configuration panel.
 3. [Frontend] Dispatches `GET /print/providers/:id/capability`.
 4. [Database] Fetches `Provider_Config` for the provider.
 5. [Backend] Returns: `materials[], colors[], layer_heights[], price_per_gram`.
 6. [Frontend] Renders the configuration form with the provider's available options.
 7. [User] Selects:
 - Material (e.g. PLA, PETG, ABS)
 - Color
 - Layer height (quality tier)
 - Quantity
 8. [Frontend] Computes estimated price: `weight_grams × price_per_gram × quantity`.
 9. [Frontend] Displays cost estimate.
 10. [User] Confirms configuration.
 11. [Frontend] Stores configuration as `{ material, color, layer_height, quantity }` in the order payload. Enables “Submit Print Order” button.
-

9.4 Decision Points Summary

Decision	Path/Condition	Outcome
<i>(selections driven by provider options only)</i>	—	—

9.5 Key Implementation Details

- **Dynamic Options:** The form is always populated from the provider's live capability config — no hardcoded options. This ensures the user only selects materials and specs the provider actually supports.
- **Cost Estimation:** The price estimate is computed client-side for instant feedback. The server revalidates the final price on order submission to prevent manipulation.
- **UCD Relationship:** This is an `<<include>>` of “Proceed to checkout” in the print flow — configuration is mandatory before a print order can be placed.

10 Activity Diagram - Create Curated Wishlist/Collection

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_crea

10.1 Overview

This activity diagram describes the dual-path logic of wishlisting a model. It covers how a single heart click checks current status and branches to either create (add) or delete (remove) the wishlist record.

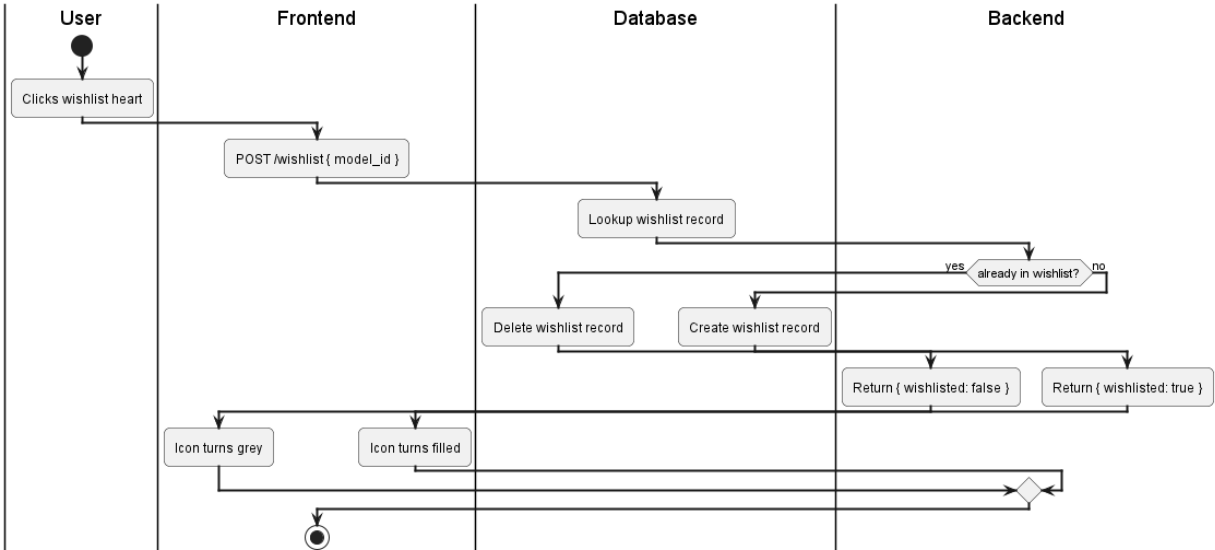


Figure 10: Activity Diagram - Create Curated Wishlist/Collection

10.2 Swim Lanes

Lane	Responsibility
User	Clicks the heart icon
Frontend	Issues the toggle request and handles optimistic UI
Backend	Verifies session and routes toggle action
Database	Performs lookup, insert, or delete operations

10.3 Process Flow

1. [User] Clicks the “Wishlist” (heart) icon on a model.
2. [Frontend] Dispatches `POST /wishlist/toggle` with `{ model_id }`.
3. [Backend] Verifies the requesting user is authenticated.
 - If not authenticated: returns 401 Unauthorized. Flow ends.
4. [Database] Queries `Wishlist` table for a record matching the `(user_id, model_id)` unique pair.
5. [Backend] Evaluates the check:

Decision Branch: Exist or Not?

- **Path A: Record Exists (Remove from Wishlist)**
 1. [Database] Deletes the `Wishlist` record.
 2. [Backend] Returns 200 OK with `{ wished: false }`.
 3. [Frontend] Changes heart icon to outline.
- **Path B: Record Does Not Exist (Add to Wishlist)**
 1. [Database] Inserts a new `Wishlist` record.
 2. [Backend] Returns 200 OK with `{ wished: true }`.
 3. [Frontend] Changes heart icon to filled.

10.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	No	Return 401, stop
Record exists?	Yes	Delete wishlist record, return wishlist: false
Record exists?	No	Create wishlist record, return wishlist: true

10.5 Key Implementation Details

- **Atomic Unique Constraint:** The database has a unique index on (`user_id`, `model_id`). Even in highly concurrent environments, this prevents duplicate entries.
- **Toggle Endpoint Pattern:** Rather than exposing separate `POST` (add) and `DELETE` (remove) endpoints, the `/wishlist/toggle` endpoint simplifies frontend state management by returning the new status as a boolean, allowing the client to adjust its styling dynamically.

11 Activity Diagram - Define Licenses & Pricings

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_define_

11.1 Overview

This activity diagram describes the process by which an Artist sets or updates the license type and pricing for their uploaded 3D model. This use case is included (`<<include>>`) as part of the “Upload 3D Models” use case in the UCD.

11.2 Swim Lanes

Lane	Responsibility
Artist	Selects license type and enters price
Frontend	Submits the license configuration
Backend	Validates ownership and input data
Database	Upserts the license record

11.3 Process Flow

1. **[Artist]** Selects a license type (FREE, PERSONAL, COMMERCIAL, EXCLUSIVE) for a model.
2. **[Frontend]** Dispatches `POST /models/:id/licenses { license_type, price }`.
3. **[Backend]** Verifies the requestor is the model owner.
 - If not owner: returns 403 Forbidden.
4. **[Backend]** Validates the `license_type` is a valid enum value.
 - If invalid: returns 400.
5. **[Backend]** Validates `price >= 0`.
 - If negative: returns 400.
6. **[Database]** Upserts the `License` record for (`model_id`, `license_type`).
7. **[Backend]** Returns 200 OK with the license details.

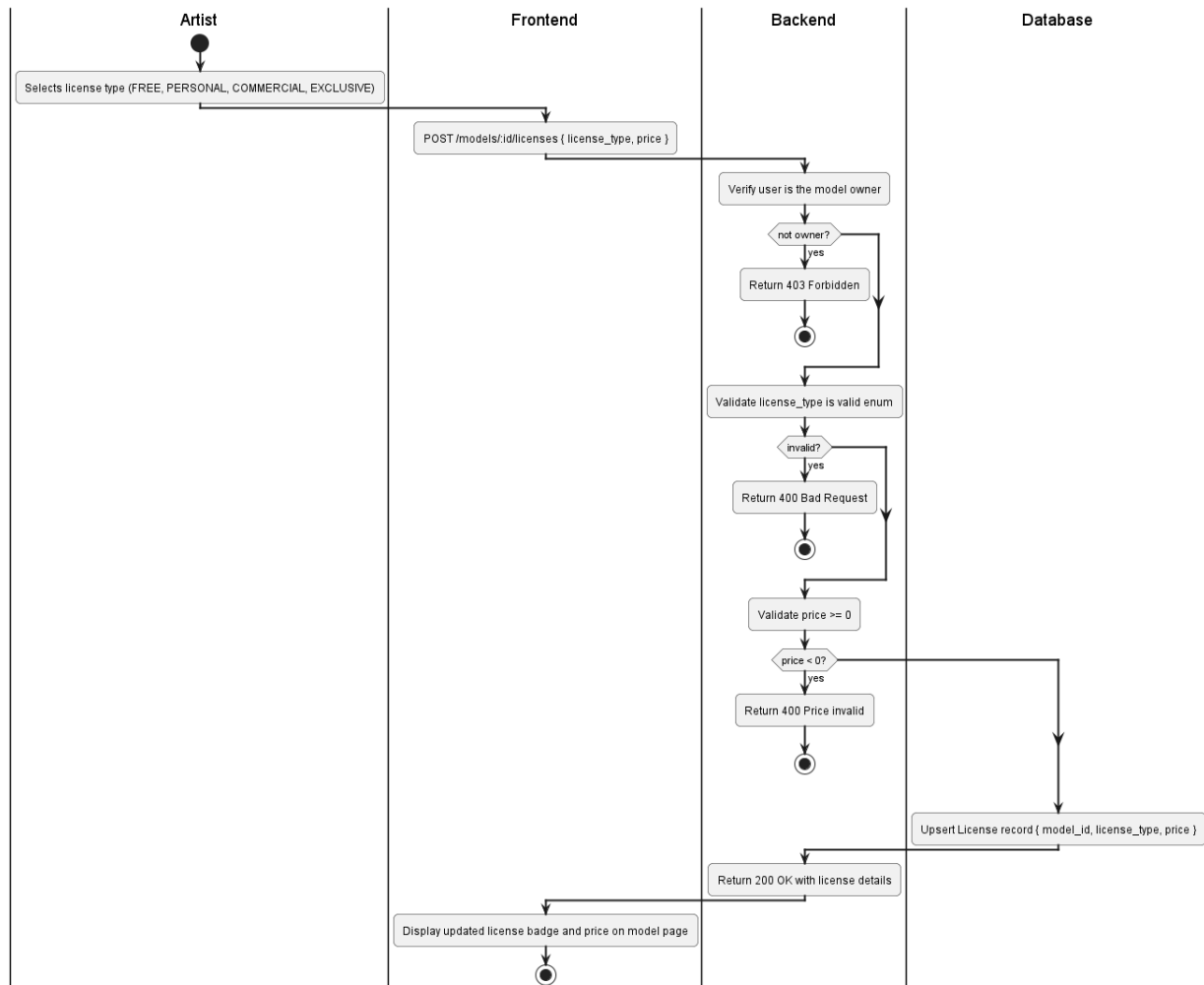


Figure 11: Activity Diagram - Define Licenses & Pricings

8. **[Frontend]** Updates the license badge and price display on the model page.

11.4 Decision Points Summary

Decision	Path/Condition	Outcome
Is owner?	No	Return 403 Forbidden
License type valid?	No	Return 400 Bad Request
Price $\geq$ 0?	No	Return 400 Invalid price

11.5 Key Implementation Details

- **License Types:** The platform supports four license tiers — FREE (no charge, open use), PERSONAL (paid, personal projects only), COMMERCIAL (paid, commercial use permitted), and EXCLUSIVE (paid, single buyer exclusivity).
- **Upsert Pattern:** Artists can update their license configuration at any time. The upsert ensures a single active license record per (`model_id`, `license_type`) combination, preventing duplicate entries.

12 Activity Diagram - Define Print Capability

UML Type: Activity Diagram **Category:** Print **Source:** docs/diagrams/activity/print/activity_define_print

12.1 Overview

This activity diagram describes the process by which a Provider configures their 3D printing capabilities on the platform. This is what enables them to appear in the print provider discovery system when users request a print job.

12.2 Swim Lanes

Lane	Responsibility
Provider	Fills in print capability details
Frontend	Submits configuration to the backend
Backend	Validates role and input data
Database	Upserts the provider configuration record

12.3 Process Flow

1. **[Provider]** Opens Provider Settings.
2. **[Provider]** Fills the print capability form:
 - Materials supported (PLA, PETG, ABS, etc.)
 - Layer heights available (quality tiers)
 - Build volume and max print size
 - Price per gram

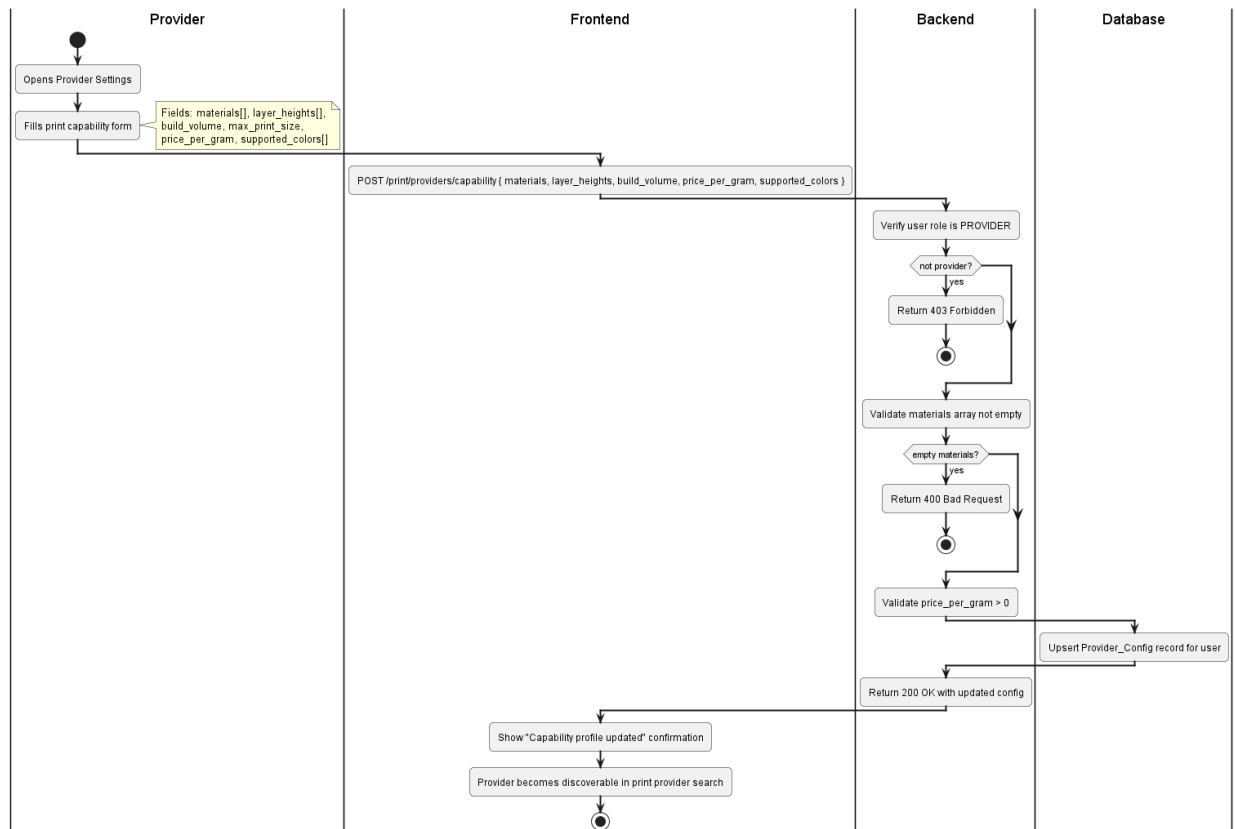


Figure 12: Activity Diagram - Define Print Capability

- Supported colors
3. **[Frontend]** Dispatches `POST /print/providers/capability { materials, layer_heights, build_volume, price_per_gram, supported_colors }`.
 4. **[Backend]** Verifies the requestor's role is `PROVIDER`.
 - If not provider: returns 403 Forbidden.
 5. **[Backend]** Validates `materials` array is not empty.
 - If empty: returns 400 Bad Request.
 6. **[Backend]** Validates `price_per_gram > 0`.
 7. **[Database]** Upserts `Provider_Config` record for the user.
 8. **[Backend]** Returns 200 OK with the updated config.
 9. **[Frontend]** Shows “Capability profile updated” confirmation. Provider becomes discoverable in print provider search.

12.4 Decision Points Summary

Decision	Path/Condition	Outcome
User is <code>PROVIDER</code> ?	No	Return 403 Forbidden
Materials non-empty?	No	Return 400 Bad Request
<code>price_per_gram > 0</code> ?	No	Validation error

12.5 Key Implementation Details

- **Discoverability:** A provider without a configured `Provider_Config` record is invisible in the print order flow. This configuration is a prerequisite for accepting print jobs.
- **Upsert Pattern:** Providers can update their capabilities at any time. The upsert ensures only one configuration record per provider exists.

13 Activity Diagram - Dexie Picks

UML Type: Activity Diagram **Category:** Dexie **Source:** docs/diagrams/activity/dexie/activity_dexie_picks

13.1 Overview

This activity diagram describes the personalized product recommendation flow powered by Dexie AI. It details the process of collecting user interactions (likes, purchases, wishlist items), generating semantic search query embeddings, and querying PostgreSQL with cosine similarity to retrieve personalized recommendations.

13.2 Swim Lanes

Lane	Responsibility
Customer	Navigates to the home page or catalog feed
Frontend	Issues recommendations retrieval API call
Backend	Retrieves taste vectors and constructs queries

Lane	Responsibility
Embedding Service	Converts interaction keywords into a continuous space vector
Database	Executes high-performance cosine similarity queries

13.3 Process Flow

1. **[Customer]** Accesses the home screen or personalized dashboard.
2. **[Frontend]** Dispatches `GET /dexie/picks?limit=8` with user headers.
3. **[Backend]** Verifies the requesting user session.
 - If user is anonymous (guest): Backend queries for the newest 8 approved models as general fallbacks and skips personal embedding paths. Flow rejoins at step 8.
4. **[Database]** Queries for the user’s recent interactions:
 - Fetches the last 5 wishlisted items (including tags and categories).
 - Fetches the last 5 purchased items (including tags and categories).
5. **[Backend]** Checks if any interaction history exists.
 - **No history found:** Queries for the newest 8 approved models as fallbacks. Flow rejoins at step 8.
 - **History exists:** Combines all tags and categories from the historical items into a single, clean text string.
6. **[Embedding Service]** Converts the compiled interaction keywords into a 384-dimensional dense vector using the `all-MiniLM-L6-v2` transformer model.
7. **[Database]** Executes a raw SQL parameter-safe query comparing the query vector against the pre-computed `model.embedding` column using PostgreSQL’s cosine distance operator (`<=>`).
8. **[Database]** Retrieves the top 8 models with the smallest cosine distance (highest semantic similarity) that have `status == APPROVED` and do not belong to the user themselves.
9. **[Backend]** Returns 200 OK along with the recommendations payload.
10. **[Frontend]** Renders the curated products in the “Dexie’s Picks” carousel.

13.4 Decision Points Summary

Decision	Path/Condition	Outcome
Logged in?	No	Load newest 8 models as general fallbacks
Taste data exists?	No	Load newest 8 models as fallbacks
Taste data exists?	Yes	Generate embedding, execute raw pgvector cosine similarity

13.5 Key Implementation Details

- **Raw SQL Cosine Similarity:** Because standard Prisma queries do not support PostgreSQL `<=>` vector distance operators, the database interaction uses the `prisma.$queryRaw` helper to execute the comparison securely.
- **Cold Start Recovery:** By checking for empty interaction arrays and automatically reverting to a chronological fallback queue, the recommendation system is safe against cold-start bottlenecks.

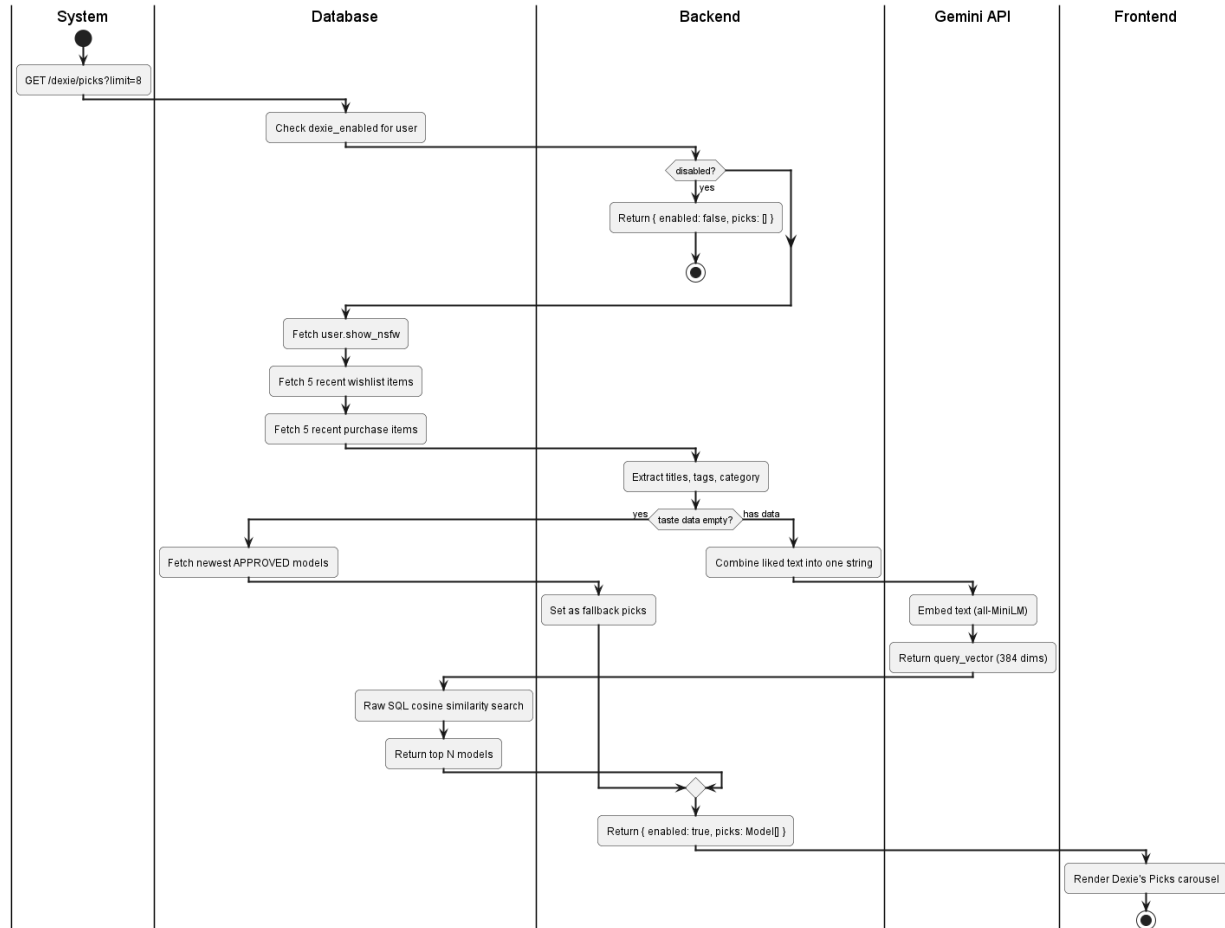


Figure 13: Activity Diagram - Dexie Picks

14 Activity Diagram - Dexie Tagline

UML Type: Activity Diagram **Category:** Dexie **Source:** docs/diagrams/activity/dexie/activity_dexie_tagli

14.1 Overview

This activity diagram describes the process of generating contextual AI taglines (assistant speech bubbles) when a user navigates the platform. It maps the integration of user settings checks, multi-tier caching (Redis/Memory), Gemini Flash API fallback flows, and frontend display deduplication.

14.2 Swim Lanes

Lane	Responsibility
User	Navigates the application
Frontend	Tracks path changes, sends requests, and filters repetitive dialogue
Backend	Verifies feature flags, routes cache lookups, and manages fallback states
Cache Service	Resolves per-user and global AI tagline keys
Gemini API	Generates personalized text dynamically

14.3 Process Flow

1. [User] Navigates to a platform path (e.g., /catalog/:id).
2. [Frontend] Identifies path change and constructs a context coordinate: { ctx: "catalog", tag: model_id }.
3. [Frontend] Dispatches request: GET /dexie/tagline?ctx=catalog&tag=model_id.
4. [Backend] Queries user.dexie_enabled.
 - If disabled: returns { enabled: false, message: null }. Flow ends.
5. [Cache Service] Looks up the per-user contextual tagline cache key (1-hour TTL).

Cache Evaluation:

- **Path A: Cache Hit**
 1. Cache Service returns the saved tagline directly. Rejoins at step 9.
- **Path B: Cache Miss**
 1. Backend checks the global contextual cache key (6-hour TTL).
 2. If global cache hits: Returns global tagline. Rejoins at step 9.
 3. If global cache misses: Backend initiates request to [Gemini API] with custom context prompts.

Gemini API Call and Fallback Evaluation:

- **If Gemini returns 200 OK:**
 - Saves the resulting string to the per-user cache (1-hour TTL) and global cache (6-hour TTL). Rejoins at step 9.
- **If Gemini returns 503 (Rate Limit or Error):**
 - Backend runs exponential backoff retries (up to 2 times).
 - If retries fail: Resolves a random, statically stored fallback tagline from the local library. Rejoins at step 9.

Rejoin and Frontend Handshake:

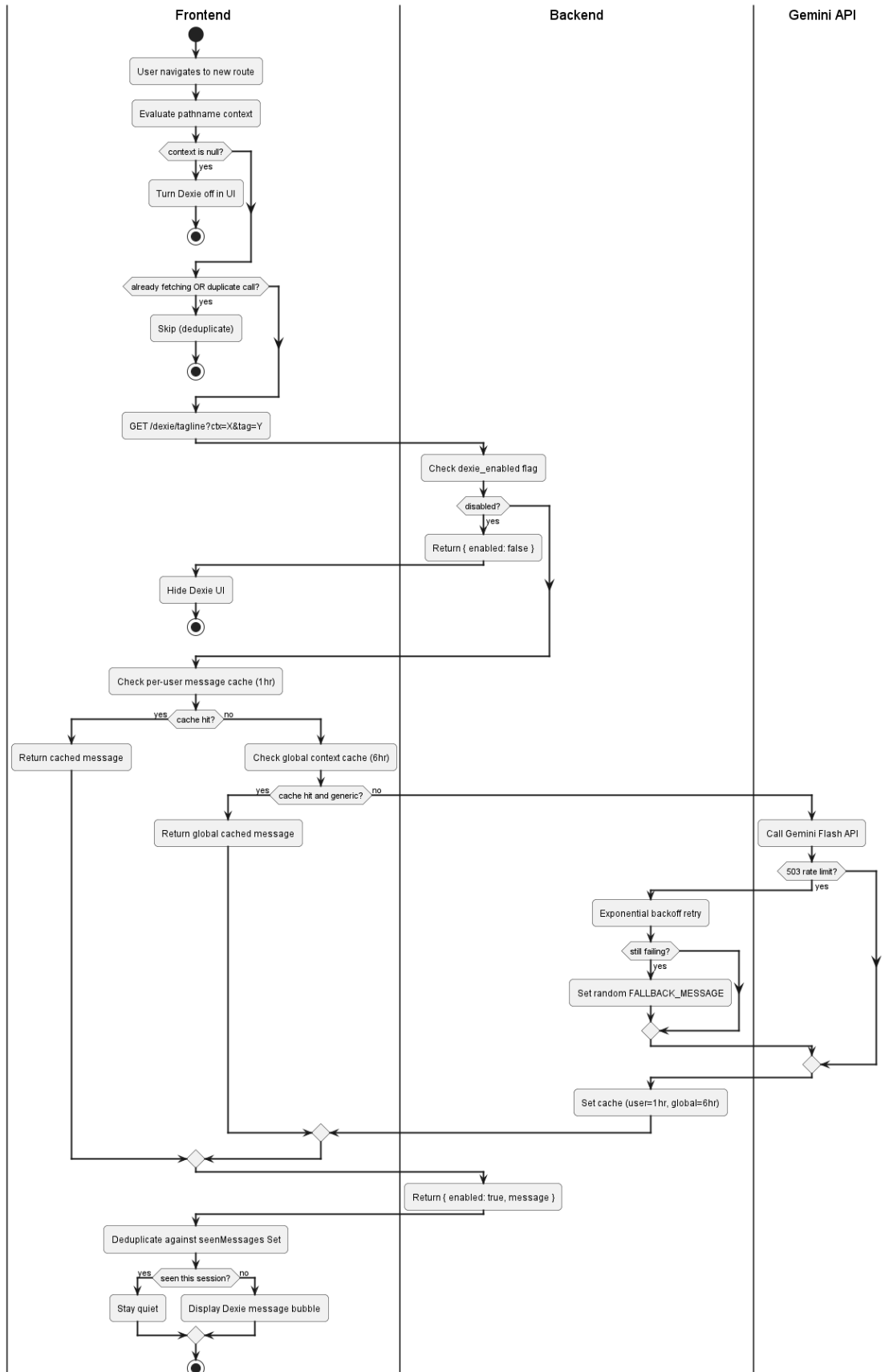


Figure 14: Activity Diagram - Dexie Tagline

6. **[Backend]** Returns 200 OK with the finalized tagline.
7. **[Frontend]** Receives the tagline and compares it against `seenMessages` in local state.
 - If already seen in the current session: Hides bubble. Flow ends.
 - If completely new: Adds message to `seenMessages`, sets local state, and plays the tagline animation overlay.

14.4 Decision Points Summary

Decision	Path/Condition	Outcome
Dexie enabled?	No	Return enabled: false, stop
User Cache Hit?	Yes	Return cached tagline, stop
Global Cache Hit?	Yes	Return cached tagline, stop
Gemini API status?	Fail after retries	Return random fallback tagline
Message already seen?	Yes	Hide bubble, stop

14.5 Key Implementation Details

- **Two-Tier Cache Strategy:** A per-user cache protects the system from repeated calls when the same user navigates back and forth. A global cache protects the system by sharing generic page tagline prompts (like home page welcoming lines) across different users.
- **Session Deduplication:** The frontend keeps a tracking state (`seenMessages`) which is initialized as an empty Set when the user mounts the app. This prevents the AI assistant from showing the same pop-up repeatedly, which would lead to poor user experience.

15 Activity Diagram - Enable 2FA Security

UML Type: Activity Diagram **Category:** Catalog (Profile Management) **Source:** docs/diagrams/activity/catalog/activity_enable_2fa.puml

15.1 Overview

This activity diagram describes the flow for enabling Time-based One-Time Password (TOTP) two-factor authentication on a user's account. In the UCD, this is an <<extend>> of the "Manage user profile" use case.

15.2 Swim Lanes

Lane	Responsibility
User	Initiates 2FA setup and verifies OTP
Frontend	Orchestrates setup flow and displays QR code
Backend	Generates TOTP secret and validates OTP
Database	Stores TOTP secret and 2FA enabled flag

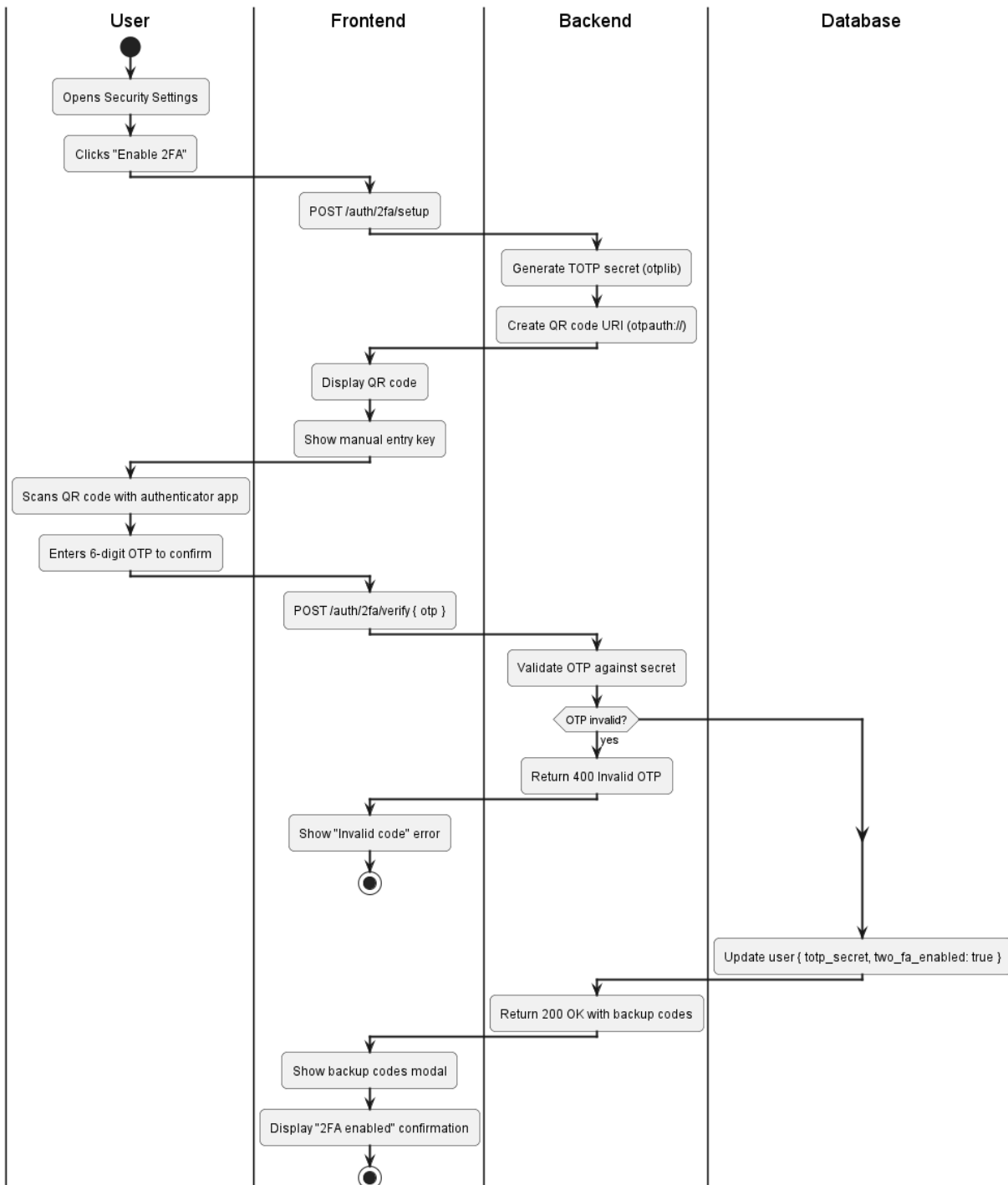


Figure 15: Activity Diagram - Enable 2FA Security

15.3 Process Flow

1. [User] Opens Security Settings and clicks “Enable 2FA”.
2. [Frontend] Dispatches `POST /auth/2fa/setup`.
3. [Backend] Generates TOTP secret using `otplib`. Creates QR code URI (`otpauth://`).
4. [Frontend] Displays QR code and manual entry key.
5. [User] Scans QR code with authenticator app (Google Authenticator, Authy, etc.).
6. [User] Enters the 6-digit OTP to confirm setup.
7. [Frontend] Dispatches `POST /auth/2fa/verify { otp }`.
8. [Backend] Validates OTP against the secret.
 - If invalid OTP: returns 400. Frontend shows “Invalid code” error.
9. [Database] Updates user: `{ totp_secret, two_fa_enabled: true }`.
10. [Backend] Returns 200 OK with backup codes.
11. [Frontend] Displays backup codes modal and “2FA enabled” confirmation.

15.4 Decision Points Summary

Decision	Path/Condition	Outcome
OTP valid?	No	Return 400, show error

15.5 Key Implementation Details

- **TOTP Standard:** The system uses the RFC 6238 TOTP standard, compatible with all standard authenticator apps.
- **Backup Codes:** One-time backup codes are generated and shown once on setup. Users must save them securely before closing the modal.
- **UCD Relationship:** This is a profile extension (`<<extend>>`) — enabling 2FA is optional and triggered from within the Manage user profile flow.

16 Activity Diagram - Follow Toggle

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_follow_toggle

16.1 Overview

This activity diagram describes the dual-path follow/unfollow toggle flow. It details how the backend handles session validation, checks follow states in the database, and dispatches in-app notifications only on fresh follow events.

16.2 Swim Lanes

Lane	Responsibility
User	Clicks the follow/unfollow toggle button
Frontend	Sends the toggle request and handles UI states
Backend	Runs credentials checks and triggers notifications

Lane	Responsibility
Database	Performs lookups, inserts, and deletions
Notification Service	Sends alerts to the target user

16.3 Process Flow

1. [User] Clicks the “Follow” or “Unfollow” button on a profile page.
2. [Frontend] Dispatches `POST /social/follow/toggle` with `{ target_user_id }`.
3. [Backend] Verifies the requesting user is authenticated.
 - If not authenticated: returns 401 Unauthorized. Flow ends.
4. [Backend] Verifies `user_id != target_user_id` (users cannot follow themselves).
 - If attempting self-follow: returns 400 Bad Request. Flow ends.
5. [Database] Queries `Follow` table for a record matching the unique pair (`follower_id`, `following_id`).
6. [Backend] Evaluates the check:

Decision Branch: Follow Record Exists?

- **Path A: Record Exists (Unfollow)**
 1. [Database] Deletes the `Follow` record.
 2. [Backend] Returns 200 OK with `{ following: false }`.
 3. [Frontend] Reverts button text to “Follow”.
- **Path B: Record Does Not Exist (Follow)**
 1. [Database] Inserts a new `Follow` record.
 2. [Notification Service] Dispatches a notification to the target user:
 - `type = FOLLOW`
 - `message = "@follower_username started following you!"`
 3. [Backend] Returns 200 OK with `{ following: true }`.
 4. [Frontend] Updates button text to “Following”.

16.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	No	Return 401, stop
Self-follow?	Yes	Return 400 Bad Request, stop
Record exists?	Yes	Delete follow record, return <code>following: false</code>
Record exists?	No	Create follow record, notify target, return <code>following: true</code>

16.5 Key Implementation Details

- **Database-Level Integrity:** The unique composite index on (`follower_id`, `following_id`) ensures that a follower can never create multiple records for the same target artist in the database.
- **Asymmetric Notification Rules:** While a new follow event triggers an instant notification for the artist, an unfollow event silently deletes the record without sending any alerts, protecting user privacy.

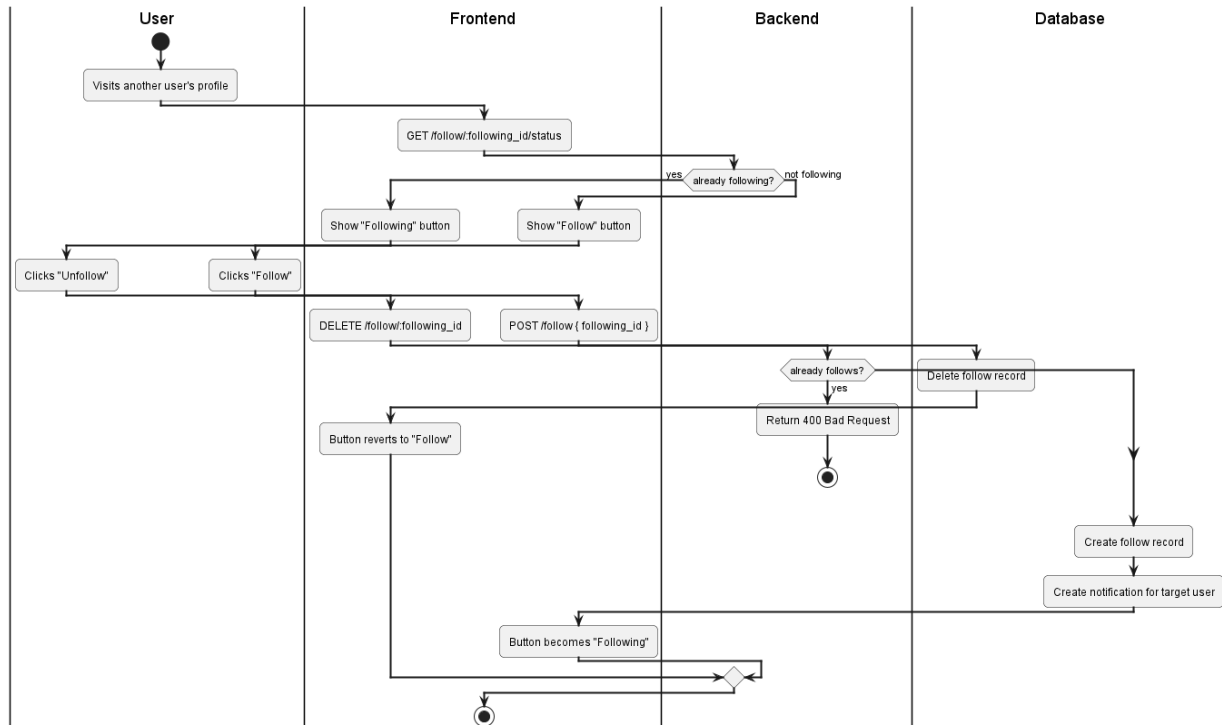


Figure 16: Activity Diagram - Follow Toggle

17 Activity Diagram - Google OAuth

UML Type: Activity Diagram **Category:** Auth **Source:** docs/diagrams/activity/auth/activity_google_oauth

17.1 Overview

This activity diagram maps the full process flow for Google OAuth authentication through swim lanes. It is more detailed than the sequence diagram in that it explicitly shows the three-way user-lookup branch.

17.2 Swim Lanes

Lane	Responsibility
User	Initiates the OAuth flow
Frontend	Manages the Google prompt and response
Backend	Verifies token and handles all three lookup paths
Database	Stores and returns user data

17.3 Process Flow

1. [User] Clicks "Continue with Google".
2. [Frontend] Triggers Google One Tap or OAuth redirect.

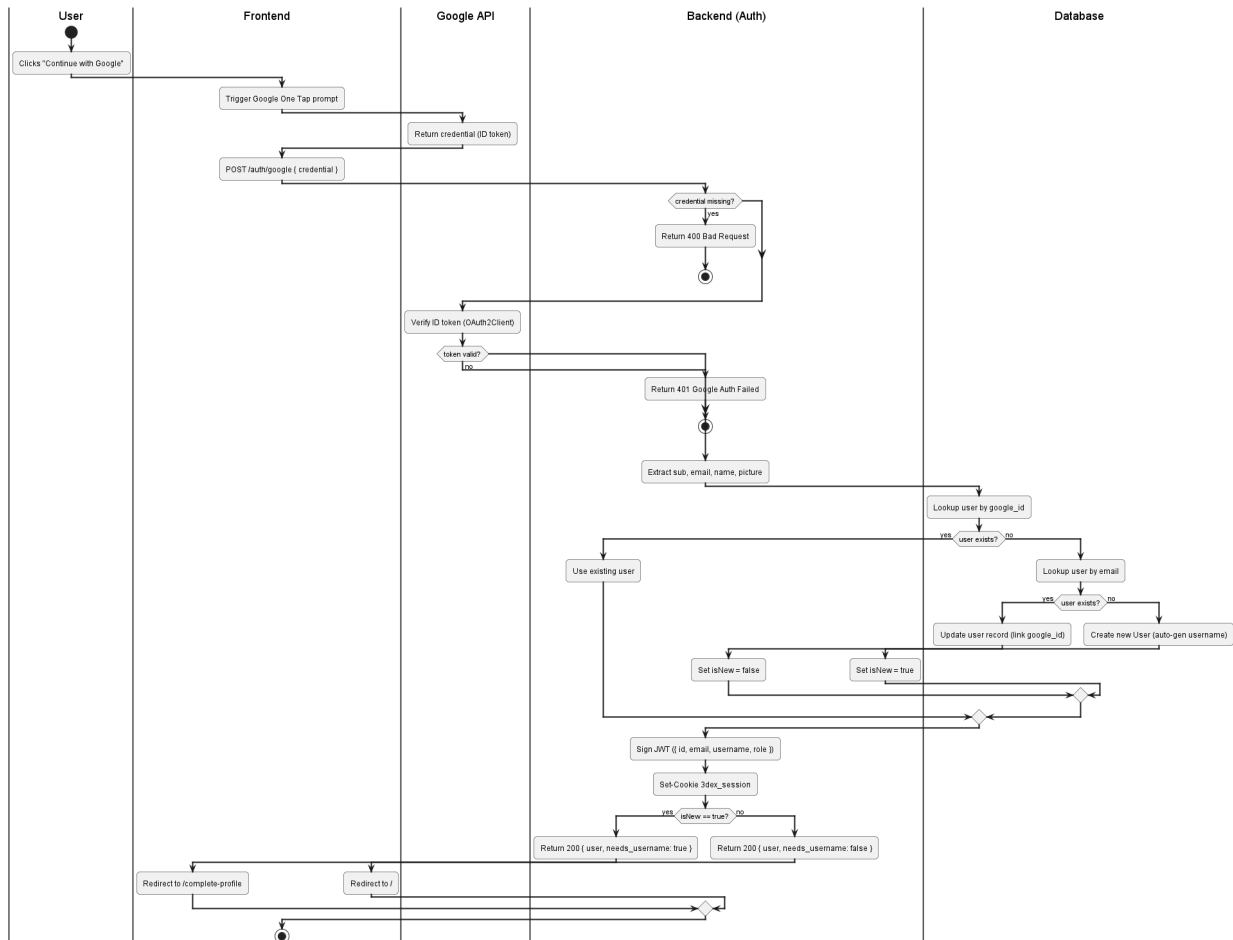


Figure 17: Activity Diagram - Google OAuth

3. **[Google Auth]** Returns a credential (ID token).
4. **[Frontend]** Sends POST /auth/google { credential }.
5. **[Backend]** Validates that credential is present.
 - If missing: returns 400.
6. **[Backend]** Calls Google's tokeninfo to verify the token.
 - If verification fails: returns 401.
7. **[Backend]** Extracts { sub, email, name, picture } from payload.
8. **[Database]** Attempts to find user by google_id = sub.

Three-way decision:

Path A - Google user found: 9A. **[Backend]** Uses the existing user directly.

Path B - No Google match, existing email user: 9B. **[Database]** Finds user by email. 10B. **[Database]** Updates that user's record: google_id = sub. 11B. **[Backend]** Sets isNew = false.

Path C - Completely new user: 9C. **[Database]** Creates new user: { google_id: sub, email, role: CUSTOMER, account_status: APPROVED }. 10C. **[Backend]** Sets isNew = true.

Rejoin:

10. **[Backend]** Signs JWT, sets 3dex_session cookie.
11. **[Backend]** Returns 200.
12. **[Frontend]** Checks needs_username:
 - If true (isNew): redirects to /complete-profile.
 - Otherwise: redirects to home.

17.4 Decision Points Summary

Decision	Path
Missing credential	400, stop
Token verification fails	401, stop
User found by google_id?	Path A
No google match, user found by email?	Path B
Neither match	Path C
needs_username?	Redirect to complete-profile

17.5 Notes

- Path B (account linking) silently links the Google account to an existing local account without any user action. This is transparent to the user.
- The isNew flag controls whether the user is forced to set a username before using the platform.

18 Activity Diagram - Local Login

UML Type: Activity Diagram **Category:** Auth **Source:** docs/diagrams/activity/auth/activity_local_login.p

18.1 Overview

This activity diagram models the process flow of local (email + password) login as a series of steps and decisions distributed across the Admin, Frontend, Backend, and Database swim lanes.

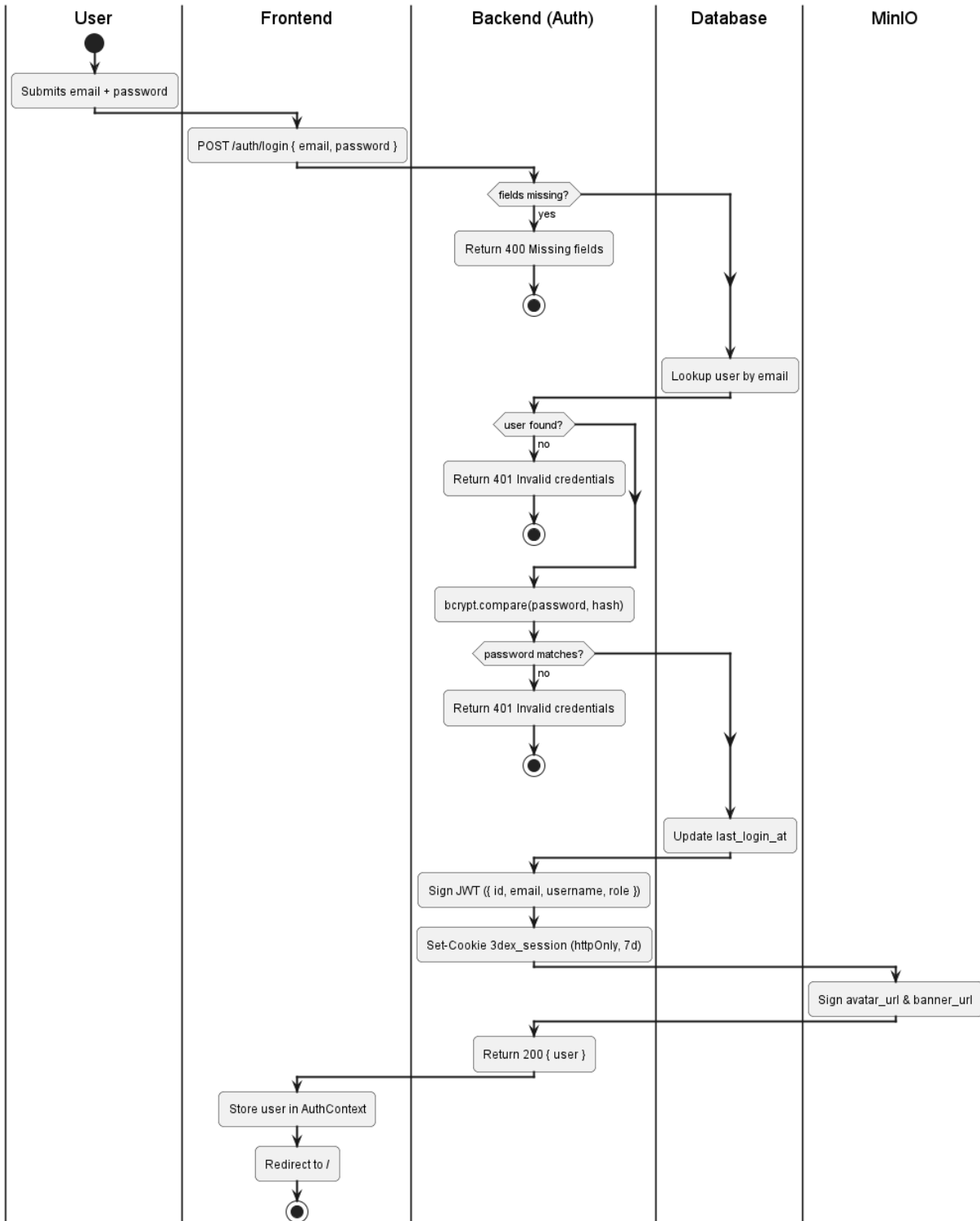


Figure 18: Activity Diagram - Local Login

18.2 Swim Lanes

Lane	Responsibility
User	Interacts with the login form
Frontend	Validates input client-side and sends the request
Backend	Executes authentication logic
Database	Stores and returns user data

18.3 Process Flow

1. [User] Opens the login form.
 2. [Frontend] User submits email and password.
 3. [Frontend] Client-side validation: checks fields are not empty.
 - If empty: shows a field required error. Flow ends.
 4. [Frontend] Sends `POST /auth/login { email, password }`.
 5. [Backend] Receives the request; runs field presence validation again.
 - If missing fields: returns 400.
 6. [Database] Looks up user by email.
 - If not found: Backend returns 401 Invalid credentials.
 7. [Backend] Calls `bcrypt.compare(input, hash)`.
 - If mismatch: returns 401 Invalid credentials.
 8. [Database] Updates `last_login_at`.
 9. [Backend] Signs JWT, sets `3dex_session` cookie.
 10. [Backend] Returns 200 with user object.
 11. [Frontend] Stores user in `AuthContext`, redirects to home.
-

18.4 Decision Points Summary

Decision	Yes Path	No Path
Fields empty?	Show error, stop	Continue
User found?	Continue	Return 401
Password match?	Continue	Return 401

18.5 Relationship to Sequence Diagram

This diagram reinforces the same flow shown in `sequence_local_login.puml` but uses swim lanes to emphasize which system component is responsible for each step, making the responsibility boundaries clearer for implementation purposes.

18.6 Notes

- The double validation (client-side and server-side) is intentional. Client-side validation gives immediate feedback; server-side validation is the authoritative gate.

- The `bcrypt.compare` step happens only after confirming the user exists, preventing unnecessary computation on non-existent accounts.

19 Activity Diagram - Manage User Profile

UML Type: Activity Diagram Category: Catalog Source: docs/diagrams/activity/catalog/activity_manage_

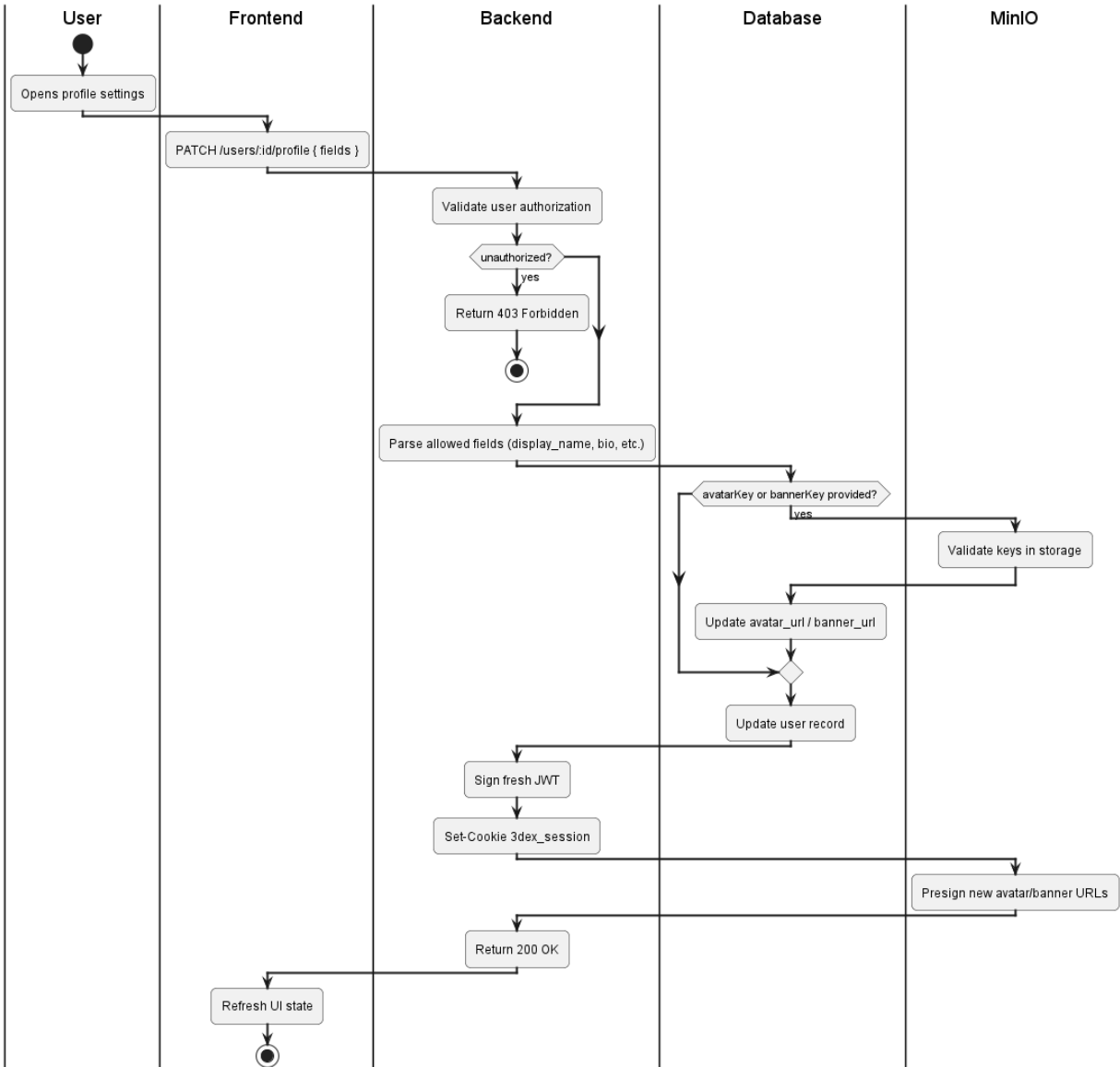


Figure 19: Activity Diagram - Manage User Profile

19.1 Overview

This activity diagram details the operational path when a user modifies their profile settings, including profile image uploads to S3 storage, JWT token re-signing, and session update coordination.

19.2 Swim Lanes

Lane	Responsibility
User	Modifies profile fields and saves settings
Frontend	Captures fields and dispatches request
Backend	Runs field sanitization, updates DB, and issues new tokens
Database	Updates User record details
MinIO	Validates uploaded asset keys and issues presigned URLs

19.3 Process Flow

1. [User] Opens their profile settings page.
2. [Frontend] Dispatches `PATCH /users/:id/profile` containing modified fields (display name, bio, banner keys, etc.).
3. [Backend] Verifies the request authentication and authorization context.
 - If unauthorized: returns 403 Forbidden. Flow ends.
4. [Backend] Parses and filters the input payload, extracting allowed fields.
5. [Backend] Checks if new S3 file keys (`avatarKey` or `bannerKey`) are provided.

Conditional Path: Image Update

- **If image keys are present:**
 1. [MinIO] Backend validates that the keys exist inside the storage bucket.
 2. [Database] Updates the user record with the new `avatar_url` and/or `banner_url` storage keys.
6. [Database] Persists all other changes to the `User` table.
 7. [Backend] Generates a fresh JWT containing updated profile summaries (display name, avatar URL, etc.).
 8. [Backend] Issues the fresh token in a new `3dex_session` Set-Cookie header.
 9. [MinIO] Generates updated presigned URLs for the user's avatar and banner image.
 10. [Backend] Returns 200 OK along with the updated user data.
 11. [Frontend] Updates its internal `AuthContext` state, refreshing the navigation bar and current view.

19.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authorized?	No	Return 403, stop
S3 keys provided?	Yes	Validate keys, update image URLs in database

19.5 Key Implementation Details

- **Sliding JWT Sessions:** When a user updates their profile, their session must reflect these updates immediately (e.g., in the navbar display name). Re-signing the JWT and returning a new session cookie prevents stale token states.
- **S3 Key Verification:** Rather than letting the user submit arbitrary strings for image paths, the backend calls MinIO storage APIs to verify the physical files exist before updating paths in the database.

20 Activity Diagram - Model Upload

UML Type: Activity Diagram Category: Catalog Source: docs/diagrams/activity/catalog/activity_model_upload

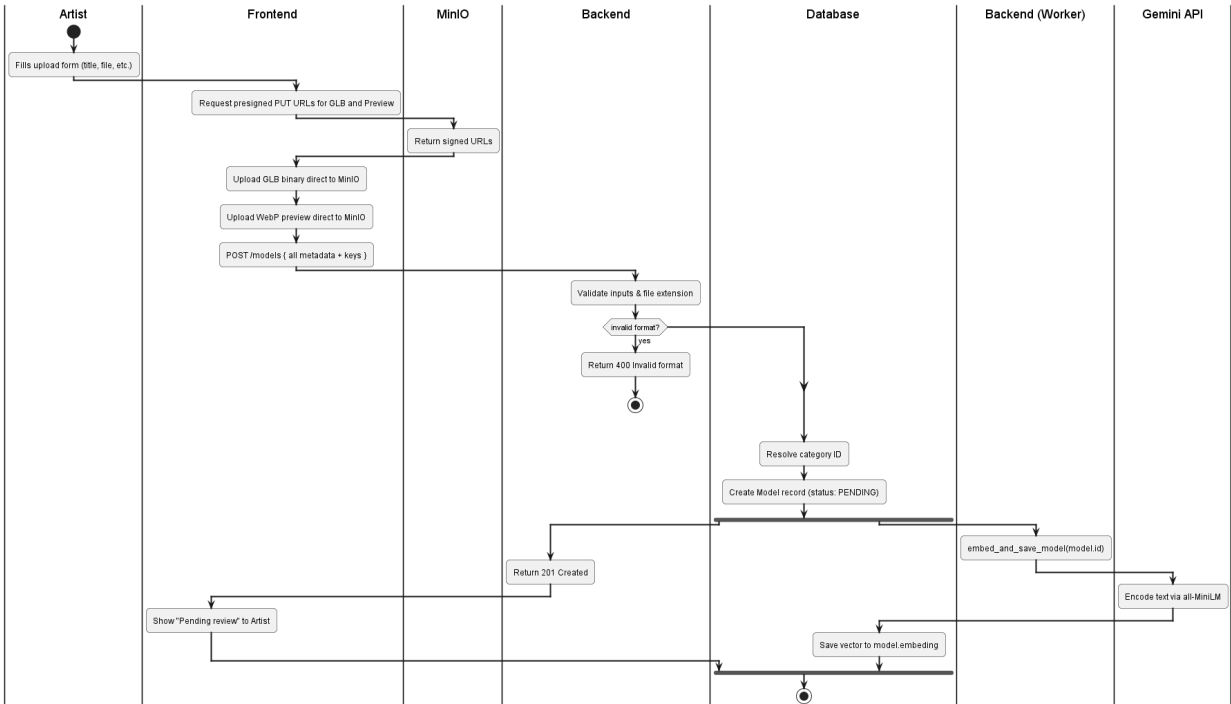


Figure 20: Activity Diagram - Model Upload

20.1 Overview

This activity diagram shows the model upload process with swim lanes, emphasizing the parallel fork between the immediate 201 Created response and the asynchronous background embedding generation.

20.2 Swim Lanes

Lane	Responsibility
Artist	Fills and submits the upload form
Frontend	Requests presigned URLs and sends file data
MinIO	S3-compatible storage for file upload
Backend	Validates metadata and creates the model record
Backend (Worker)	Asynchronous embedding generation
Gemini API	Text embedding model endpoint
Database	Persists model record and vector embedding

20.3 Process Flow

1. [Artist] Fills upload form (title, file, tags, category, etc.).

2. **[Frontend]** Requests presigned PUT URLs for GLB and WebP preview files.
3. **[MinIO]** Returns signed URLs.
4. **[Frontend]** Uploads GLB binary directly to MinIO.
5. **[Frontend]** Uploads WebP preview directly to MinIO.
6. **[Frontend]** Sends `POST /models { all metadata + S3 keys }`.
7. **[Backend]** Validates inputs and checks file extension.
 - If invalid format: returns 400. Flow ends.
8. **[Database]** Resolves `category_id` from slug.
9. **[Database]** Creates `Model` record with `status: PENDING`.

Fork (parallel):

Main path: 10A. **[Backend]** Returns 201 Created. 11A. **[Frontend]** Shows “Pending review” to Artist.

Background path: 10B. **[Backend (Worker)]** Calls `embed_and_save_model(model.id)`. 11B. **[Gemini API]** Encodes model title/description text via `all-MiniLM`. 12B. **[Database]** Saves 384-dim vector to `model.embedding`.

20.4 Decision Points Summary

Decision	Yes Path	No Path
File format valid (glb/gltf)?	Continue	Return 400, stop

20.5 Notes

- The parallel fork after model creation reflects the fire-and-forget nature of embedding generation. The user does not wait for the embedding to complete.
- The diagram labels the embedding model as “all-MiniLM” which refers to `all-MiniLM-L6-v2`. It is noted in the diagram as executed via Gemini API context, though in practice the embedding is done using a sentence-transformer library, not the Gemini Flash text generation API.

21 Activity Diagram - Notifications

UML Type: Activity Diagram **Category:** Notifications **Source:** docs/diagrams/activity/notifications/activi

21.1 Overview

This activity diagram describes the notification pull and mark-as-read workflow. It highlights the operations executed when loading lists, updating single notification records, and clearing all notifications at once.

21.2 Swim Lanes

Lane	Responsibility
User	Interacts with notification panel trigger and item links
Frontend	Issues API calls and updates local state counters
Backend	Runs session validation and constructs updates

Lane	Responsibility
Database	Queries and updates the <code>Notification</code> table

21.3 Process Flow

1. [User] Clicks the notification bell icon.
2. [Frontend] Dispatches `GET /notifications` to load history.
3. [Database] Queries `Notification` table: `findMany({ user_id, orderBy: created_at desc })`.
4. [Backend] Returns 200 OK along with notifications array.
5. [Frontend] Renders the list and updates the global unread count badge.

User Interaction Fork (mutually exclusive choices):

- **Choice A: Click Single Notification**
 1. [User] Clicks a single notification item in the popover.
 2. [Frontend] Dispatches `PATCH /notifications/:id/read`.
 3. [Database] Updates `Notification` record: `is_read = true`.
 4. [Backend] Returns 200 OK.
 5. [Frontend] Decrements the navbar unread count by 1 and navigates to the notification's destination path.
- **Choice B: Click "Mark All as Read"**
 1. [User] Clicks "Mark all as read" button.
 2. [Frontend] Dispatches `POST /notifications/read-all`.
 3. [Database] Updates all matching records: `updateMany({ user_id, is_read: false }, { is_read: true })`.
 4. [Backend] Returns 200 OK.
 5. [Frontend] Sets the unread count badge to 0.

21.4 Decision Points Summary

Decision	Path/Condition	Outcome
Click action?	Single	Send patch, update single item, decrement badge count, navigate
Click action?	Mark All	Send read-all post, update all records to read, zero badge count

21.5 Key Implementation Details

- **Deep Linking:** The `Notification` model includes a `JSON data` column. This holds contextual details (e.g., `{ model_id: "...", post_id: "..." }`), which the frontend reads when a notification is clicked to determine where to redirect the user.
- **Atomic Read-All Updates:** The `updateMany` operation on the Database runs as a single SQL update command, ensuring fast execution even if the user has many unread records.

22 Activity Diagram - Post Create

UML Type: Activity Diagram Category: Social Source: docs/diagrams/activity/social/activity_post_create

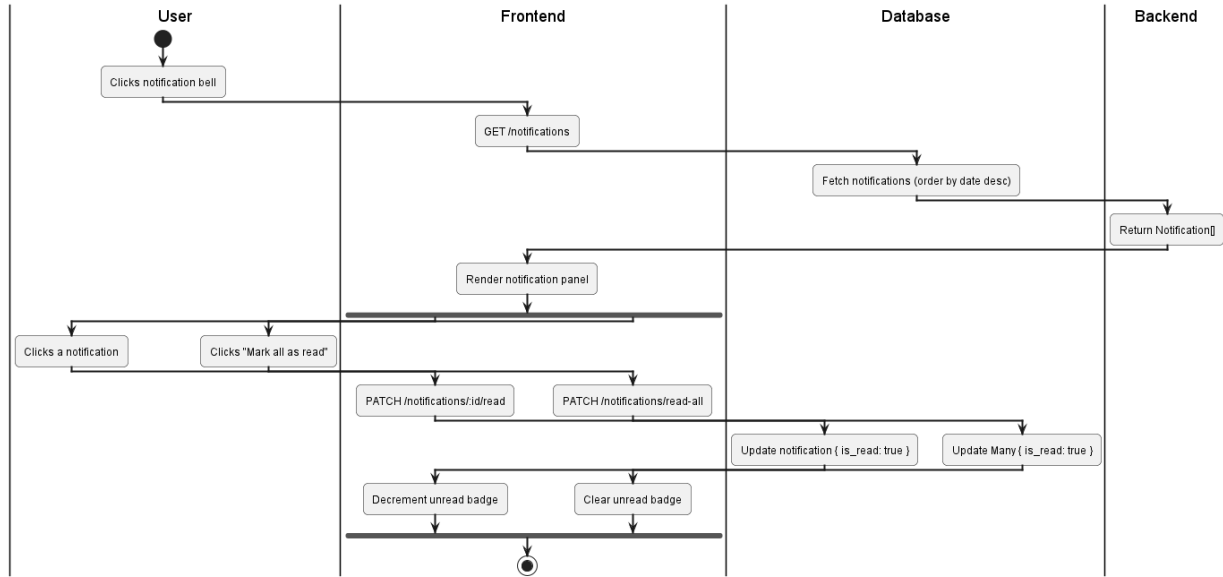


Figure 21: Activity Diagram - Notifications

22.1 Overview

This activity diagram describes the process of creating a new community post. It outlines the role-based validation checks (only Artists and Providers are allowed to post) and S3 file attachment flows.

22.2 Swim Lanes

Lane	Responsibility
Artist or Provider	Fills out caption and uploads images
Frontend	Requests presigned URLs, uploads to MinIO, and sends metadata
MinIO	S3-compatible storage backend
Backend	Runs role checks, validates input, and persists the post
Database	Inserts the new Post record

22.3 Process Flow

1. **[Artist or Provider]** Fills out the caption, sets NSFW toggles, and selects images.
2. **[Frontend]** Requests presigned PUT URLs from the backend for each image.
3. **[MinIO]** Returns signed S3 upload URLs.
4. **[Frontend]** Uploads raw image binaries directly to MinIO using the presigned URLs.
5. **[Frontend]** Dispatches `POST /posts` with `{ caption, media_urls: [keys], is_nsfw }`.
6. **[Backend]** Verifies user credentials and role.
 - If user role is `CUSTOMER` or `ADMIN`: returns 403 Forbidden (“Only Artists and Providers can create community posts”). Flow ends.

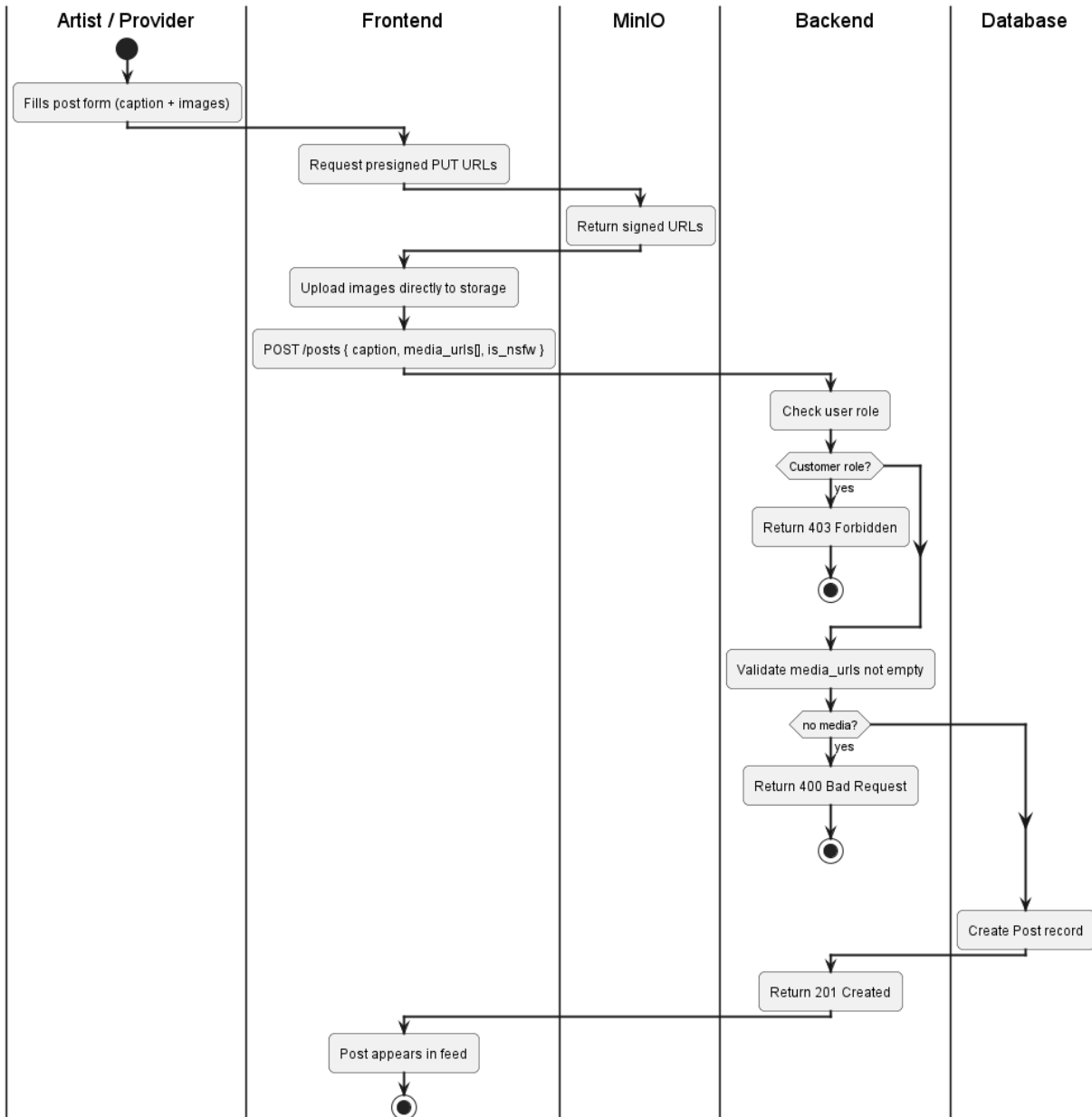


Figure 22: Activity Diagram - Post Create

7. **[Backend]** Validates the request payload:
 - Verifies the caption length and formats.
 - Verifies `media_urls` is not empty (community posts must contain at least one image/media attachment).
 - If invalid: returns 400 Bad Request. Flow ends.
 8. **[Database]** Inserts the new `Post` record containing the media paths and initial zeroed statistics (`like_count = 0, comment_count = 0`).
 9. **[Backend]** Returns 201 Created along with the new post details.
 10. **[Frontend]** Appends the post to the top of the community feed.
-

22.4 Decision Points Summary

Decision	Path/Condition	Outcome
Role is Artist or Provider?	No	Return 403 Forbidden, stop
Media attachments exist?	No	Return 400 Bad Request, stop

22.5 Key Implementation Details

- **Role Constraints:** The `Post` table represents promotional or portfolio updates. To maintain high-quality feed content, only approved creators (**ARTIST** or **PROVIDER**) are authorized to upload posts. Regular users (**CUSTOMER**) can interact through likes and comments but cannot post directly.
- **Direct S3 Uploads:** Similar to model uploads, image binaries are uploaded directly to MinIO storage from the frontend to keep the application server highly responsive.

23 Activity Diagram - Proceed to Checkout

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_proceed_to_checkout

23.1 Overview

This activity diagram covers the checkout flow from initiating order validation to generating a payment gateway token and displaying the Midtrans payment popup.

23.2 Swim Lanes

Lane	Responsibility
Customer	Initiates checkout and pays
Frontend	Issues API request and renders Midtrans Snap iframe
Backend	Runs safety checks, aggregates prices, and creates orders
Database	Performs purchase validation and creates Order record
Midtrans API	Registers transaction and issues Snap tokens

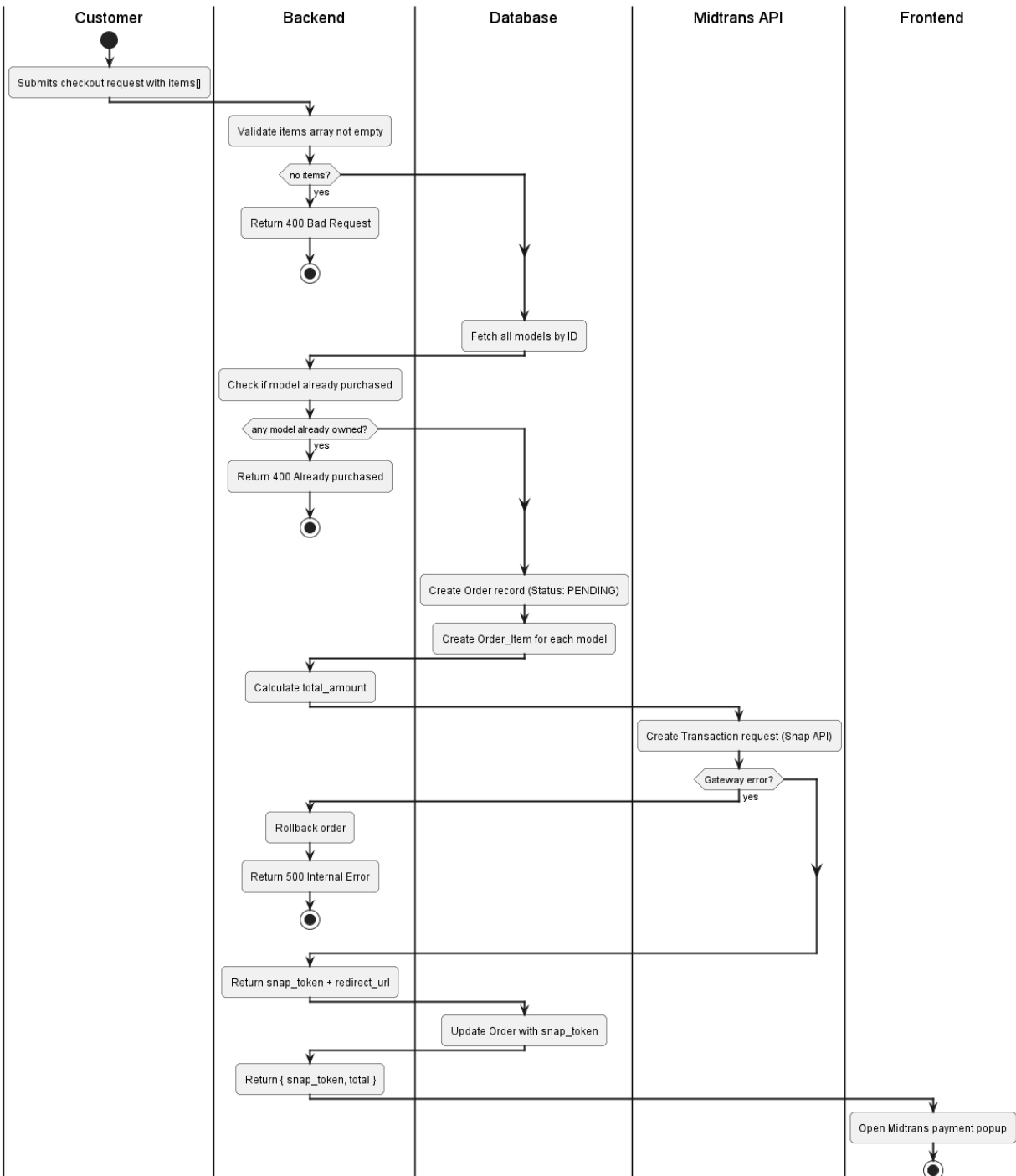


Figure 23: Activity Diagram - Proceed to Checkout

23.3 Process Flow

1. [Customer] Clicks “Checkout” inside the Cart page.
 2. [Frontend] Dispatches `POST /orders/checkout` with `{ items: [{ model_id, quantity }] }`.
 3. [Database] Looks up the selected models.
 4. [Backend] Validates all items are available and `APPROVED`.
 5. [Database] Queries `Purchase` history to ensure none of these items are already owned by the user.
 - If a duplicate purchase is detected: returns 400 Bad Request. Flow ends.
 6. [Database] Computes the sum of all model prices and inserts a new `Order` record:
 - `status = PENDING`
 - `type = ASSET`
 - `total_amount = sum(prices)`
 7. [Backend] Initiates request to [Midtrans API] Snap endpoint with order metadata and gross amount.
 8. [Midtrans API] Registers the transaction and issues a unique `{ snap_token, redirect_url }`.
 9. [Database] Updates the `Order` record, writing the retrieved `snap_token` and `snap_redirect_url`.
 10. [Backend] Returns 200 OK along with the token and order summaries.
 11. [Frontend] Triggers the global window script `snap.pay(token)`.
 12. [Customer] Sees the interactive payment modal overlay (Midtrans Snap widget).
-

23.4 Decision Points Summary

Decision	Path/Condition	Outcome
Items exist & <code>APPROVED</code> ?	No	Return 404/400, stop
Already purchased?	Yes	Return 400 Bad Request, stop

23.5 Key Implementation Details

- **Duplicate Check:** To prevent users from buying items they already own, the checkout endpoint performs a strict verification against the `Purchase` table. If any item in the cart is already purchased, checkout is blocked.
- **Midtrans Snap Integration:** The transaction is registered with Midtrans *before* the customer pays. The resulting token is saved in the database, allowing the user to resume payment if the popup is closed.
- **Transactional Consistency:** If the Midtrans API call fails, the local `Order` record remains in `PENDING` state or can be rolled back.

24 Activity Diagram - Proceed with Payment

UML Type: Activity Diagram Category: Commerce Source: docs/diagrams/activity/commerce/activity_proceed_with_payment

24.1 Overview

This activity diagram describes the system’s asynchronous response to the Midtrans payment notification webhook. It details how the transaction status is read, how payments are upserted for idempotency, and how the user’s purchased assets are unlocked.

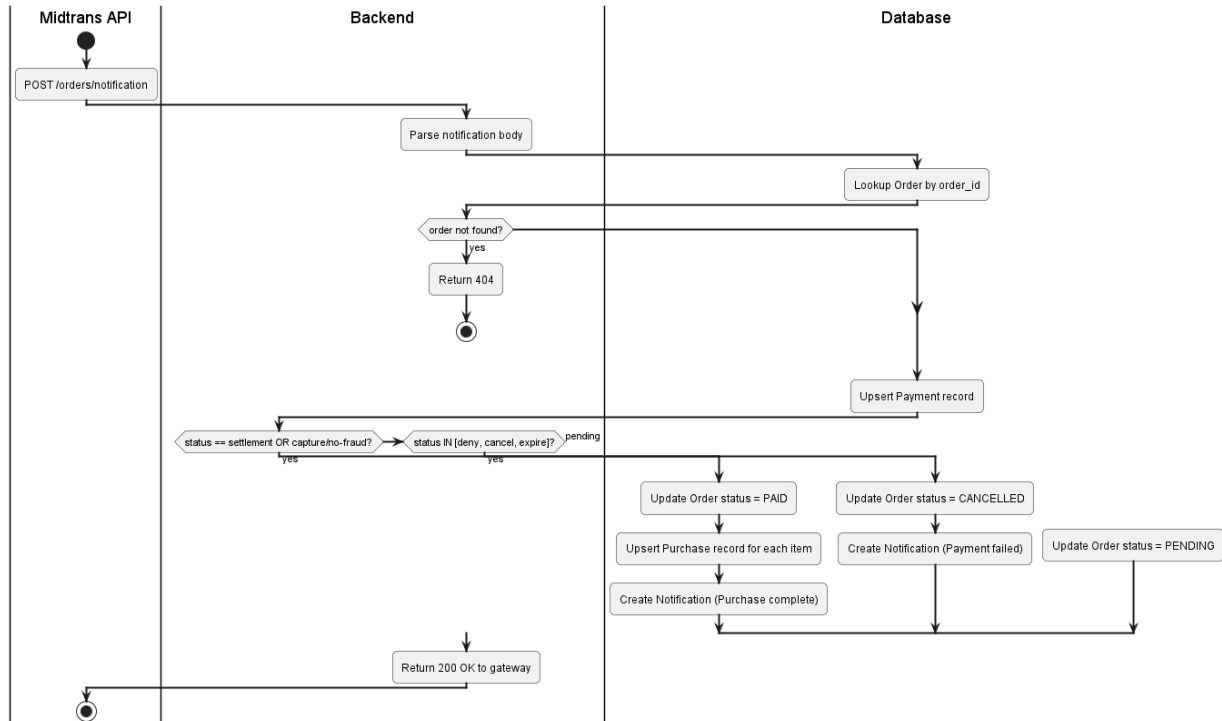


Figure 24: Activity Diagram - Proceed with Payment

24.2 Swim Lanes

Lane	Responsibility
Midtrans API	Dispatches POST notification webhook
Backend	Parses notifications, routes logic, and verifies signatures
Database	Updates Orders, creates Purchases, and logs transactions
Notification Service	Sends alerts to the user's notification list

24.3 Process Flow

1. [Midtrans API] Sends POST /orders/notification with transaction details.
2. [Backend] Verifies the webhook signature using the shared merchant server key.
 - If signature is invalid: returns 401 Unauthorized. Flow ends.
3. [Database] Looks up the target Order by the provided order_id.
 - If order not found: Backend returns 404. Flow ends.
4. [Database] Upserts a Payment record using the transaction_id as the key. This ensures the webhook handling is idempotent.
5. [Backend] Evaluates the transaction status parameter.

Conditional Path Branching:

- **Path A: Settlement or Capture (Success)**
 - Conditions: `transaction_status == settlement OR (transaction_status == capture and fraud_status != deny)`.

1. [Database] Updates `Order.status = PAID`.
2. [Database] Creates a `Purchase` record for each `Order_Item` in the order, unlocking the model for download.
3. [Notification Service] Dispatches a notification to the customer.
- **Path B: Deny, Cancel, or Expire (Failure)**
 - Conditions: `transaction_status` is in `[deny, cancel, expire]`.
 - 1. [Database] Updates `Order.status = CANCELLED`.
 - 2. [Notification Service] Dispatches a notification to the customer indicating failure.
- **Path C: Pending**
 - Conditions: `transaction_status == pending`.
 - 1. [Database] Leaves `Order.status = PENDING`.

Rejoin:

6. [Backend] Returns 200 OK to Midtrans.

24.4 Decision Points Summary

Decision	Path/Condition	Outcome
Signature Valid?	No	Return 401 Unauthorized, stop
Status Settlement/Capture?	Yes	Order = PAID, Create Purchases, notify user
Status Deny/Cancel/Expire?	Yes	Order = CANCELLED, notify user
Status Pending?	Yes	Keep Order = PENDING

24.5 Key Implementation Details

- **Idempotency via Upsert:** Since Midtrans may retry webhooks if network glitches occur, the `Payment` upsert avoids duplicate payment processing. If a payment record with the same transaction ID already exists, its details are updated but duplicate `Purchase` records are not created.
- **Secure Webhook Verification:** The system computes a SHA-512 hash using `order_id`, `status_code`, `gross_amount`, and the server key, comparing it to the webhook's `signature_key` to prevent fraud.

25 Activity Diagram - Rate Counterparty

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_rate_count

25.1 Overview

This activity diagram describes the peer-to-peer rating flow for rating an Artist or Provider. It enforces eligibility checks (transaction-gating and duplicate prevention) before allowing a review submission. This corresponds to the “Rate counterparty” use case in the UCD, which includes (<<include>>) “View user profile”.

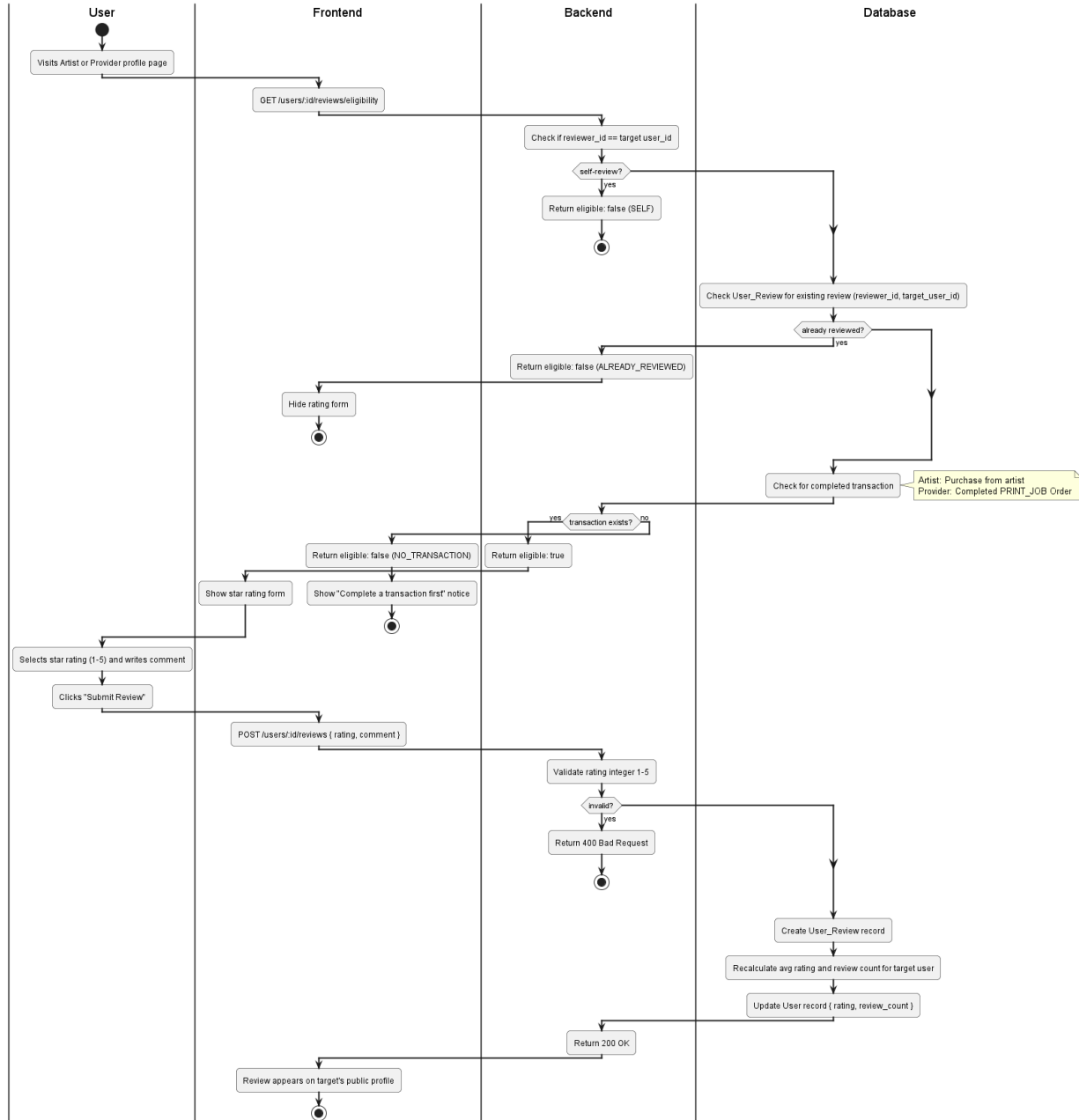


Figure 25: Activity Diagram - Rate Counterparty

25.2 Swim Lanes

Lane	Responsibility
User	Visits profile and submits star rating
Frontend	Checks eligibility and renders form
Backend	Validates eligibility, rating, and self-review prevention
Database	Checks transactions, creates review, aggregates rating

25.3 Process Flow

25.3.1 Phase 1: Eligibility Check

1. [User] Visits an Artist or Provider profile page.
 2. [Frontend] Dispatches `GET /users/:id/reviews/eligibility`.
 3. [Backend] Checks if reviewer is the target.
 - If self-review: returns `eligible: false (SELF)`.
 4. [Database] Checks `User_Review` for an existing review from this user to this target.
 - If already reviewed: returns `eligible: false (ALREADY_REVIEWED)`. Frontend hides form.
 5. [Database] Checks for a completed transaction:
 - For an Artist: looks for a `Purchase` record from this artist.
 - For a Provider: looks for a completed `PRINT_JOB` order from this provider.
 6. [Backend] Evaluates transaction:
 - Transaction exists: returns `eligible: true`. Frontend shows form.
 - No transaction: returns `eligible: false (NO_TRANSACTION)`. Frontend shows “Complete a transaction first” notice.
-

25.3.2 Phase 2: Review Submission

7. [User] Selects star rating (1–5) and writes comment. Clicks “Submit Review”.
 8. [Frontend] Dispatches `POST /users/:id/reviews { rating, comment }`.
 9. [Backend] Validates `rating` is integer 1–5.
 - If invalid: returns 400.
 10. [Database] Creates `User_Review` record.
 11. [Database] Recalculates avg rating and review count for target user.
 12. [Database] Updates `User` record `{ rating, review_count }`.
 13. [Backend] Returns 200 OK.
 14. [Frontend] Review appears on the target’s public profile.
-

25.4 Decision Points Summary

Decision	Path/Condition	Outcome
Self-review?	Yes	Return <code>eligible: false (SELF)</code>
Already reviewed?	Yes	Return <code>eligible: false (ALREADY_REVIEWED)</code> , hide form
Completed transaction?	No	Return <code>eligible: false (NO_TRANSACTION)</code> , show notice

Decision	Path/Condition	Outcome
Rating valid (1-5)?	No	Return 400 Bad Request

25.5 Key Implementation Details

- **Transaction Gate:** Prevents review manipulation. You must have completed a real transaction with the user before rating them.
- **Peer-to-Peer vs. Model Reviews:** This flow stores in `User_Review` and aggregates onto `User.rating`. It is distinct from model reviews (`Review` table $\rightarrow$ `Model.avg_rating`).

26 Activity Diagram - User Registration

UML Type: Activity Diagram Category: Auth Source: docs/diagrams/activity/auth/activity_register.puml

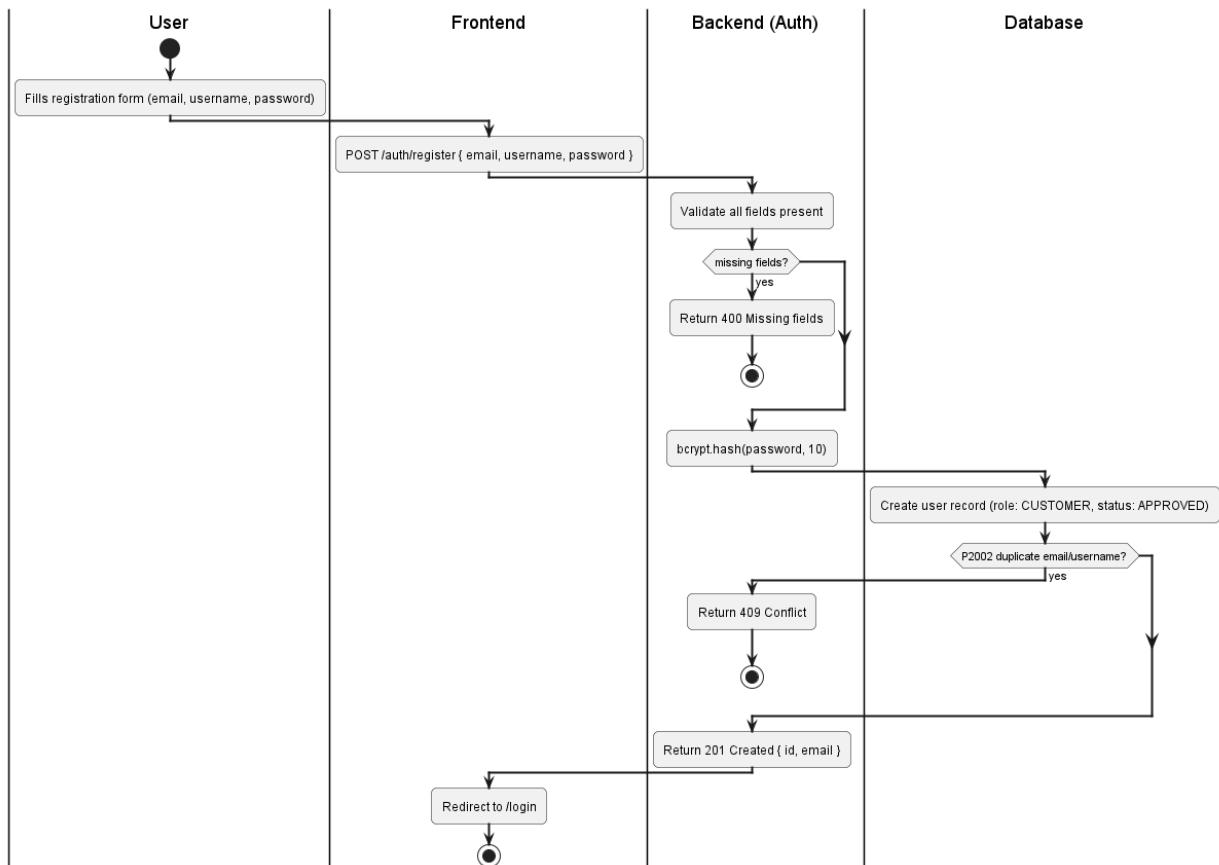


Figure 26: Activity Diagram - User Registration

26.1 Overview

This activity diagram shows the user registration process across swim lanes, emphasizing the validation steps, password hashing, and uniqueness enforcement.

26.2 Swim Lanes

Lane	Responsibility
User	Fills and submits the registration form
Frontend	Client-side validation
Backend	Server-side validation and persistence
Database	Data creation and constraint checking

26.3 Process Flow

1. [User] Fills in email, username, and password.
 2. [Frontend] Client validates that all fields are present and the username meets format rules (3-30 characters, alphanumeric, no spaces).
 - If invalid: shows field error. Flow ends.
 3. [Frontend] Sends `POST /auth/register { email, username, password }`.
 4. [Backend] Validates presence of all fields.
 - If missing: returns 400.
 5. [Backend] Calls `bcrypt.hash(password, 10)`.
 6. [Database] Calls `user.create({ email, username, password_hash, role: CUSTOMER, account_status: APPROVED })`.
 7. [Database] Checks unique constraints on `email` and `username`.
 - If Prisma error P2002 (unique violation): Backend returns 409 Conflict (“Email or username already taken”).
 8. [Backend] Returns 201 Created.
 9. [Frontend] Shows success message, redirects to `/login`.
-

26.4 Decision Points Summary

Decision	Yes Path	No Path
Client fields valid?	Continue	Show error, stop
Server fields present?	Continue	Return 400
DB unique constraint passes?	Continue	Return 409

26.5 Notes

- Registration creates only `CUSTOMER` accounts. Role upgrades require a separate application flow.
- No session token is issued at registration. The user must log in separately after completing registration.
- The username validation rules (3-30 chars, alphanumeric) are enforced at both the client level and should be enforced at the Prisma level via `@db.VarChar(30)`.

27 Activity Diagram - Report Content/User

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_report_con

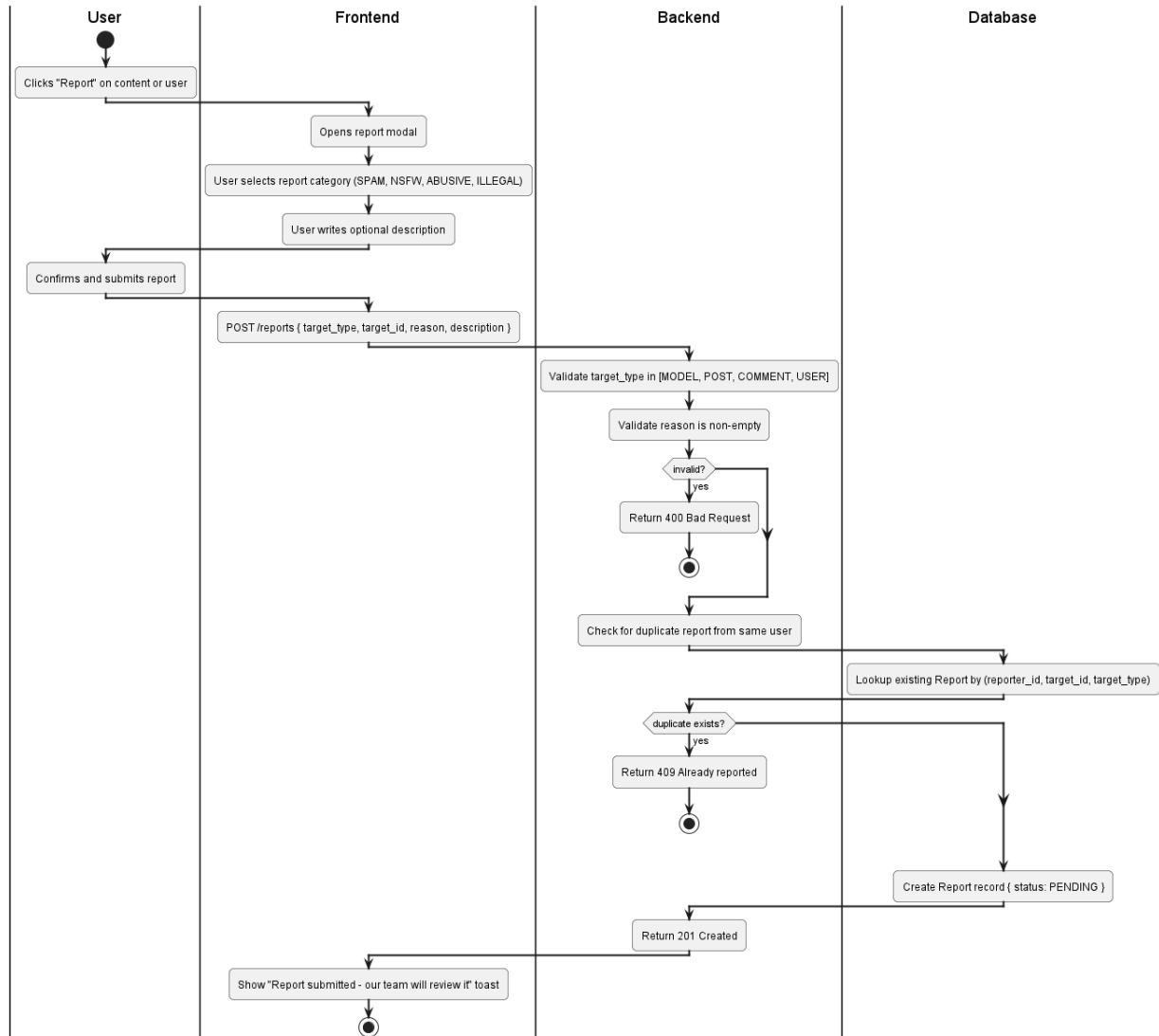


Figure 27: Activity Diagram - Report Content/User

27.1 Overview

This activity diagram describes the flow for reporting abusive, inappropriate, or illegal content or users on the platform. The resulting report enters the admin’s moderation queue for resolution.

27.2 Swim Lanes

Lane	Responsibility
User	Triggers the report action
Frontend	Renders the report modal and submits the report
Backend	Validates the report payload and checks for duplicates
Database	Creates and stores the pending report record

27.3 Process Flow

1. [User] Clicks the “Report” option on a model, post, comment, or user.
 2. [Frontend] Opens the report modal.
 3. [User] Selects a report category (SPAM, NSFW, ABUSIVE, ILLEGAL) and optionally writes a description.
 4. [User] Confirms and submits the report.
 5. [Frontend] Dispatches `POST /reports { target_type, target_id, reason, description }`.
 6. [Backend] Validates `target_type` is one of `[MODEL, POST, COMMENT, USER]`.
 7. [Backend] Validates `reason` is non-empty.
 - If invalid: returns 400 Bad Request.
 8. [Backend] Checks for a duplicate report from the same user.
 9. [Database] Looks up existing `Report` by `(reporter_id, target_id, target_type)`.
 - If duplicate exists: returns 409 Already reported.
 10. [Database] Creates `Report` record `{ status: PENDING }`.
 11. [Backend] Returns 201 Created.
 12. [Frontend] Shows “Report submitted — our team will review it” toast.
-

27.4 Decision Points Summary

Decision	Path/Condition	Outcome
Target type valid?	No	Return 400 Bad Request
Reason non-empty?	No	Return 400 Bad Request
Duplicate report?	Yes	Return 409 Already reported

27.5 Key Implementation Details

- **Admin Queue:** Submitted reports go into the admin moderation queue, where they are processed through the `activity_resolve_reports` flow.
- **Duplicate Prevention:** The unique constraint on `(reporter_id, target_id, target_type)` prevents the same user from spamming multiple reports on the same content item.

28 Activity Diagram - Request Earnings Payout

UML Type: Activity Diagram Category: Print Source: docs/diagrams/activity/print/activity_request_pay

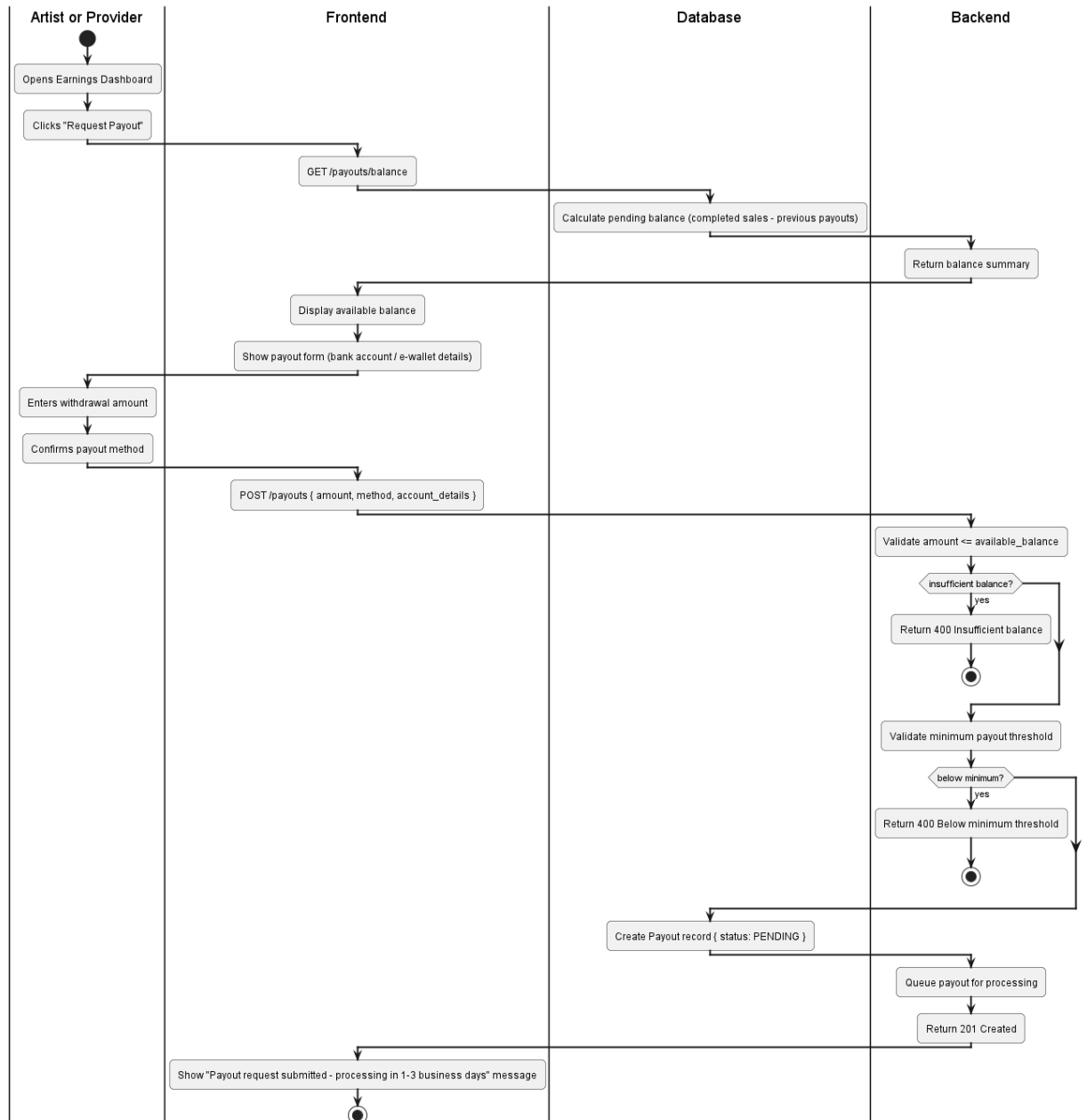


Figure 28: Activity Diagram - Request Earnings Payout

28.1 Overview

This activity diagram describes the payout request flow for Artists and Providers who have accumulated earnings from model sales or print jobs. The system validates the available balance before creating a pending payout record.

28.2 Swim Lanes

Lane	Responsibility
Artist or Provider	Initiates payout from their earnings dashboard
Frontend	Fetches balance, renders form, submits request
Backend	Validates amount against balance and minimum threshold
Database	Calculates available balance and creates payout record

28.3 Process Flow

1. **[Artist or Provider]** Opens the Earnings Dashboard.
 2. **[Artist or Provider]** Clicks “Request Payout”.
 3. **[Frontend]** Dispatches GET `/payouts/balance`.
 4. **[Database]** Calculates pending balance: completed sales — previous payouts.
 5. **[Backend]** Returns balance summary.
 6. **[Frontend]** Displays available balance and payout form (bank account / e-wallet details).
 7. **[Artist or Provider]** Enters withdrawal amount and confirms payout method.
 8. **[Frontend]** Dispatches POST `/payouts { amount, method, account_details }`.
 9. **[Backend]** Validates `amount <= available_balance`.
 - If insufficient: returns 400 Insufficient balance.
 10. **[Backend]** Validates amount meets the minimum payout threshold.
 - If below minimum: returns 400 Below minimum threshold.
 11. **[Database]** Creates Payout record `{ status: PENDING }`.
 12. **[Backend]** Queues payout for processing. Returns 201 Created.
 13. **[Frontend]** Shows “Payout request submitted — processing in 1–3 business days” message.
-

28.4 Decision Points Summary

Decision	Path/Condition	Outcome
Amount <code><=</code> available balance?	No	Return 400 Insufficient balance
Amount <code>>=</code> minimum threshold?	No	Return 400 Below minimum

28.5 Key Implementation Details

- **Balance Calculation:** Available balance is computed dynamically from `completed orders - existing payouts`, ensuring no overpayment edge cases.
- **Processing Queue:** Payouts are not instant — they are queued for manual or automated processing and marked `COMPLETED` once funds are transferred.

29 Activity Diagram - Request Print

UML Type: Activity Diagram Category: Print Source: docs/diagrams/activity/print/activity_request_print

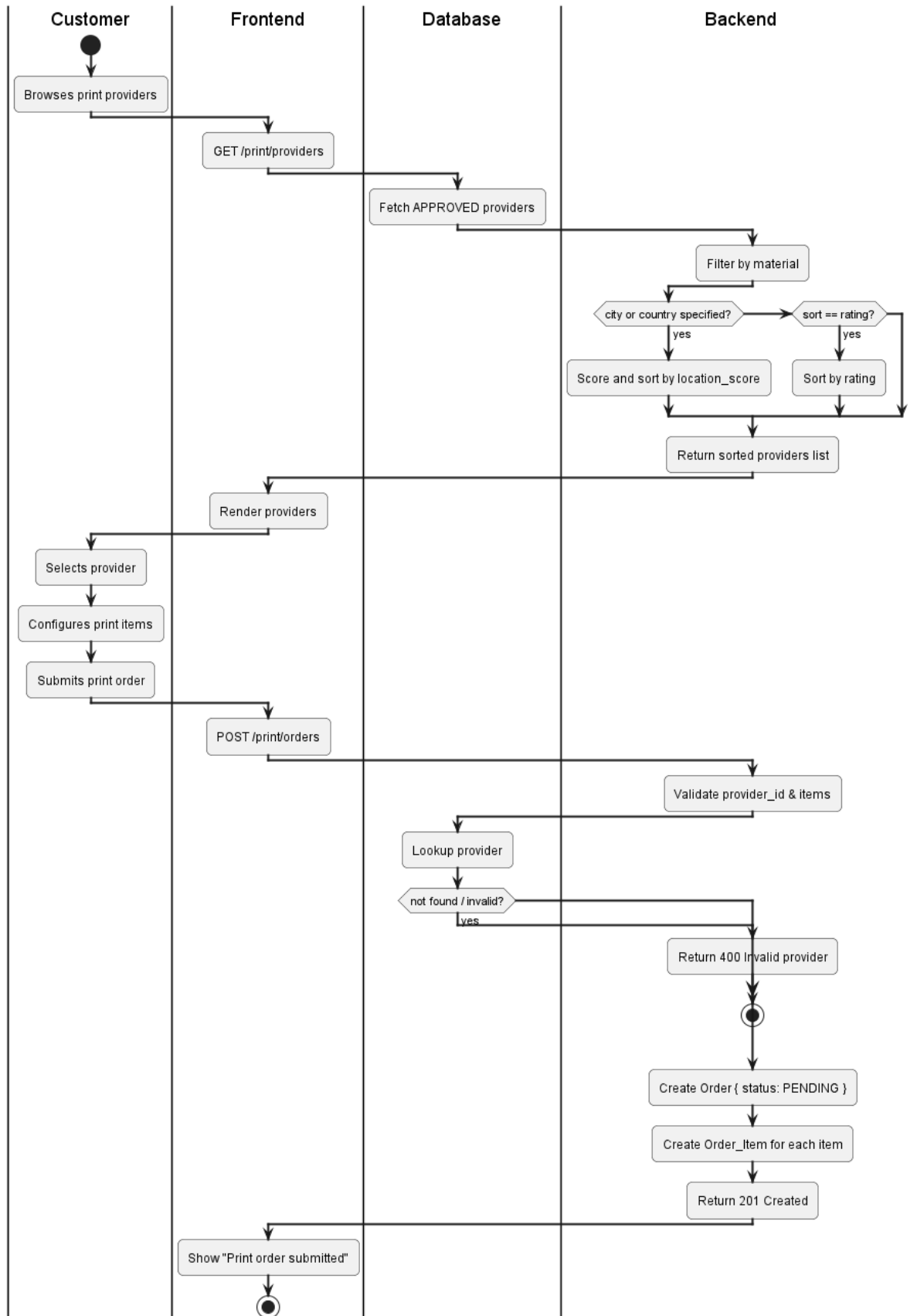


Figure 29: Activity Diagram - Request Print

29.1 Overview

This activity diagram describes the physical 3D print order creation process. It maps how customers browse localized print providers by compatibility parameters and how the order is registered in the database for the provider's queue.

29.2 Swim Lanes

Lane	Responsibility
Customer	Browses providers and configures print jobs
Frontend	Handles filters, queries, and checkout assembly
Backend	Executes routing logic, checks credentials, and processes orders
Database	Performs lookups and creates the Print Job Order record

29.3 Process Flow

1. [Customer] Initiates the “Print this model” wizard from a model detail page.
2. [Frontend] Dispatches query `GET /print/providers?material=PLA&city=Bandung`.
3. [Database] Queries `User` table for records where `role == PROVIDER` and `account_status == APPROVED`.
4. [Backend] Filters the returned provider lists:
 - Matches the requested material against the provider's `provider_config.materials` array.
 - Sorts results prioritising local proximity (city/country match).
5. [Backend] Returns matching providers list.
6. [Customer] Selects a provider, configures shipping details, and clicks “Submit Print Order”.
7. [Frontend] Dispatches `POST /print/orders` with:
 - `provider_id`, `shipping_address`, `courier_name`, `items` (model IDs and specifications like layer heights or fill densities).
8. [Database] Looks up target provider and model records.
9. [Backend] Verifies the provider exists and is active.
 - If invalid: returns 400 Bad Request. Flow ends.
10. [Database] Inserts a new `Order` record:
 - `type = PRINT_JOB`
 - `status = PENDING`
 - `shipping_address` written as JSON
 - `items` created with `print_status = PENDING`
11. [Backend] Returns 201 Created.
12. [Frontend] Directs the user to the print order tracking page.

29.4 Decision Points Summary

Decision	Path/Condition	Outcome
Provider valid and active?	No	Return 400 Bad Request, stop
Proximity matches?	Yes	Bubble provider to the top of search list

29.5 Key Implementation Details

- **Print vs. Asset Orders:** In contrast to digital asset orders which immediately generate a Midtrans payment token, a `PRINT_JOB` order is saved in the database first to let the provider calculate custom pricing (shipping, handling, scaling adjustments) or accept/reject the job before final billing occurs.
- **Provider Specifications:** The provider configurations (capabilities, equipment, localized pricing) are stored in the database as a single structured `Json` field (`provider_config`), which makes it easy to add custom capabilities without needing migrations.

30 Activity Diagram - Resolve Abuse Reports

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_resolve_re

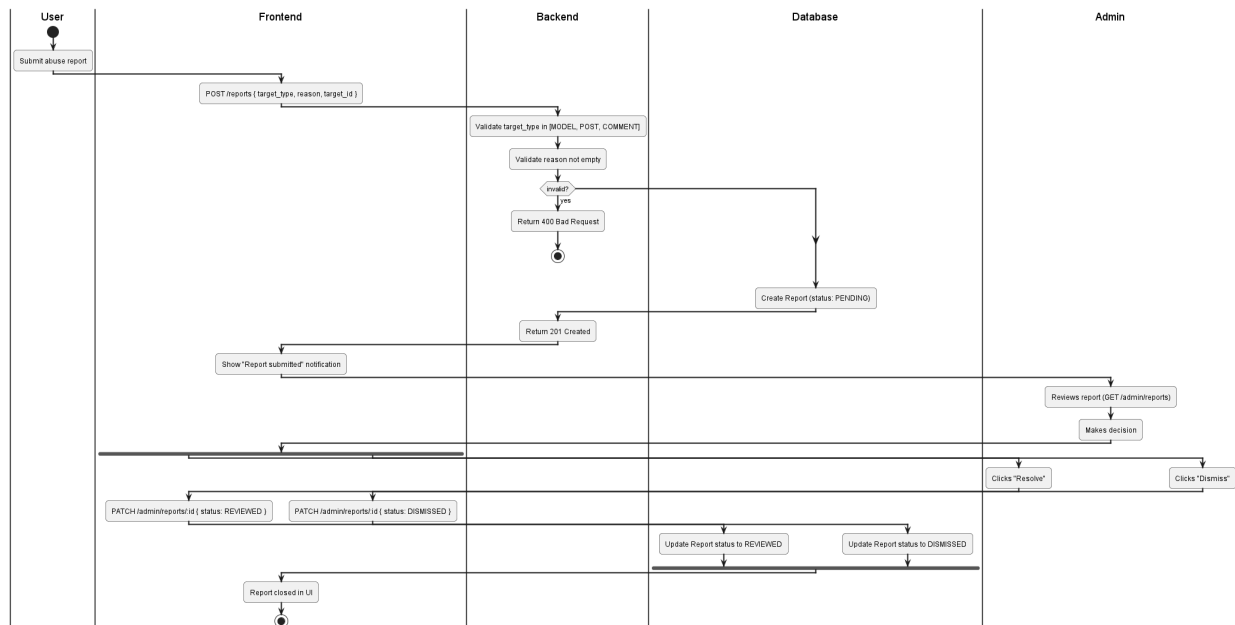


Figure 30: Activity Diagram - Resolve Abuse Reports

30.1 Overview

This activity diagram shows the end-to-end report lifecycle from user submission to admin resolution, using parallel forks to represent the two resolution outcomes: Resolve or Dismiss.

30.2 Swim Lanes

Lane	Responsibility
User	Submits the abuse report
Frontend	Manages the report form and confirmation
Backend	Validates and persists
Database	Stores report and updates status
Admin	Reviews and resolves reports

30.3 Process Flow

User Submission Phase:

1. [User] Clicks “Report” on a model, post, or comment.
2. [Frontend] Sends `POST /reports { target_type, reason, target_id }`.
3. [Backend] Validates:
 - `target_type` must be in `[MODEL, POST, COMMENT]`.
 - `reason` must not be empty.
 - If invalid: returns 400. Flow ends.
4. [Database] Creates `Report { status: PENDING }`.
5. [Backend] Returns 201 Created.
6. [Frontend] Shows “Report submitted” notification to user.

Admin Resolution Phase:

7. [Admin] Reviews the reports list at `GET /admin/reports`.
8. [Admin] Makes a resolution decision.

Fork:

Resolve path:

9A. [Admin] Clicks “Resolve”. 10A. [Frontend] Sends `PATCH /admin/reports/:id { status: REVIEWED }`. 11A. [Database] Updates `Report.status = REVIEWED`.

Dismiss path:

9B. [Admin] Clicks “Dismiss”. 10B. [Frontend] Sends `PATCH /admin/reports/:id { status: DISMISSED }`. 11B. [Database] Updates `Report.status = DISMISSED`.

Rejoin:

12. [Frontend] Report closed in UI.

30.4 Decision Points Summary

Decision	Yes Path	No Path
<code>target_type</code> valid?	Continue	Return 400
<code>reason</code> present?	Continue	Return 400
Admin resolves?	REVIEWED path	-
Admin dismisses?	DISMISSED path	-

30.5 Notes

- Report resolution does not automatically take action on the reported content. If the admin decides the report is valid, they must separately delete the content or ban the user.
- The `Report_Status` enum has three values: `PENDING` (submitted, awaiting review), `REVIEWED` (admin has seen and resolved it), and `DISMISSED` (admin has seen and rejected it as invalid).

31 Activity Diagram - Search Models & Filters

UML Type: Activity Diagram Category: Catalog Source: docs/diagrams/activity/catalog/activity_search_

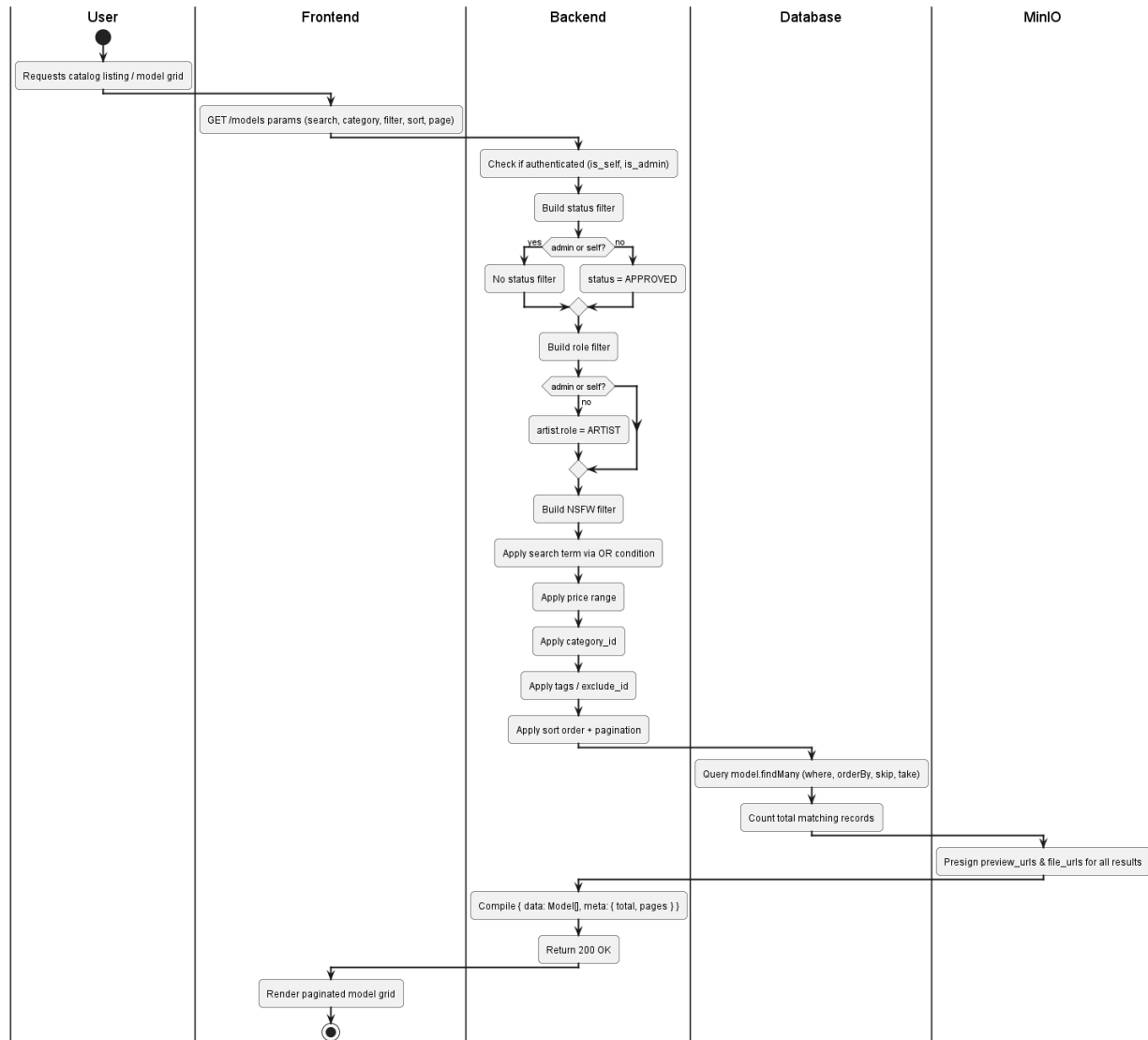


Figure 31: Activity Diagram - Search Models & Filters

31.1 Overview

This activity diagram details the WHERE clause construction logic for the catalog listing endpoint, showing exactly how each filter parameter affects the database query.

31.2 Swim Lanes

Lane	Responsibility
User	Initiates a catalog request
Frontend	Sends the parameterized request
Backend	Constructs the query dynamically
Database	Executes the query
MinIO	Signs preview URLs

31.3 Process Flow

1. [User] Requests the catalog or model grid.
2. [Frontend] Sends GET /models with params: `search`, `category`, `filter`, `sort`, `page`.
3. [Backend] Checks caller identity (`is_admin`, `is_self`).

Status filter construction: 4. [Backend] If admin or self-view: no status filter (all statuses visible). Otherwise: `status = APPROVED`.

Role filter construction: 5. [Backend] If not admin and not self: adds `artist.role = ARTIST` to ensure only artist-uploaded models are shown.

Other filters applied in sequence: 6. [Backend] Build NSFW filter based on `show_nsfw` parameter. 7. [Backend] Apply search term via OR across title, description, tags, category. 8. [Backend] Apply price range (`min_price` / `max_price`). 9. [Backend] Apply `category_id`. 10. [Backend] Apply tags inclusion / `exclude_id`. 11. [Backend] Apply sort order and pagination (`skip`, `take`).

Execution: 12. [Database] Executes `model.findMany(where, orderBy, skip, take)`. 13. [Database] Counts total matching records for pagination metadata. 14. [MinIO] Presigns `preview_url` and `file_url` for each result. 15. [Backend] Compiles `{ data: Model[], meta: { total, pages } }`. 16. [Backend] Returns 200 OK. 17. [Frontend] Renders paginated model grid.

31.4 Decision Points Summary

Decision	Yes Path	No Path
Caller is admin or self?	No status filter	<code>status = APPROVED</code>
Not admin and not self?	Add artist role filter	No role filter

31.5 Notes

- The filter chain is additive. Each parameter appends to the WHERE clause.
- The NSFW filter respects both the user's personal preference (`show_nsfw`) and the model's `is_nsfw` flag.
- The `exclude_id` parameter allows the frontend to exclude the currently viewed model from “similar models” sections on detail pages.

32 Activity Diagram - Stats Aggregation

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_stats_aggr

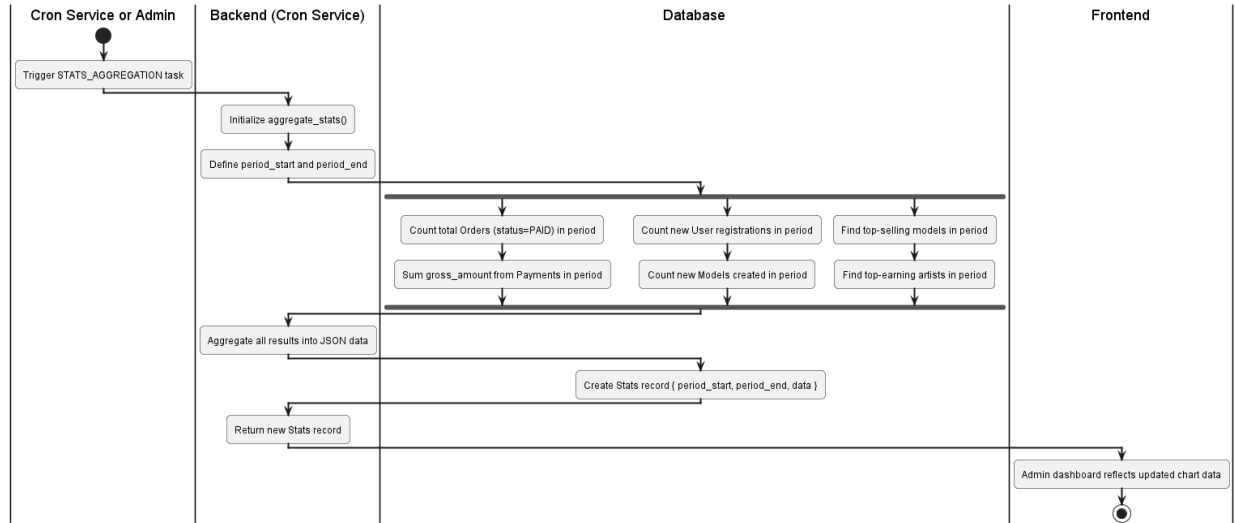


Figure 32: Activity Diagram - Stats Aggregation

32.1 Overview

This activity diagram describes the background stats aggregation process, which computes platform-level metrics for a defined time period and stores them in the **Stats** table for dashboard display.

32.2 Swim Lanes

Lane	Responsibility
Cron Service or Admin	Triggers the aggregation task
Backend (Cron Service)	Orchestrates the computation
Database	Executes parallel count/sum queries and stores results
Frontend	Reflects updated data in the admin chart

32.3 Process Flow

1. **[Cron Service or Admin]** Triggers `STATS_AGGREGATION` (either via scheduled cron or manual dashboard button).
2. **[Backend (Cron Service)]** Calls `initialize aggregate_stats()`.
3. **[Backend (Cron Service)]** Defines `period_start` and `period_end` timestamps.
4. **[Database]** Executes parallel aggregation queries:

Fork A:

- Count total `Order` records with `status = PAID` in the period.
- Sum `gross_amount` from all `Payment` records in the period.

Fork B:

- Count new `User` registrations in the period.
- Count new `Model` records created in the period.

Fork C:

- Find top-selling models (most `Order_Items` fulfilled in the period).
 - Find top-earning artists (sum of sales by artist in the period).
5. **[Backend (Cron Service)]** Aggregates all results into a single JSON data object.
 6. **[Database]** Creates a new `Stats` record: { `period_start`, `period_end`, `data` }.
 7. **[Backend (Cron Service)]** Returns the new `Stats` record.
 8. **[Frontend]** Admin dashboard chart reflects the updated data on next load.
-

32.4 Decision Points Summary

There are no conditional branches in this flow. The process is always executed fully once triggered.

32.5 Notes

- The parallel fork in the Database lane shows that multiple aggregation queries should ideally run concurrently. In practice, Prisma does not natively parallelize queries in a transaction; a `Promise.all` wrapper would be needed.
- Each `Stats` record is a timestamped snapshot. Multiple records accumulate over time, providing a history of platform metrics. The dashboard displays the most recent record and can chart the history.
- The `data` JSON field structure is application-defined and would include fields like { `total_orders`, `total_revenue`, `new_users`, `new_models`, `top_models`: [], `top_artists`: [] }.
- The cron trigger interval is not specified in the diagram. A reasonable interval for a marketplace would be daily or weekly aggregation.

33 Activity Diagram - Submit Review

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_submit_

33.1 Overview

This activity diagram describes the validation sequence and denormalization updates when a customer submits a review (rating and optional text comment) for a 3D model.

33.2 Swim Lanes

Lane	Responsibility
Customer	Submits the review details
Frontend	Form validation and UI updates
Backend	Runs business logic, verifies purchase, and triggers updates
Database	Persists review, checks constraints, and recalculates averages

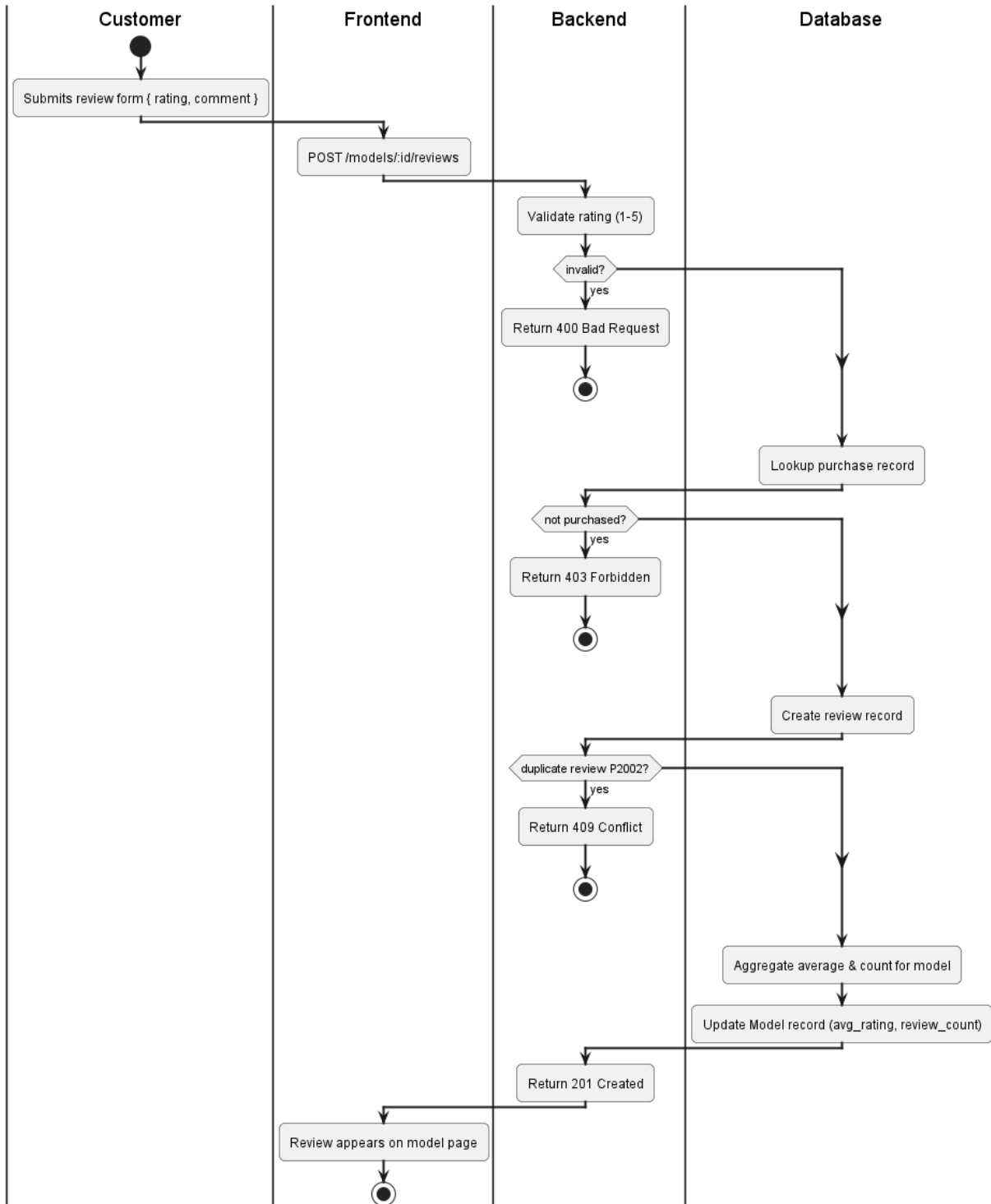


Figure 33: Activity Diagram - Submit Review

33.3 Process Flow

1. [Customer] Fills out the review form (rating between 1-5 and comment text) and clicks submit.
2. [Frontend] Dispatches `POST /models/:id/reviews` with the request body.
3. [Backend] Validates that `rating` is an integer between 1 and 5.
 - If invalid: returns 400 Bad Request. Flow ends.
4. [Database] Looks up the `Purchase` record for the user and model.
5. [Backend] Checks if the model was purchased.
 - If not purchased: returns 403 Forbidden. Flow ends.
6. [Database] Attempts to insert a new `Review` record.
7. [Database] Checks for duplicate constraint violations (Prisma unique constraint P2002 on (`user_id`, `model_id`)).
 - If a review already exists: Backend returns 409 Conflict. Flow ends.
8. [Database] Executes aggregation query: computes new count and average rating for the model.
9. [Database] Updates the `Model` record with the updated `avg_rating` and `review_count`.
10. [Backend] Returns 201 Created.
11. [Frontend] Appends the review to the model detail page review list.

33.4 Decision Points Summary

Decision	Path/Condition	Outcome
Rating is valid (1-5)?	No	Return 400, stop
Purchased by user?	No	Return 403, stop
Already reviewed?	Yes (P2002)	Return 409, stop

33.5 Key Implementation Details

- **Denormalization Strategy:** To avoid executing heavy database aggregations (`AVG` and `COUNT`) on every catalog listing query, the system keeps denormalized `avg_rating` and `review_count` fields directly on the `Model` record. These are updated atomically whenever a new review is successfully persisted.
- **Purchase Restriction:** Verified purchase validation is mandatory. Users cannot review models they do not own.

34 Activity Diagram - Submit Role Application

UML Type: Activity Diagram **Category:** Catalog (Profile Management) **Source:** docs/diagrams/activity/catalog/activity_submit_role_app.puml

34.1 Overview

This activity diagram describes the process of submitting a role upgrade application (to become an Artist or Provider). In the UCD, this is an `<<extend>>` of the “Manage user profile” use case.

34.2 Swim Lanes

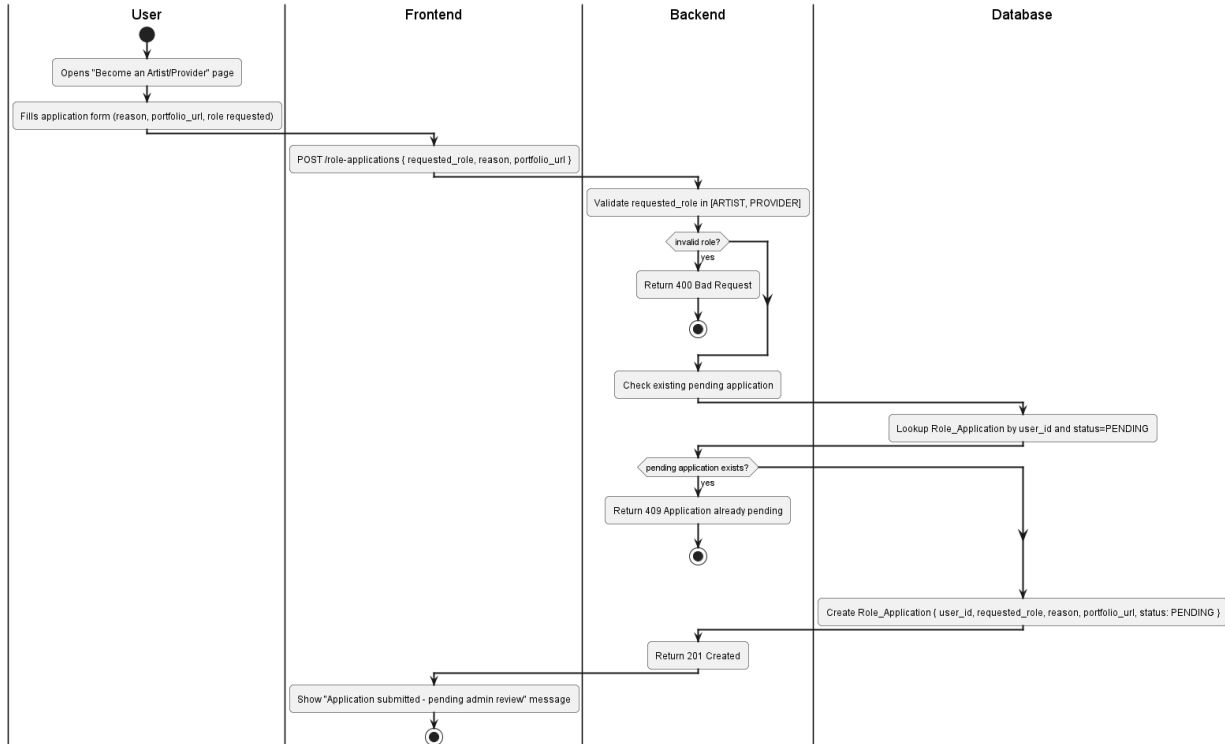


Figure 34: Activity Diagram - Submit Role Application

Lane	Responsibility
User	Fills and submits the role application form
Frontend	Submits the application
Backend	Validates inputs and checks for duplicates
Database	Stores the pending application

34.3 Process Flow

1. [User] Opens the “Become an Artist/Provider” page.
2. [User] Fills in the application form: reason, portfolio URL, requested role.
3. [Frontend] Dispatches `POST /role-applications { requested_role, reason, portfolio_url }`.
4. [Backend] Validates `requested_role` is `ARTIST` or `PROVIDER`.
 - If invalid: returns 400.
5. [Backend] Checks for an existing pending application.
6. [Database] Looks up `Role_Application` by `user_id` and `status = PENDING`.
7. [Backend] If a pending application exists: returns 409 Conflict.
8. [Database] Creates `Role_Application { user_id, requested_role, reason, portfolio_url, status: PENDING }`.
9. [Backend] Returns 201 Created.
10. [Frontend] Shows “Application submitted — pending admin review” confirmation.

34.4 Decision Points Summary

Decision	Path/Condition	Outcome
Requested role valid?	No	Return 400 Bad Request
Pending application exists?	Yes	Return 409 Conflict

34.5 Key Implementation Details

- **Admin Review:** The submitted application enters the admin moderation queue, where it appears alongside the `activity_admin_user_moderation` flow for approval or rejection.
- **UCD Relationship:** This is a profile extension (`<<extend>>`) — applying for a role is optional and triggered from within the Manage user profile flow.

35 Activity Diagram - Suspend Malicious Nodes

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_suspend_no

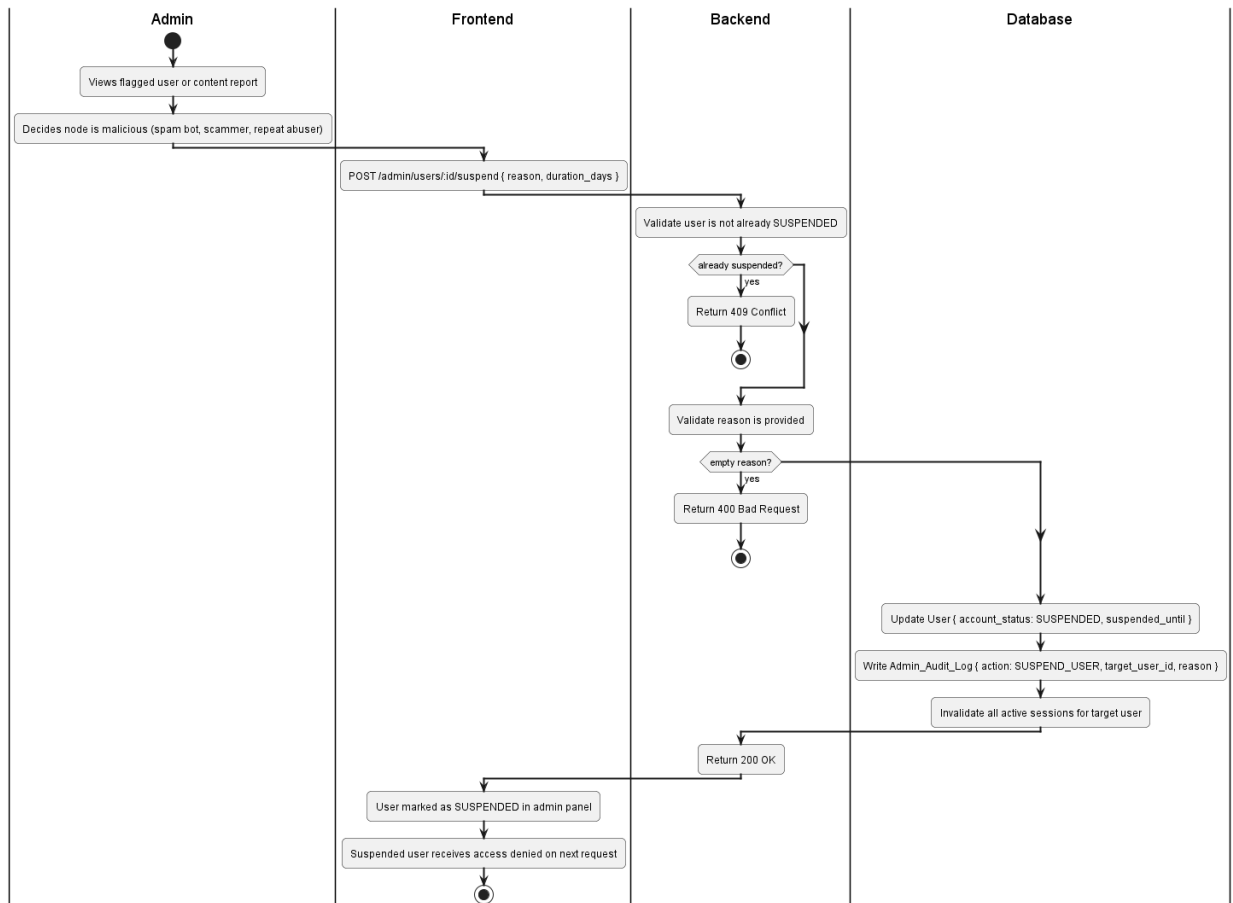


Figure 35: Activity Diagram - Suspend Malicious Nodes

35.1 Overview

This activity diagram describes the process by which an Admin suspends a malicious user account (spam bot, scammer, or repeat policy abuser). This corresponds to the “Suspend malicious nodes” use case in the UCD.

35.2 Swim Lanes

Lane	Responsibility
Admin	Reviews flagged user and initiates suspension
Frontend	Submits the suspension request
Backend	Validates inputs and executes suspension
Database	Updates user status, writes audit log, and invalidates sessions

35.3 Process Flow

1. **[Admin]** Views a flagged user or content report and identifies malicious behavior.
2. **[Admin]** Decides the account is a malicious node (spam bot, scammer, repeat abuser).
3. **[Frontend]** Dispatches `POST /admin/users/:id/suspend { reason, duration_days }`.
4. **[Backend]** Validates the user is not already `SUSPENDED`.
 - If already suspended: returns 409 Conflict.
5. **[Backend]** Validates the `reason` field is provided.
 - If empty: returns 400 Bad Request.
6. **[Database]** Updates `User { account_status: SUSPENDED, suspended_until }`.
7. **[Database]** Writes `Admin_Audit_Log { action: SUSPEND_USER, target_user_id, reason }`.
8. **[Database]** Invalidates all active sessions for the target user.
9. **[Backend]** Returns 200 OK.
10. **[Frontend]** User marked as `SUSPENDED` in the admin panel.
11. Suspended user receives “access denied” on next request.

35.4 Decision Points Summary

Decision	Path/Condition	Outcome
Already suspended?	Yes	Return 409 Conflict
Reason provided?	No	Return 400 Bad Request

35.5 Key Implementation Details

- **Session Invalidation:** All active JWT sessions for the suspended user are invalidated immediately, blocking instant re-access without requiring a new login.
- **Audit Trail:** Every suspension is recorded in `Admin_Audit_Log` with the responsible admin’s ID, the target user, and the reason — ensuring accountability and traceability.
- **Terminology:** “Malicious nodes” in the UCD refers to accounts actively violating platform trust — bots, fraudsters, or serial abusers.

36 Activity Diagram - Sync Social Platform

UML Type: Activity Diagram **Category:** Catalog (Profile Management) **Source:** docs/diagrams/activity/catalog/activity_sync_social.puml

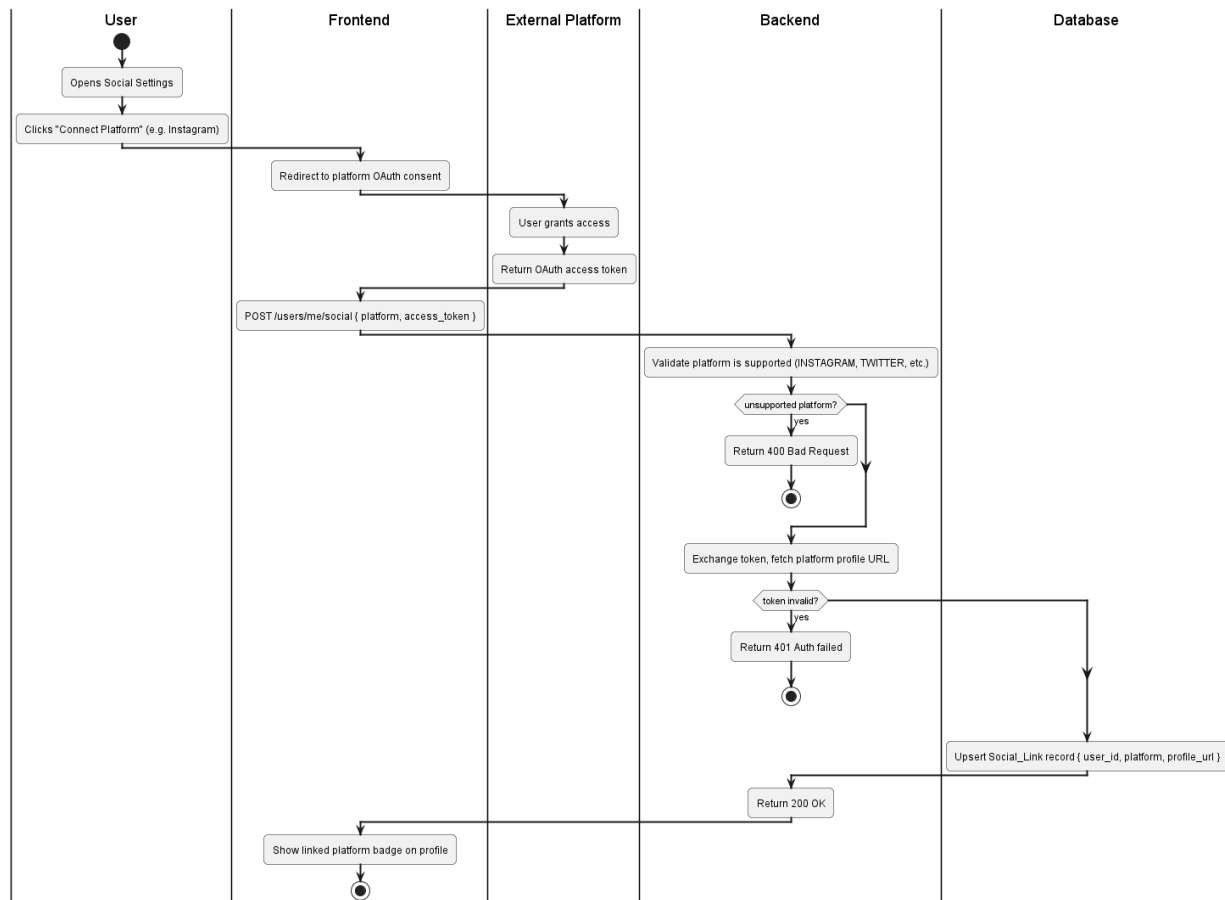


Figure 36: Activity Diagram - Sync Social Platform

36.1 Overview

This activity diagram describes the flow for linking an external social media account (e.g. Instagram, Twitter/X) to a user's 3Dex profile. In the UCD, this is an <<extend>> of the "Manage user profile" use case.

36.2 Swim Lanes

Lane	Responsibility
User	Initiates the social link process
Frontend	Redirects to platform OAuth and submits the token
External Platform	Handles OAuth authorization and returns an access token
Backend	Validates the platform and exchanges the token

Lane	Responsibility
Database	Upserts the social link record

36.3 Process Flow

1. [User] Opens Social Settings and clicks “Connect Platform” (e.g. Instagram).
2. [Frontend] Redirects to the external platform’s OAuth consent page.
3. [External Platform] User grants access. Returns OAuth access token.
4. [Frontend] Dispatches `POST /users/me/social { platform, access_token }`.
5. [Backend] Validates that the platform is a supported type (INSTAGRAM, TWITTER, etc.).
 - If unsupported: returns 400 Bad Request.
6. [Backend] Exchanges the token and fetches the user’s profile URL from the platform.
 - If token invalid: returns 401 Auth failed.
7. [Database] Upserts `Social_Link` record `{ user_id, platform, profile_url }`.
8. [Backend] Returns 200 OK.
9. [Frontend] Displays the linked platform badge on the user’s public profile.

36.4 Decision Points Summary

Decision	Path/Condition	Outcome
Platform supported?	No	Return 400 Bad Request
Token valid?	No	Return 401 Auth failed

36.5 Key Implementation Details

- **OAuth Flow:** The external platform handles the auth — 3Dex only receives and validates the resulting token. The platform profile URL is stored, not the raw token.
- **UCD Relationship:** This is a profile extension (`<<extend>>`) — syncing social accounts is optional and triggered from within the Manage user profile flow.

37 Activity Diagram - Toggle Dexie On/Off

UML Type: Activity Diagram **Category:** Dexie **Source:** docs/diagrams/activity/dexie/activity_toggle_dexi

37.1 Overview

This activity diagram describes the flow for enabling or disabling the Dexie AI assistant for a user’s account. In the UCD, this is an `<<extend>>` of the “Manage user profile” use case and an `<<include>>` of the “Display contextual tagline” use case.

37.2 Swim Lanes

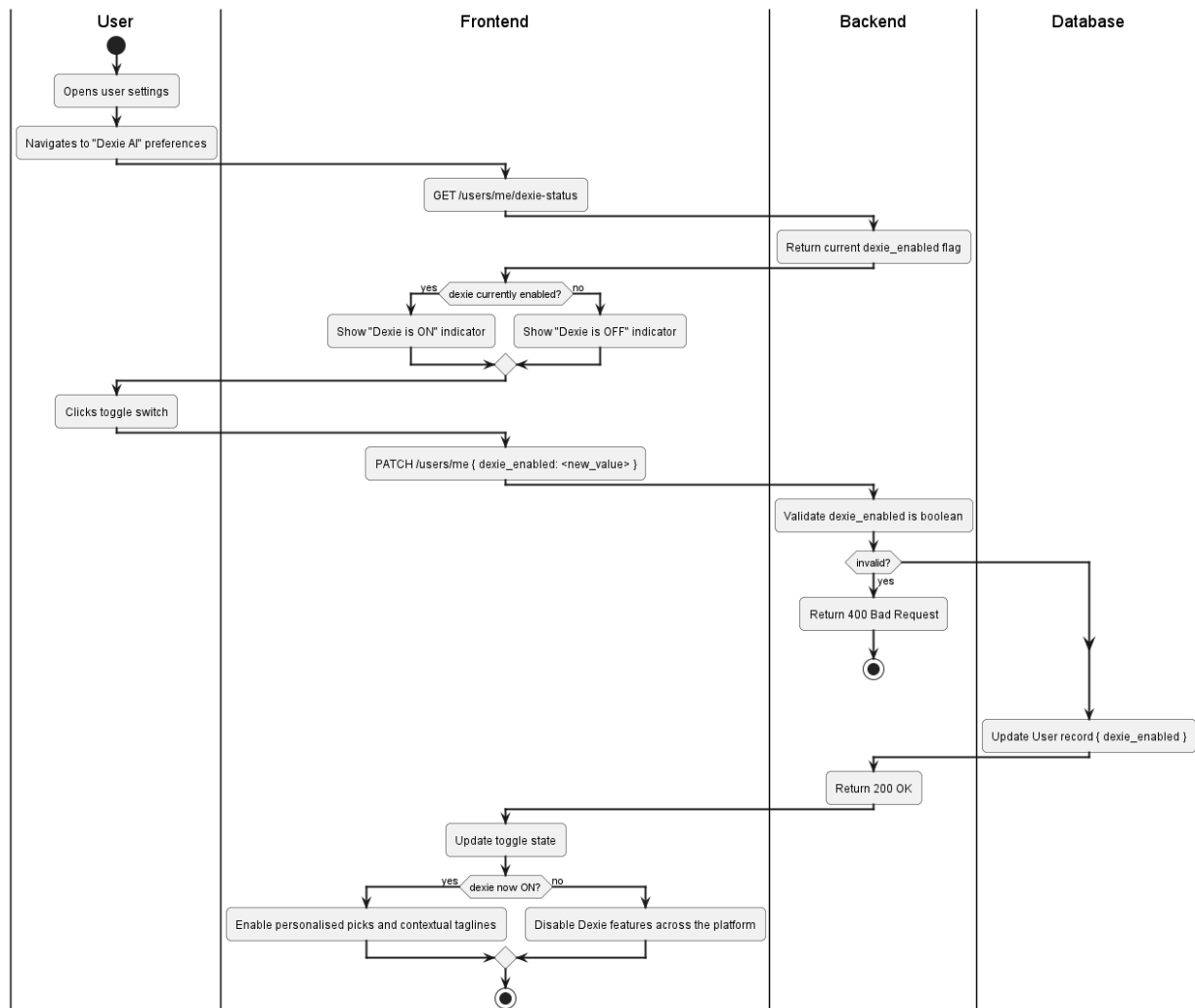


Figure 37: Activity Diagram - Toggle Dexie On/Off

Lane	Responsibility
User	Navigates to Dexie preferences and clicks the toggle
Frontend	Fetches current state, submits change, and updates UI
Backend	Validates and persists the preference
Database	Updates the user's Dexie enabled flag

37.3 Process Flow

1. [User] Opens User Settings and navigates to “Dexie AI” preferences.
2. [Frontend] Dispatches `GET /users/me/dexie-status`.
3. [Backend] Returns current `dexie_enabled` flag.
4. [Frontend] Renders the toggle in the correct initial state (ON/OFF).
5. [User] Clicks the toggle switch.
6. [Frontend] Dispatches `PATCH /users/me { dexie_enabled: <new_value> }`.
7. [Backend] Validates `dexie_enabled` is a boolean.
 - If invalid: returns 400 Bad Request.
8. [Database] Updates `User` record `{ dexie_enabled }`.
9. [Backend] Returns 200 OK.
10. [Frontend] Updates toggle state.
 - **If now ON:** Enables personalised picks and contextual taglines.
 - **If now OFF:** Disables all Dexie AI features across the platform.

37.4 Decision Points Summary

Decision	Path/Condition	Outcome
<code>dexie_enabled</code> is boolean?	No	Return 400 Bad Request
Dexie now ON?	Yes	Enable picks + taglines
Dexie now OFF?	Yes	Disable all Dexie features

37.5 Key Implementation Details

- **Feature Gate:** The `dexie_enabled` flag acts as a user-level feature gate. When disabled, both the “Display contextual tagline” and “Generate personalised picks” flows return early without calling the Dexie AI service.
- **UCD Relationship:** “Toggle dexie on/off” is included in the “Display contextual tagline” flow (<<include>>) and extends “Manage user profile” (<<extend>>), making it a cross-cutting preference that affects multiple downstream use cases.

38 Activity Diagram - Toggle Content Visibility (NSFW)

UML Type: Activity Diagram **Category:** Catalog (Profile Management) **Source:** docs/diagrams/activity/catalog/activity_toggle_nsfw.puml

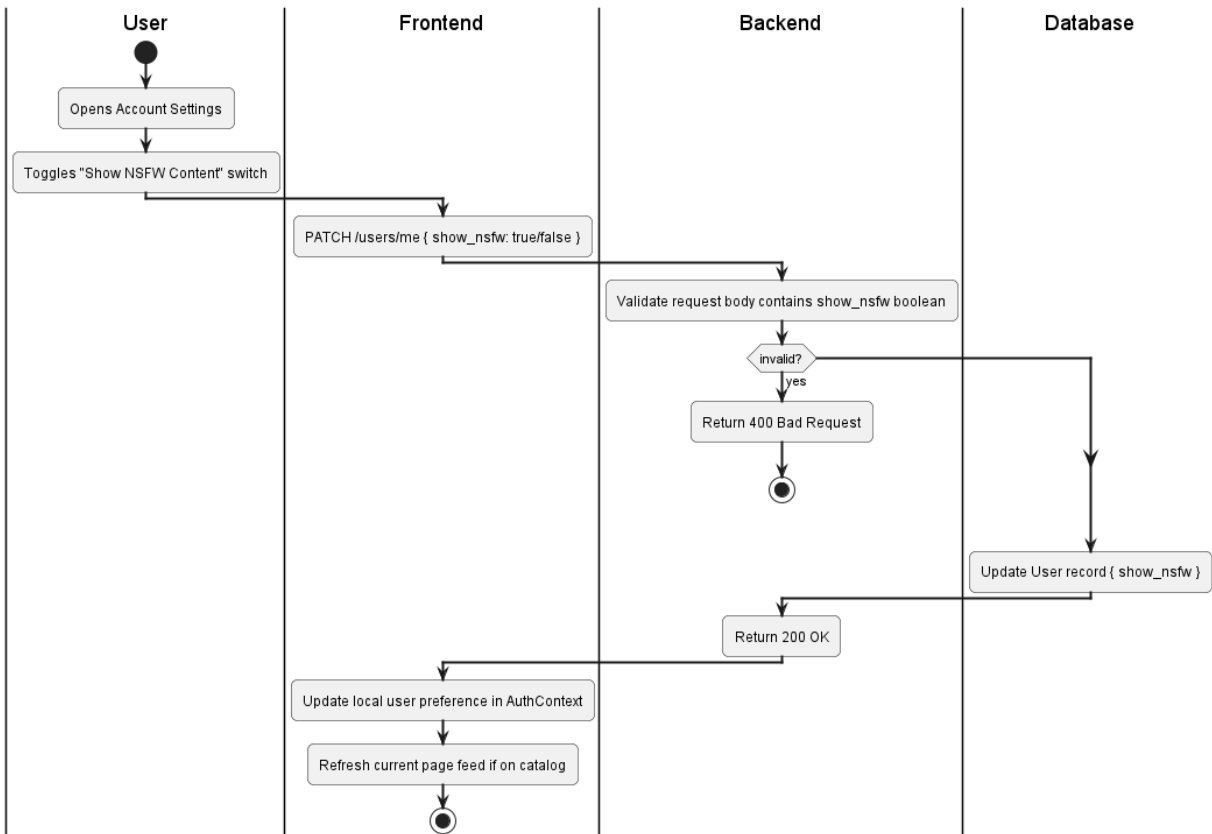


Figure 38: Activity Diagram - Toggle Content Visibility (NSFW)

38.1 Overview

This activity diagram describes the flow for toggling the NSFW (Not Safe For Work) content visibility preference on a user's account. In the UCD, this is an <<extend>> of the "Manage user profile" use case.

38.2 Swim Lanes

Lane	Responsibility
User	Triggers the NSFW toggle
Frontend	Submits the preference and updates the local state
Backend	Validates and persists the preference change
Database	Updates the user record

38.3 Process Flow

1. [User] Opens Account Settings and navigates to the Content preferences section.
2. [User] Toggles the "Show NSFW Content" switch.
3. [Frontend] Dispatches `PATCH /users/me { show_nsfw: true/false }`.
4. [Backend] Validates the request body contains a boolean `show_nsfw` field.
 - If invalid: returns 400 Bad Request.
5. [Database] Updates User record: `{ show_nsfw }`.
6. [Backend] Returns 200 OK.
7. [Frontend] Updates local user preference in `AuthContext`.
8. [Frontend] If currently on the catalog, refreshes the feed to apply/remove the NSFW filter.

38.4 Decision Points Summary

Decision	Path/Condition	Outcome
Request body valid?	No	Return 400 Bad Request

38.5 Key Implementation Details

- **Server-Side Enforcement:** The NSFW preference is enforced server-side. The catalog filter (`GET /models`) reads `show_nsfw` from the session and filters results accordingly — preventing client-side manipulation of content visibility.
- **UCD Relationship:** This is a profile extension (<<extend>>) — toggling NSFW visibility is optional and triggered from within the Manage user profile flow.

39 Activity Diagram - Unlock Downloads

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_unlock_

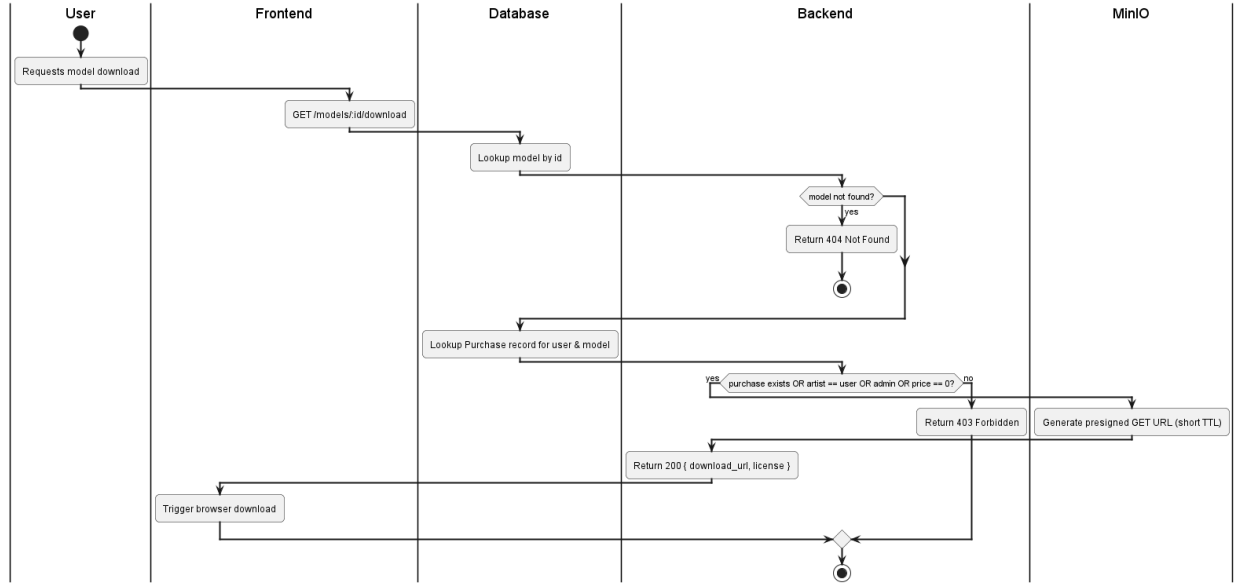


Figure 39: Activity Diagram - Unlock Downloads

39.1 Overview

This activity diagram describes the validation sequence and delivery mechanisms of the model download process. It focuses on the server-side checks that must pass before a secure S3 presigned URL is served to the client browser.

39.2 Swim Lanes

Lane	Responsibility
User	Requests model download
Frontend	Issues download API call and handles browser trigger
Backend	Runs access rules and generates short-lived URLs
Database	Performs model lookup and purchase verification
MinIO	Issues timed presigned download URL

39.3 Process Flow

1. [User] Clicks the “Download” trigger on the model page.
2. [Frontend] Sends GET /models/{id}/download with user credentials.
3. [Database] Looks up the model by its unique ID.
4. [Backend] Checks if the model exists.
 - If not found: returns 404. Flow ends.
5. [Database] Queries for a valid Purchase record matching the requesting user_id and target model_id.
6. [Backend] Evaluates download eligibility based on the following OR rules:
 - Does a validated Purchase record exist in the database?
 - Is the current user_id equal to the model’s artist_id (owner check)?
 - Does the user possess the ADMIN role?
 - Is the model’s listing price equal to 0 (free asset)?

7. **[Backend]** Branching:
 - **No (none of the above are true):** Returns 403 Forbidden. Flow ends.
 - **Yes (at least one is true):**
 1. **[MinIO]** Backend requests MinIO to generate a short-lived presigned GET URL (short TTL).
 2. **[Backend]** Returns 200 OK along with { `download_url`, `license` }.
 3. **[Frontend]** Intercepts the payload and triggers a native browser download pointing to the generated presigned S3 URL.

39.4 Decision Points Summary

Decision	Path/Condition	Outcome
Model exists?	No	Return 404, stop
Eligible? (Purchase, Owner, Admin, or Free)	Yes	Call MinIO, generate presigned URL, return 200
Eligible?	No	Return 403 Forbidden, stop

39.5 Key Implementation Details

- **Security Gatekeeping:** Access verification is performed strictly on the Backend prior to generating any S3 URLs. Simply having access to the client model page does not bypass this gate.
- **Short TTL URLs:** The S3 URL uses a very short Time-To-Live (e.g., 5 to 15 minutes) to ensure that even if the link is intercepted or shared, it becomes inactive shortly after generation.
- **Free Models:** The check `price == 0` allows any logged-in user to download free assets immediately, bypassing the need for a `Purchase` record.

40 Activity Diagram - Update Shipping Address

UML Type: Activity Diagram **Category:** Catalog (Profile Management) **Source:** docs/diagrams/activity/catalog/activity_update_shipping.puml

40.1 Overview

This activity diagram describes the flow for updating a user's saved shipping address. In the UCD, this is an <<extend>> of the "Manage user profile" use case.

40.2 Swim Lanes

Lane	Responsibility
User	Opens account settings and edits address
Frontend	Submits the address update
Backend	Validates fields and format
Database	Upserts the shipping address record

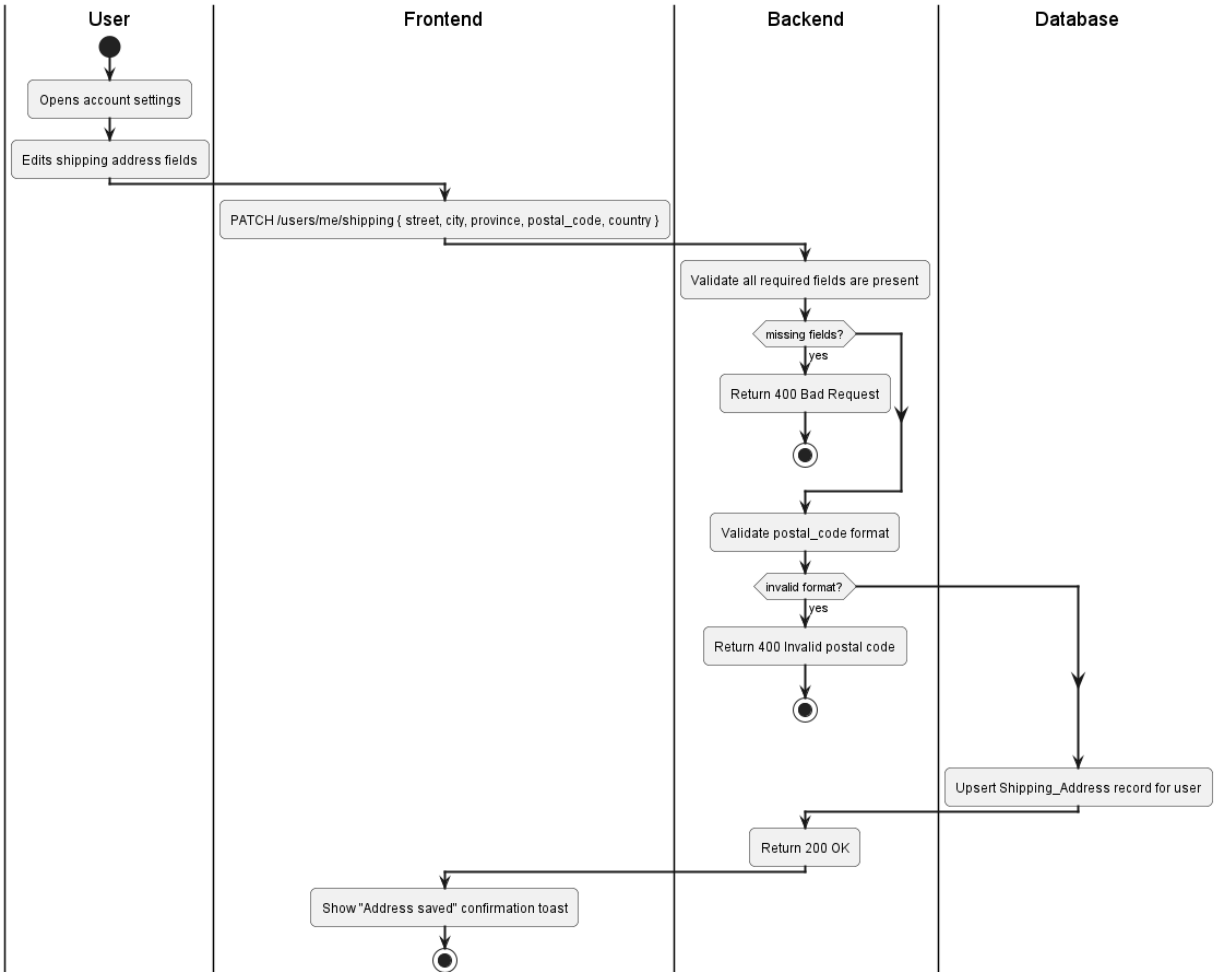


Figure 40: Activity Diagram - Update Shipping Address

40.3 Process Flow

1. [User] Opens account settings and navigates to shipping address section.
 2. [User] Edits shipping address fields (street, city, province, postal code, country).
 3. [Frontend] Dispatches `PATCH /users/me/shipping { street, city, province, postal_code, country }`.
 4. [Backend] Validates all required fields are present.
 - If missing: returns 400 Bad Request.
 5. [Backend] Validates the `postal_code` format.
 - If invalid format: returns 400.
 6. [Database] Upserts `Shipping_Address` record for the user.
 7. [Backend] Returns 200 OK.
 8. [Frontend] Displays “Address saved” confirmation toast.
-

40.4 Decision Points Summary

Decision	Path/Condition	Outcome
Required fields present?	No	Return 400 Bad Request
Postal code format valid?	No	Return 400 Invalid postal code

40.5 Key Implementation Details

- **Upsert Pattern:** Each user has one canonical shipping address. The upsert operation creates it on first save or overwrites it on subsequent updates, avoiding duplicate address records.
- **UCD Relationship:** This is a profile extension (`<<extend>>`) — the shipping address is optional and the user triggers it conditionally from within their profile management flow.

41 Activity Diagram - Update Tracking Info

UML Type: Activity Diagram **Category:** Print **Source:** docs/diagrams/activity/print/activity_update_tracking

41.1 Overview

This activity diagram describes the flow for updating delivery tracking information on a print order. The `<<external>>` **Delivery service** actor in the UCD triggers this update via a webhook when shipment status changes.

41.2 Swim Lanes

Lane	Responsibility
<> Delivery service	Sends webhook on status change
Backend	Validates webhook signature and processes the update
Database	Updates the order and creates a notification
Frontend	Reflects updated status on the customer's order page

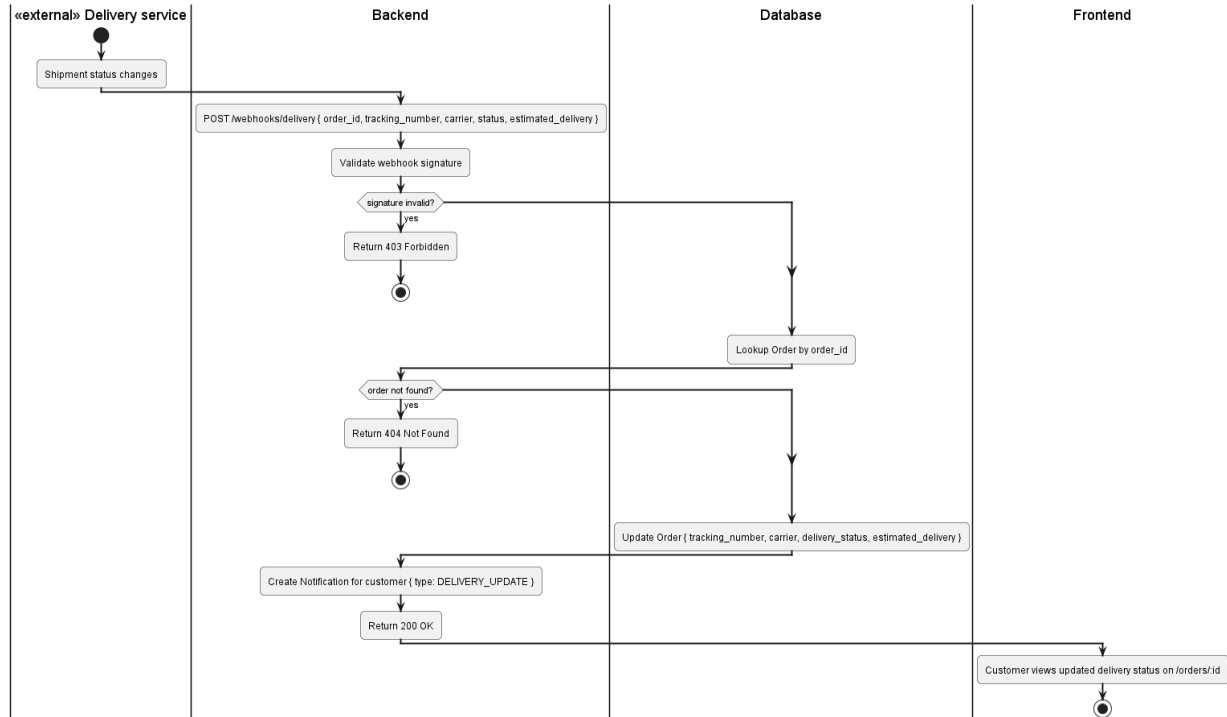


Figure 41: Activity Diagram - Update Tracking Info

41.3 Process Flow

1. [**<> Delivery service**] Detects shipment status change and sends a webhook.
2. [**Backend**] Receives `POST /webhooks/delivery { order_id, tracking_number, carrier, status, estimated_delivery }`.
3. [**Backend**] Validates the webhook signature.
 - If invalid: returns 403 Forbidden.
4. [**Database**] Looks up Order by `order_id`.
5. [**Backend**] If order not found: returns 404.
6. [**Database**] Updates Order `{ tracking_number, carrier, delivery_status, estimated_delivery }`.
7. [**Backend**] Creates a Notification for the customer `{ type: DELIVERY_UPDATE }`.
8. [**Backend**] Returns 200 OK.
9. [**Frontend**] Customer views the updated delivery status on `/orders/:id`.

41.4 Decision Points Summary

Decision	Path/Condition	Outcome
Webhook signature valid?	No	Return 403 Forbidden
Order found?	No	Return 404 Not Found

41.5 Key Implementation Details

- **External Actor:** The <<external>> `Delivery service` is the UCD actor for this use case — the update is initiated externally, not by the user or provider.
- **Webhook Signature:** To prevent spoofed delivery updates, the webhook payload must carry a valid HMAC signature that the backend verifies against a shared secret.
- **Notification Side Effect:** A push/in-app notification is generated for the customer whenever delivery status changes, keeping them informed without polling.

42 Activity Diagram - Verify Licensing & Terms

UML Type: Activity Diagram Category: Commerce Source: docs/diagrams/activity/commerce/activity_veri

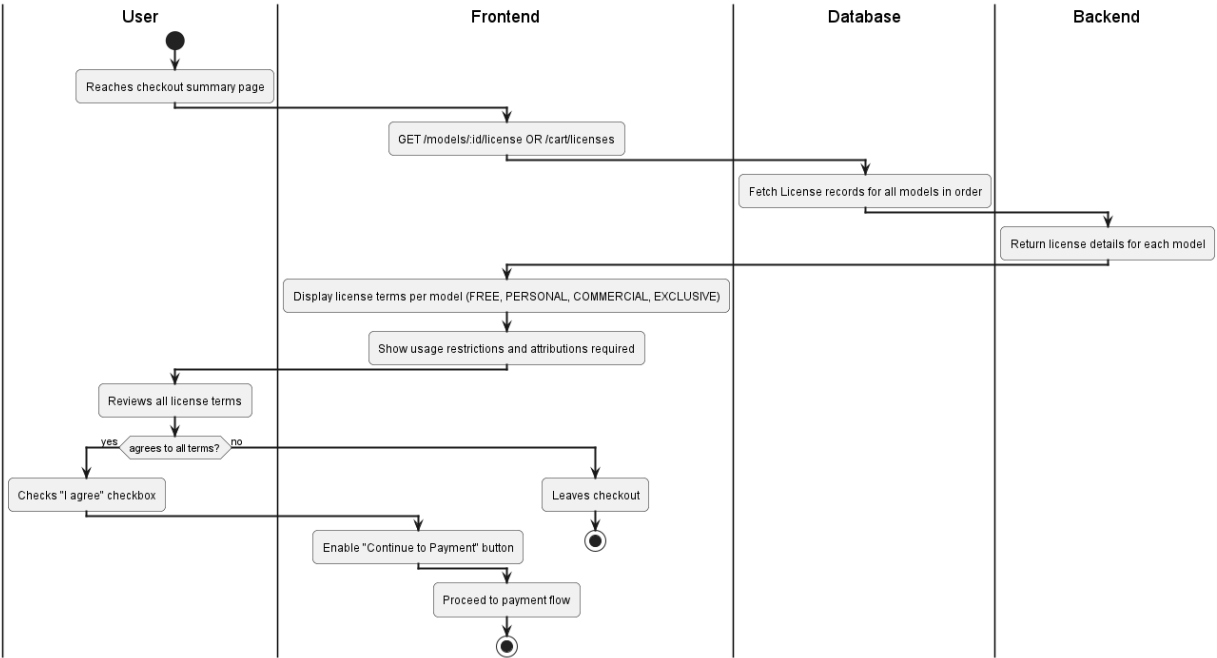


Figure 42: Activity Diagram - Verify Licensing & Terms

42.1 Overview

This activity diagram describes the license verification step embedded within the checkout flow. In the UCD, “Verify licensing & terms” is an <<include>> of “Proceed to checkout” — it is mandatory and always executed before payment.

42.2 Swim Lanes

Lane	Responsibility
User	Reviews and agrees to license terms
Frontend	Fetches license data and gates the payment button
Backend	Returns license details per model

Lane	Responsibility
Database	Provides license records for all items in the order

42.3 Process Flow

1. [User] Reaches the checkout summary page.
2. [Frontend] Fetches license records for all models in the order.
3. [Database] Returns **License** records for each model.
4. [Backend] Returns license details per model.
5. [Frontend] Displays license terms for each model:
 - License type (FREE, PERSONAL, COMMERCIAL, EXCLUSIVE)
 - Usage restrictions
 - Attribution requirements
6. [User] Reviews all license terms.
 - If agrees: checks the “I agree” checkbox. Frontend enables “Continue to Payment” button.
 - If does not agree: leaves checkout.

42.4 Decision Points Summary

Decision	Path/Condition	Outcome
User agrees to all terms?	No	Checkout abandoned

42.5 Key Implementation Details

- **Gate Pattern:** The payment button is disabled until the user explicitly checks the “I agree to all license terms” checkbox. This is a UI-side gate backed by a server-side audit trail of accepted terms.
- **UCD Relationship:** This is an <<include>> of “Proceed to checkout” — not optional. Every checkout must pass through license verification before payment can proceed.

43 Activity Diagram - View Cart

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_view

43.1 Overview

This activity diagram describes the process of viewing, managing, and interacting with a user’s shopping cart. It covers fetching cart contents, adjusting quantities, removing items, and adding new models.

43.2 Swim Lanes

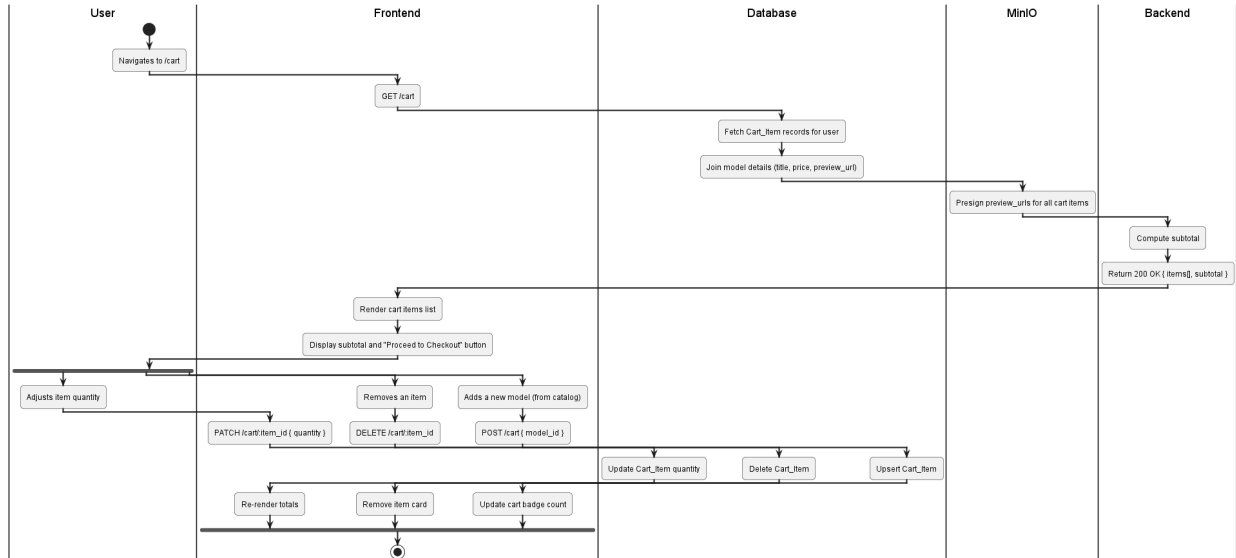


Figure 43: Activity Diagram - View Cart

Lane	Responsibility
User	Navigates to cart and performs item management
Frontend	Fetches cart data and dispatches item mutations
Backend	Computes totals and returns cart data
Database	Stores and retrieves cart item records
MinIO	Generates presigned preview URLs for cart items

43.3 Process Flow

1. [User] Navigates to /cart.
2. [Frontend] Dispatches GET /cart.
3. [Database] Fetches `Cart_Item` records for the user, joined with model details.
4. [MinIO] Presigns `preview_urls` for all cart items.
5. [Backend] Computes subtotal and returns `200 OK { items[], subtotal }`.
6. [Frontend] Renders cart items list, subtotal, and “Proceed to Checkout” button.
7. [User] Interacts with the cart in parallel branches:
 - **Adjust quantity:** `PATCH /cart/:item_id { quantity }` → Database updates → Frontend re-renders totals.
 - **Remove item:** `DELETE /cart/:item_id` → Database deletes → Frontend removes card.
 - **Add new model (from catalog):** `POST /cart { model_id }` → Database upserts → Frontend updates badge count.

43.4 Decision Points Summary

Decision	Path/Condition	Outcome
<i>(none — all cart mutations are validated by type)</i>	—	—

43.5 Key Implementation Details

- **Upsert on Add:** Adding a model already in the cart upserts the `Cart_Item` quantity rather than creating a duplicate entry, preventing double-counting.
- **Real-time Totals:** Subtotal is recomputed client-side on quantity changes for instant feedback, with the server as source of truth on final checkout.

44 Activity Diagram - View Delivery Status

UML Type: Activity Diagram **Category:** Print **Source:** docs/diagrams/activity/print/activity_view_delivery

44.1 Overview

This activity diagram describes the flow for viewing the delivery status of a print order, including tracking information and proof photos provided by the provider. It also covers the customer's ability to confirm delivery, which triggers the provider's payout.

44.2 Swim Lanes

Lane	Responsibility
User	Navigates to order and confirms delivery
Frontend	Fetches and renders order details and status
Backend	Validates access and triggers payout on confirmation
Database	Stores order updates and status changes

44.3 Process Flow

1. [User] Navigates to `/orders/:id` or `/orders`.
 2. [Frontend] Dispatches `GET /orders/:id`.
 3. [Database] Fetches `Order` with `Order_Items` and provider details.
 4. [Backend] Validates existence and authorization.
 - If not found: returns 404.
 - If not owner/provider/admin: returns 403.
 5. [Backend] Returns order details.
 6. [Frontend] Renders:
 - Order summary (items, quantities, totals)
 - Current delivery status badge
 - Tracking number and carrier (if shipped)
 - Estimated delivery date
 - Proof photos uploaded by provider
 7. [User] If delivery has been received:
 - Clicks "Confirm Delivery".
 - [Frontend] Dispatches `PATCH /orders/:id { action: CONFIRM_DELIVERY }`.
 - [Database] Updates `Order { status: COMPLETED }`.
 - [Backend] Triggers payout to provider.
 - [Backend] Returns 200 OK.
 - [Frontend] Shows "Order completed" status.
-

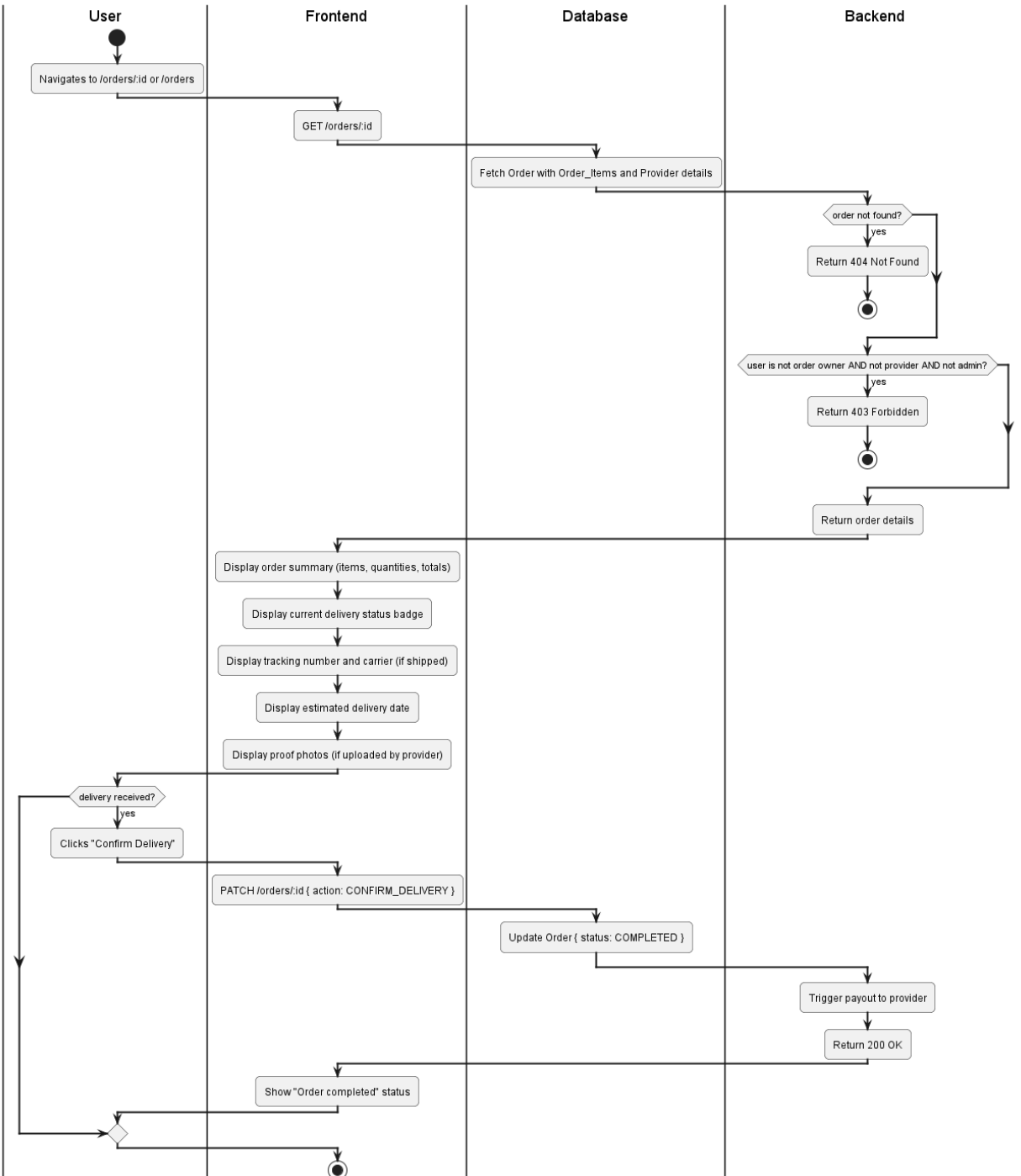


Figure 44: Activity Diagram - View Delivery Status

44.4 Decision Points Summary

Decision	Path/Condition	Outcome
Order found?	No	Return 404
Authorized?	No	Return 403 Forbidden
Delivery received?	Yes	Customer confirms, order completes, payout triggers

44.5 Key Implementation Details

- **Payout Trigger:** The customer’s “Confirm Delivery” action is the final gate before the provider receives their earnings. This protects against premature payouts on undelivered orders.
- **Access Control:** The order detail page is accessible to the customer, the assigned provider, and admins — no other users can view it.

45 Activity Diagram - View Model Details

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_view_model

45.1 Overview

This activity diagram describes the flow when a user or guest navigates to a model’s detail page. It covers model lookup, access control, signed URL generation, and the rendering of the full model detail view including the 3D viewer, license, and reviews.

45.2 Swim Lanes

Lane	Responsibility
User or Guest Frontend	Initiates navigation to a model page Dispatches the API request and renders the detail view
Backend	Validates access, compiles data, and enforces visibility rules
Database	Fetches model record, reviews summary, and artist data
MinIO	Generates presigned URLs for the model file and preview image

45.3 Process Flow

1. [User or Guest] Navigates to `/models/:id`.
2. [Frontend] Dispatches GET `/models/:id`.
3. [Database] Looks up model by ID.
4. [Backend] Checks model existence.
 - If not found: returns 404. Frontend redirects to `/catalog`.

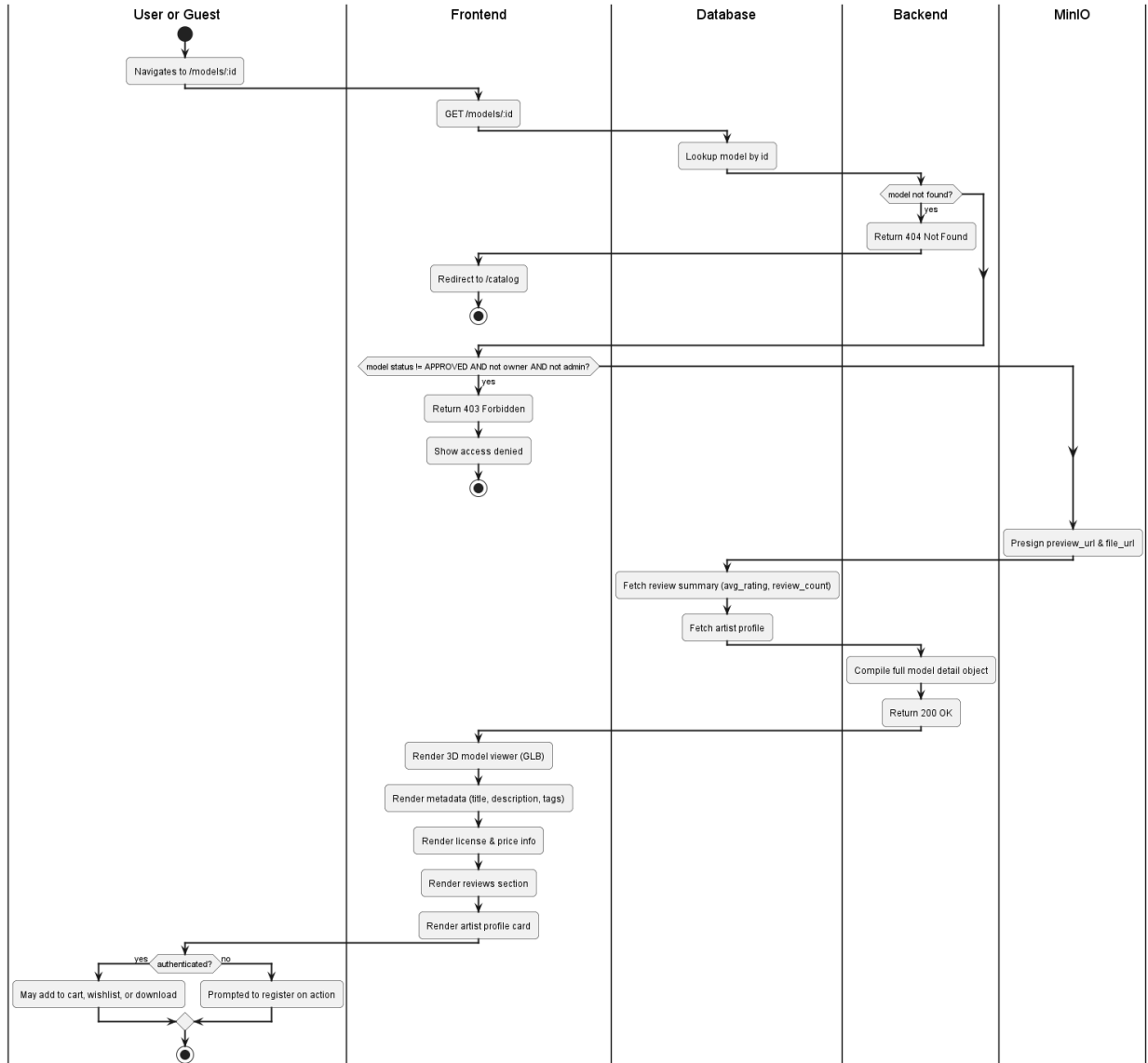


Figure 45: Activity Diagram - View Model Details

5. **[Backend]** Checks visibility:
 - If model status is not **APPROVED** and the requestor is not the owner or admin: returns 403.
6. **[MinIO]** Generates presigned `preview_url` and `file_url`.
7. **[Database]** Fetches review summary (`avg_rating`, `review_count`) and artist profile.
8. **[Backend]** Compiles full model detail object. Returns 200 OK.
9. **[Frontend]** Renders: 3D GLB viewer, metadata, license info, price, reviews section, artist card.
10. **[User or Guest]** If authenticated: can add to cart, wishlist, or download. If not: prompted to register on action.

45.4 Decision Points Summary

Decision	Path/Condition	Outcome
Model found?	No	Return 404, redirect to catalog
Model visible?	PENDING/REJECTED & not owner/admin	Return 403
Authenticated?	No	Gated actions prompt registration

45.5 Key Implementation Details

- **Role-based Visibility:** Pending and rejected models are only visible to their owner and platform admins, ensuring unapproved content doesn't surface publicly.
- **Presigned URLs:** Short-lived signed URLs from MinIO protect model files from being directly scraped or linked without proper authorization.

46 Activity Diagram - View Post

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_view_post.

46.1 Overview

This activity diagram describes the process of loading and rendering the community feed. It focuses on the server-side operations that assemble social posts and dynamically inject user-specific social states (such as whether the current user has liked a post or followed its author).

46.2 Swim Lanes

Lane	Responsibility
User	Requests the community page
Frontend	Issues API calls and renders post widgets
Backend	Assembles search queries and maps custom flags
Database	Queries posts, comments, likes, and follows

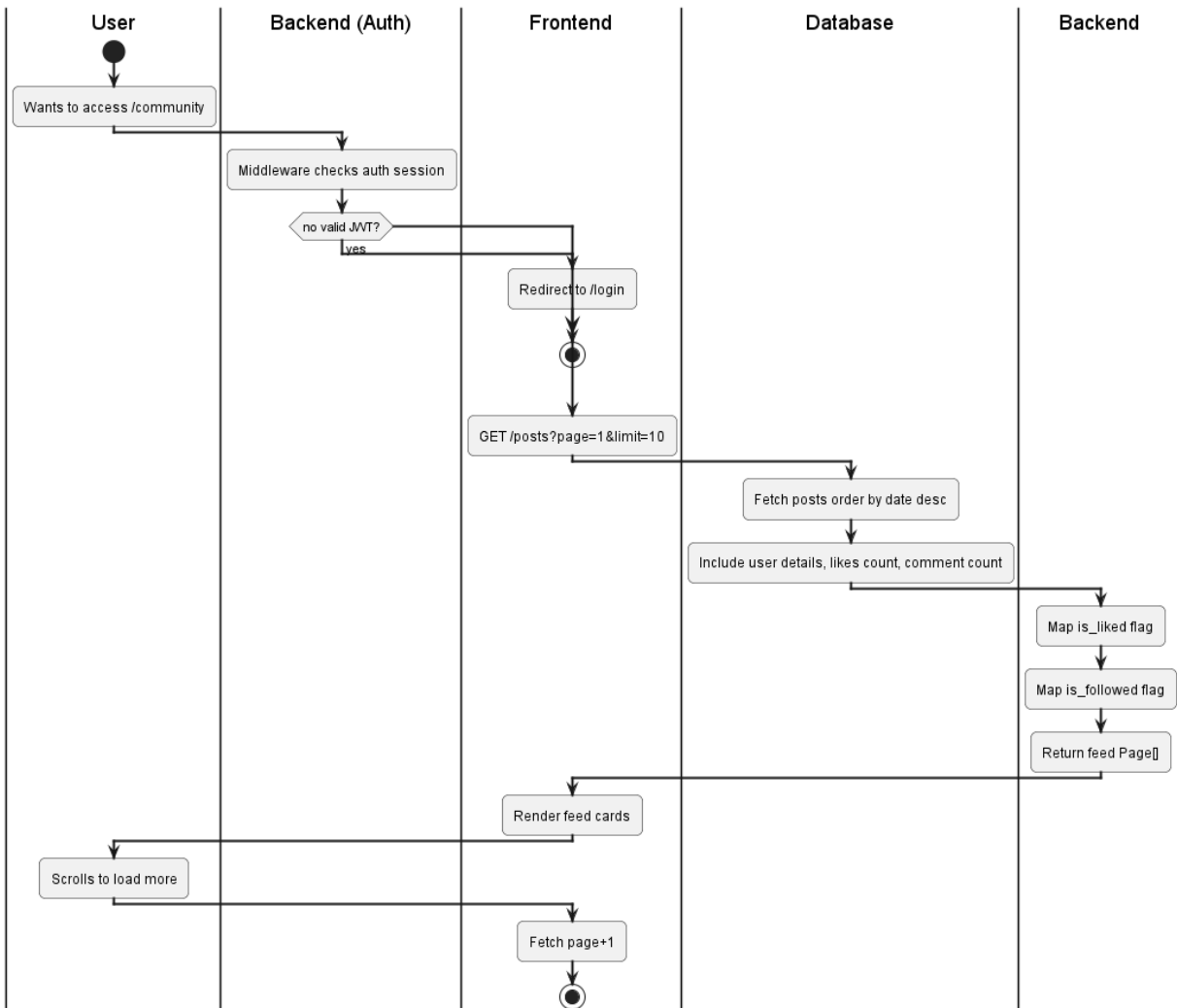


Figure 46: Activity Diagram - View Post

46.3 Process Flow

1. [User] Navigates to the Community Feed page.
2. [Frontend] Dispatches GET `/posts` with the user's session token.
3. [Database] Queries `Post` records, ordering by `created_at` descending.
4. [Database] Fetches related entity counts in the query:
 - Count of `Post_Like` records per post.
 - Count of `Post_Comment` records per post.
5. [Backend] Verifies the requesting user session status (authenticated vs. guest).

Conditional Logic Branching:

- **If User is Authenticated:**
 1. [Database] Queries `Post_Like` for records matching (`user_id`, `post_id`) to determine like status.
 2. [Database] Queries `Follow` for records matching (`follower_id`, `post.user_id`) to determine follow status.
 3. [Backend] Injects dynamic flags per post: `is_liked = true/false`, `is_followed = true/false`.
 - **If User is a Guest (Anonymous):**
 1. [Backend] Sets flags to defaults: `is_liked = false`, `is_followed = false`.
6. [Backend] Returns 200 OK along with the compiled posts payload.
 7. [Frontend] Renders the post cards, using the dynamic flags to style the like (heart) and follow buttons.

46.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	Yes	Query DB for likes and follows, inject dynamic boolean flags
Authenticated?	No	Set like/follow flags to false

46.5 Key Implementation Details

- **Server-Side Mapping:** Mapping flags like `is_liked` and `is_followed` directly on the server reduces frontend logic and prevents a flurry of additional API calls from the client to determine interaction states.
- **Denormalized Optimization:** Although comment and like counts can be calculated dynamically, the backend utilizes denormalized counters (`like_count` and `comment_count`) on the `Post` table to keep list fetches highly performant.

47 Activity Diagram - View User Profile

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_view_user_

47.1 Overview

This activity diagram describes the flow for viewing another user's public profile page. It covers lookup by username, presigned media generation, and conditional rendering based on the viewer's authentication state.

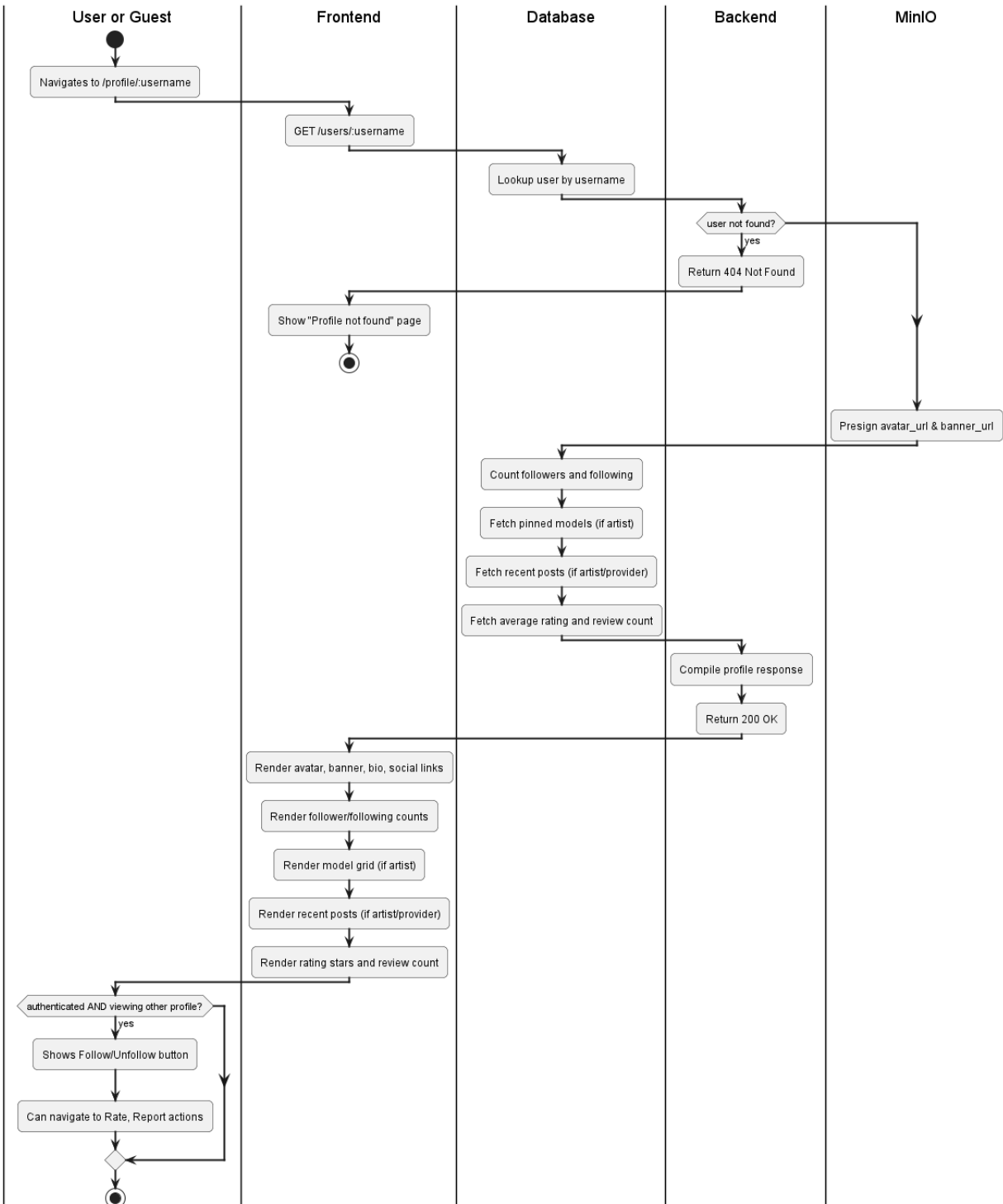


Figure 47: Activity Diagram - View User Profile

47.2 Swim Lanes

Lane	Responsibility
User or Guest	Navigates to a public profile
Frontend	Fetches and renders profile data
Backend	Compiles profile response with access control
Database	Fetches user details, social stats, and content
MinIO	Generates presigned avatar and banner URLs

47.3 Process Flow

1. **[User or Guest]** Navigates to `/profile/:username`.
 2. **[Frontend]** Dispatches `GET /users/:username`.
 3. **[Database]** Looks up user by username.
 4. **[Backend]** If not found: returns 404. Frontend shows “Profile not found”.
 5. **[MinIO]** Presigns `avatar_url` and `banner_url`.
 6. **[Database]** Counts followers and following. Fetches pinned models (artist). Fetches recent posts (artist/provider). Fetches average rating and review count.
 7. **[Backend]** Compiles profile response. Returns 200 OK.
 8. **[Frontend]** Renders: avatar, banner, bio, social links, follower/following counts, model grid (if artist), recent posts (if artist/provider), rating and review count.
 9. **[User or Guest]** If authenticated and viewing another’s profile:
 - Shows Follow/Unfollow button.
 - Can navigate to Rate or Report actions.
-

47.4 Decision Points Summary

Decision	Path/Condition	Outcome
User found?	No	Return 404, show “Profile not found”
Authenticated + viewing other profile?	Yes	Show Follow/Unfollow, Rate, Report actions

47.5 Key Implementation Details

- **Public Profile:** The profile page is publicly accessible — even unauthenticated guests can view an artist’s or provider’s public content.
- **UCD Relationship:** “View user profile” includes `<<extend>>` connections to “Follow/unfollow creator” and “Moderate content & user”, exposing these actions contextually from the profile view.